

Bayesian Quadrature

Gaussian Processes for Integration

Suggested Citation: Maren Mahsereci and Toni Karvonen (2026), “Bayesian Quadrature”, *Foundations and Trends in Machine Learning*: Vol. 19, No. 2–3, pp 121–240. DOI: [10.1108/FTMAL-06-2026-0132](https://doi.org/10.1108/FTMAL-06-2026-0132).

Maren Mahsereci

Yahoo Research

maren.mahsereci@yahooinc.com

Toni Karvonen

School of Engineering Sciences

Lappeenranta–Lahti University of Technology LUT

toni.karvonen@lut.fi

Contents

1	Introduction	122
1.1	Contents	125
1.2	Notational conventions	128
2	Bayesian quadrature	129
2.1	Gaussian processes	130
2.2	Construction of Bayesian quadrature	132
2.3	Connections to approximation theory	137
2.4	Fundamental theory for Bayesian quadrature	144
2.5	Generalisations	146
3	Taxonomy of Bayesian quadrature	149
3.1	Model	151
3.2	Inference	152
3.3	Sampling (node selection)	155
3.4	Other terms	165
3.5	Example	167
4	Practical issues	171
4.1	Kernel means and variances	171
4.2	Sampling and node selection	172
4.3	Kernel parameter estimation	180

4.4	Numerical ill-conditioning	182
4.5	Prior mean estimation	184
4.6	Main limitations	185
5	Theoretical guarantees	189
5.1	Guarantees for generic nodes	190
5.2	Guarantees for sequential Bayesian quadrature	197
5.3	Other guarantees	198
6	Empirical illustrations	199
6.1	Experimental setup	200
6.2	Results	207
6.3	Further experimental details	215
7	Proofs for Chapter 5	216
7.1	Reproducing kernel Hilbert spaces	216
7.2	Sobolev spaces	218
7.3	Proofs for Section 5.1	218
7.4	Proofs for Section 5.2	225
	References	227

Bayesian Quadrature

Maren Mahsereci¹ and Toni Karvonen²

¹ *Yahoo Research; maren.mahsereci@yahooinc.com*

² *School of Engineering Sciences, Lappeenranta–Lahti University of Technology LUT; toni.karvonen@lut.fi*

ABSTRACT

Bayesian quadrature is a probabilistic, model-based approach to numerical integration, the estimation of intractable integrals, or expectations. Although Bayesian quadrature was popularised already in the 1980s, no systematic and comprehensive treatment has been published. The purpose of this survey is to fill this gap. We review the mathematical foundations of Bayesian quadrature from different points of view; present a systematic taxonomy for classifying different Bayesian quadrature methods along the three axes of modelling, inference, and sampling; collect general theoretical guarantees; and provide a controlled numerical study that explores and illustrates the effect of different choices along the axes of the taxonomy. We also provide a realistic assessment of practical challenges and limitations to application of Bayesian quadrature methods and include an up-to-date and nearly exhaustive bibliography that covers not only machine learning and statistics literature but all areas of mathematics and engineering in which Bayesian quadrature or equivalent methods have seen use.

1

Introduction

Numerical computation of integrals, often representing expectations or normalization constants, is a pervasive practical problem throughout machine learning, statistics, scientific computing, and engineering. In the form studied in this survey, numerical integration consists in approximating the definite integral

$$I_P(f) = \int_D f(\mathbf{x}) \, dP(\mathbf{x})$$

of a real-valued integrand function f with respect to a probability measure P on a set $D \subseteq \mathbb{R}^d$. The approximation typically takes the form of a *quadrature rule* $\sum_{i=1}^N w_i f(\mathbf{x}_i)$, which is a weighted sum of integrand evaluations at some nodes $\mathbf{x}_i \in D$. Numerical integration can be done on arbitrary non-Euclidean domains and can also incorporate information other than point evaluations, generalisations that we discuss in Section 2.5. Over the past two centuries a vast number of numerical integration techniques have been developed from different starting points and for different types of integration problems. The most popular of these are classical *Gaussian quadratures* that are constructed using polynomial exactness criteria and admit an elegant mathematical theory (Gautschi, 2004); easy-to-use and widely applicable *Monte*

Carlo integration based on random sampling that is, no doubt, the most popular approach to approximating an integral (Caflish, 1998); and *quasi-Monte Carlo methods* that use low-discrepancy sequences and can compute high-dimensional integrals effectively (Dick *et al.*, 2013). Other approaches include sparse grid methods, Clenshaw–Curtis quadrature, the trapezoidal rule, and the topic of this survey, *Bayesian quadrature*.

Although its history can be traced further back, Bayesian quadrature was not popularized until the late 1980s by Persi Diaconis and Anthony O’Hagan (Diaconis, 1988; O’Hagan, 1991). Since then, there has been a steady stream of interest and contributions from machine learning and statistics communities (e.g., Kennedy, 1998; Rasmussen and Ghahramani, 2002; Osborne *et al.*, 2012a; Briol *et al.*, 2019) and, more recently, from the point of view of numerical analysis and approximation theory (Jagadeeswaran and Hickernell, 2019; Kanagawa *et al.*, 2020; Santin *et al.*, 2022). Bayesian quadrature falls within *probabilistic numerics* (Hennig *et al.*, 2022), its defining characteristic being the interpretation of numerical integration as a statistical inference problem to which the Bayesian paradigm and methods can be brought to bear.

In Bayesian quadrature, a stochastic process prior placed on the integrand is conditioned on the “data” consisting of integrand evaluations. The mean of the resulting posterior distribution provides a point estimate for the integral, while the spread of the posterior quantifies uncertainty (see Figure 1.1). That *explicit prior information*, such as smoothness or structural properties, is easy to encode both conceptually and in practice by selection of an appropriate prior for the integrand distinguishes Bayesian quadrature from other numerical integration techniques. Priors that are “correct” or “good” result in fast rates of convergence and reliable quantification of uncertainty (see Figure 1.2). Non-Bayesian integration methods too encode various types of prior information, but typically in a non-systematic and implicit manner. For example, Gaussian quadratures implicitly assume that the integrand is well approximated by polynomials; the trapezoidal rule that a sum of trapezoids approximates the integral; and Monte Carlo encodes no assumptions whatsoever besides square-integrability. While quasi-Monte Carlo methods use node placement to encode certain assumptions, Bayesian quadrature can couple *any nodes with any prior*

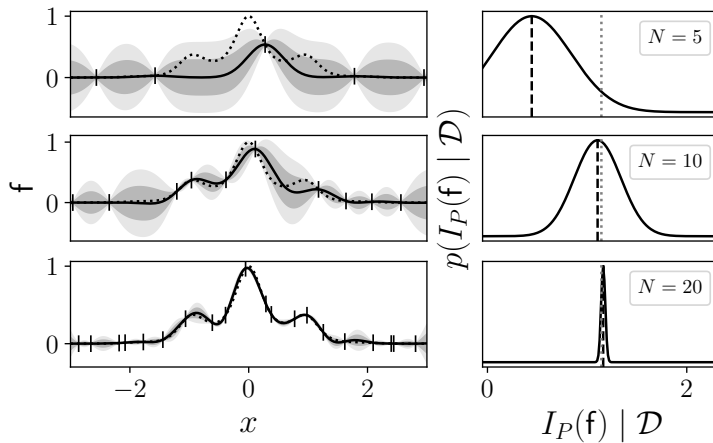


Figure 1.1: A sketch of Bayesian quadrature: A distribution over a function f gives rise to a distribution over the integral $I_P(f) = \int_D f(x) dP(x)$.

information, making it extremely versatile. That any nodes proposed in the vast literature on numerical integration can be used in Bayesian quadrature is one of its great advantages. Another distinguishing feature is the *statistically principled uncertainty quantification* that Bayesian quadrature provides: the spread of the posterior tells one how much to trust the integral estimate. Note that this uncertainty quantification arises from the prior and is wholly distinct in character from the sample variance of Monte Carlo.

Much methodology and theory has been developed for Bayesian quadrature over the years (worthy mentions include Xi *et al.* 2018; Chai and Garnett 2019; and Kanagawa *et al.* 2020), and applications have ranged from computer graphics (Marques *et al.*, 2015) and engineering (Kumar *et al.*, 2008; Dang *et al.*, 2021) to cardiac and tsunami modelling (Oates *et al.*, 2017b; Li *et al.*, 2023). Despite a wealth of such contributions, no comprehensive treatment of Bayesian quadrature as a whole has ever been published. Perhaps partly for this reason, confusion continues to plague the field. For example, “Bayesian quadrature” is often used to refer exclusively to a single *method* (or some collection of methods) to tackle integration problems of a very specific form. Although comparisons of Bayesian quadrature methods to other numerical

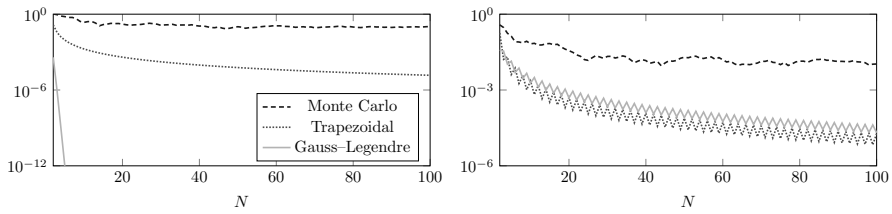


Figure 1.2: Errors when the smooth function $f_1(x) = \exp(x)$ (left) and non-smooth function $f_2(x) = \exp(-|x - \frac{1}{2}|)$ (right) are integrated on the interval $[0, 1]$ with Monte Carlo, the trapezoidal rule, and the Gauss–Legendre quadrature. Because f_1 has a Taylor series that converges fast, Gauss–Legendre reaches machine precision almost immediately as it assumes that f_1 resembles a polynomial. The only assumption the trapezoidal rule makes is that the area under the graph of f_1 can be approximated by trapezoids. Monte Carlo assumes essentially nothing and, as a result, converges very slowly. The function f_2 does not resemble a polynomial and so the assumptions Gauss–Legendre do not benefit it. *Bayesian quadrature makes prior assumptions such as these explicit and puts them on a systematic probabilistic footing.*

integration techniques abound in the literature, no systematic empirical study in a controlled environment on the effect of the multitude of choices on modelling, inference, and sampling that go into designing a Bayesian quadrature method has been undertaken. This is a pity, for the richness of these choices is one of the main attractions (and often a great source of frustration in application) of Bayesian quadrature. The purpose of this survey is to fill this gap and provide a comprehensive survey of Bayesian quadrature that covers both the theory and practice of Bayesian quadrature.

1.1 Contents

There are six main elements that make up the contents of this survey.

I — Chapter 2: Foundations. This chapter contains a foundational introduction to Bayesian quadrature. In particular, the chapter contains what appears to be the first attempt at properly defining Bayesian quadrature (see Definition 2.1) in a way that encompasses the modern usage of the term in machine learning literature. In our parlance a Bayesian quadrature refers to numerical integration based on *Gaussian*

process modelling. That is, a Bayesian quadrature method is any numerical integration method that computes or approximates a posterior distribution for the integral $I_P(f)$ of a potentially transformed Gaussian process f (i.e., $f = \varphi \circ g$ for a mapping φ and a Gaussian process g). The definition gives rise to a natural division of Bayesian quadrature methods in (a) *conjugate methods* for which f is a Gaussian process (i.e., φ is affine) and (b) *non-conjugate methods* for which f is not a Gaussian process (i.e., φ is non-affine). This anticipates the development of more detailed taxonomy in Chapter 3. We also discuss how the theory of reproducing kernel Hilbert spaces can be used to interpret Bayesian quadrature, an interpretation which underlies most theoretical analysis, and provide high-level theory that is independent of the underlying Gaussian process model.

II — Chapter 3: Taxonomy. This chapter contains an extensive taxonomy for Bayesian quadrature that provides a well-defined framework for classifying and categorizing concrete algorithms and implementations. Inspiration for the taxonomy comes from the literature on global optimization (Jones, 2001). We classify Bayesian quadrature methods along three main axes based on (i) what kind of *model* they employ, (ii) how they approach *inference*, and (iii) how they select, or *sample*, the nodes $\mathbf{x}_i$ at which the integrand is evaluated. Figure 3.1 contains an overview of the taxonomy and its most important labels. The taxonomy illustrates that Bayesian quadrature is an umbrella term and that there is a great variety of different Bayesian quadrature *methods* that differ significantly from one another.

III — Chapter 4: Practice. Like all statistical and numerical methods, Bayesian quadrature methods are faced with practical problems that limit their applicability. For Bayesian quadrature, the dominant problems are those of computational cost and computation of kernel means, as one needs to solve linear systems of N equations and compute integrals that involve the covariance function of the Gaussian process model. Although every practitioner of Bayesian quadrature is inevitably confronted by these problems, they are rarely emphasised or treated in a systematic manner. Chapter 4 reviews what we consider the most

important practical problems that a practitioner needs to solve in order to implement a Bayesian quadrature method and offer some solutions. One should recognise that Bayesian quadrature can never hope to be as flexible or general as, for example, Monte Carlo integration. This is understandable, for Bayesian quadrature is based on principled statistical modelling, which must always have a cost.

IV — Chapter 5: Theoretical guarantees. Various theoretical convergence guarantees, given in terms of upper bounds on the integration error under certain assumptions, have appeared in the literature (e.g., Briol *et al.*, 2019; Kanagawa *et al.*, 2020; Wynne *et al.*, 2021). In Chapter 5, we provide somewhat more practical versions of these guarantees by (i) making the dependency of the bounds explicit on those parameters of the covariance function that are commonly estimated from the data and by (ii) including the effect regularisation via a nugget term has. The proofs, which appear in Chapter 7, are based on results in the scattered data approximation literature and do not materially differ from those of the existing results.

V — Chapter 6: Empirical illustrations. Here we put the taxonomy and theory from Chapters 3 and 5 in action by conducting extensive and controlled numerical experiments along the three main axes of the taxonomy. That is, we systematically study how the model, inference methodology, and sampling scheme affect the accuracy and calibration of a Bayesian quadrature method, as well as how the different choices interact. In the experiments we use a collection of test integrands whose properties, such as differentiability, are well-understood. The experiments validate some of the theoretical guarantees. While almost every article on Bayesian quadrature contains an empirical study, these typically focus on comparing the new method to other candidates, such as Monte Carlo or quasi-Monte Carlo, in realistic—and hence opaque—integration problems that do not easily lend themselves to untangling the reasons behind one method outperforming the other. The purpose of this chapter is not to demonstrate the superiority of Bayesian quadrature in general or any particular method to alternative

numerical methods but to highlight the effect of different approaches and choices *within* Bayesian quadrature.

VI — Comprehensive bibliography. We do not conduct a systematic literature review. However, as a survey article such as this is naturally equipped with a long bibliography, we could not resist taking this to extremes and including every reference, no matter how obscure, on Bayesian quadrature or equivalent approaches known to us in machine learning, statistics, engineering, and mathematics literature. Consequently, in most cases we make little to no effort at providing more than a modicum of context for the references.

1.2 Notational conventions

The set of non-negative integers is denoted $\mathbb{N}$. We strive to follow the following type conventions: objects in *italics* are scalars or scalar-valued functions (x or f); **sans serif** is used to indicate random variables and stochastic processes ($\mathbf{f}$); **boldface** objects are vectors (if lowercase; $\mathbf{x}$) or matrices (if uppercase; $\mathbf{K}$); and **boldface sans serif** stands for a random vector ($\mathbf{f}$). In Section 7.3.2, we use $\lesssim$ to denote asymptotic inequality. This notation is equivalent to big O notation.

2

Bayesian quadrature

This chapter is an introduction to Bayesian quadrature, which we define (see Definition 2.1) as any method for computing or approximating the integral of a potentially transformed Gaussian process f conditioned on a number of evaluations of the integrand f . We further divide Bayesian quadrature methods to

- *conjugate methods* when the posterior over the integral is normally distributed and
- *non-conjugate methods* when the posterior is non-normal.

A detailed classification of the existing Bayesian quadrature methods is given in Chapter 3. To precede the construction of Bayesian quadrature in Section 2.2 we include a short review of Gaussian processes in Section 2.1. Moreover, in Section 2.3 we review the equivalence between conjugate Bayesian quadrature and worst-case optimal quadrature rules in reproducing kernel Hilbert spaces. This equivalence is crucial for deriving the convergence guarantees, as well as many other theoretical results, that we present in Chapter 5. Throughout this chapter it is assumed that both the set of nodes X at which the integrand is to be evaluated and the transformed Gaussian process f (in particular, its

covariance function k) used to model the integrand are fixed; different ways to select them are reviewed in Chapters 3 and 4.

2.1 Gaussian processes

Rasmussen and Williams (2006) provide a standard introduction to Gaussian processes for machine learning. For more theoretical expositions, see Bogachev (1998) and Berlinet and Thomas-Agnan (2004). Let $k: D \times D \rightarrow \mathbb{R}$ be a symmetric positive-semidefinite *covariance function* (or *kernel*) on $D \subseteq \mathbb{R}^d$, which is to say that the *kernel Gram matrix*

$$\mathbf{K}_{XX} = (k(\mathbf{x}_i, \mathbf{x}_j))_{i,j=1}^N \in \mathbb{R}^{N \times N}$$

is positive-semidefinite for any $N \in \mathbb{N}$ and any $X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subseteq D$. Throughout this survey we tacitly assume that X are such that the Gram matrix is positive-definite and hence invertible. Most covariance function that we consider are *positive-definite*, in that their Gram matrix is positive-definite whenever the points X are pairwise distinct.

Let $m: D \rightarrow \mathbb{R}$ be a function. A Gaussian process $\mathbf{g}$ with mean m and covariance function k is denoted

$$\mathbf{g} \sim \text{GP}(m, k).$$

Suppose that one wishes to do regression and is given a dataset

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$$

consisting of some noiseless observations $\mathbf{y} = (y_1, \dots, y_N) \in \mathbb{R}^N$ at pairwise distinct nodes $X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subseteq D$. The mean and covariance of the conditional Gaussian process, $\mathbf{g} \mid \mathcal{D}$, are

$$m_{\mathcal{D}}(\mathbf{x}) = \mathbb{E}[\mathbf{g}(\mathbf{x}) \mid \mathcal{D}] = m(\mathbf{x}) + \mathbf{k}_X(\mathbf{x})^\top \mathbf{K}_{XX}^{-1}(\mathbf{y} - \mathbf{m}_X) \quad (2.1)$$

and

$$\begin{aligned} k_{\mathcal{D}}(\mathbf{x}, \mathbf{x}') &= \text{Cov}[\mathbf{g}(\mathbf{x}), \mathbf{g}(\mathbf{x}') \mid \mathcal{D}] \\ &= k(\mathbf{x}, \mathbf{x}') - \mathbf{k}_X(\mathbf{x})^\top \mathbf{K}_{XX}^{-1} \mathbf{k}_X(\mathbf{x}'), \end{aligned} \quad (2.2)$$

where $\mathbf{m}_X = (m(\mathbf{x}_1), \dots, m(\mathbf{x}_N)) \in \mathbb{R}^N$ and $\mathbf{k}_X: D \rightarrow \mathbb{R}^N$ is a vector-valued function defined as

$$\mathbf{k}_X(\mathbf{x}) = (k(\mathbf{x}, \mathbf{x}_1), \dots, k(\mathbf{x}, \mathbf{x}_N)) \in \mathbb{R}^N.$$

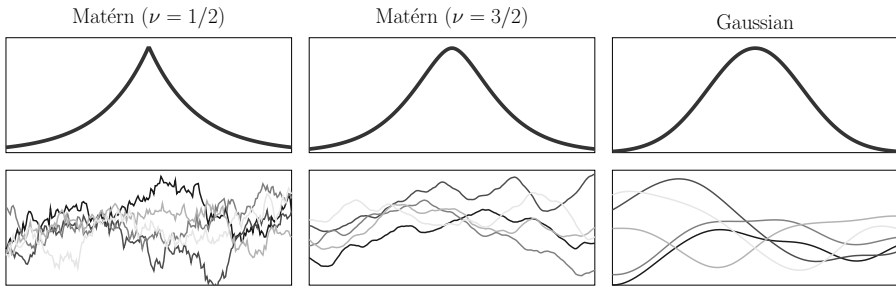


Figure 2.1: *Top:* Three isotropic covariance functions $k(\cdot, 0)$. *Bottom:* Samples from the corresponding GPs. Note how smooth covariance functions yield smooth samples.

A covariance function is *stationary* if it only depends on the difference of the inputs and *isotropic* if it depends only on their distance. That is, a covariance k is stationary if there is a function $\Phi_k: D \rightarrow \mathbb{R}$ such that $k(\mathbf{x}, \mathbf{x}') = \Phi_k(\mathbf{x} - \mathbf{x}')$ for all $\mathbf{x}, \mathbf{x}' \in D$ and isotropic if $\Phi_k: [0, \infty) \rightarrow \mathbb{R}$ and $k(\mathbf{x}, \mathbf{x}') = \Phi_k(\|\mathbf{x} - \mathbf{x}'\|)$. Commonly used covariance functions on any $D \subseteq \mathbb{R}^d$ include the *square exponential*

$$k(\mathbf{x}, \mathbf{x}') = \sigma^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right) \quad (2.3)$$

and covariance functions of the *Matérn class*

$$k(\mathbf{x}, \mathbf{x}') = \sigma^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu} \|\mathbf{x} - \mathbf{x}'\|}{\ell}\right)^\nu K_\nu\left(\frac{\sqrt{2\nu} \|\mathbf{x} - \mathbf{x}'\|}{\ell}\right), \quad (2.4)$$

where Γ denotes the gamma function and K_ν the modified Bessel function of the second kind of order $\nu > 0$. Both the square exponential and all Matérn covariance functions are isotropic. *Product Matérn* covariance functions, which are defined as dimension-wise products of one-dimensional Matérn kernels (and are therefore stationary),

$$k(\mathbf{x}, \mathbf{x}') = \sigma^2 \prod_{i=1}^d \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu} |x_i - x'_i|}{\ell}\right)^\nu K_\nu\left(\frac{\sqrt{2\nu} |x_i - x'_i|}{\ell}\right), \quad (2.5)$$

are also often used. Note that the square exponential kernel (2.3) is the product of one-dimensional square-exponential kernels because

$\|\mathbf{x}\|^2 = x_1^2 + \dots + x_d^2$.¹ The *Brownian motion* covariance function

$$k(x, x') = \min\{x, x'\}, \quad (2.6)$$

which is defined on $D = [0, T] \subseteq \mathbb{R}$ for any $T > 0$ is important due to its connections to spline interpolation and the trapezoidal rule. The Brownian motion kernel is not stationary. Figure 1.1 shows three different covariance functions and GPs. Although most covariance functions used in Bayesian quadrature are stationary, there are several exceptions besides the Brownian motion kernel. For example, Fitzsimons *et al.* (2017) use a piecewise constant covariance function.

2.2 Construction of Bayesian quadrature

Let $\varphi: \mathbb{R} \rightarrow \mathbb{R}$ be a function. Bayesian quadrature methods proceed by taking a Gaussian process prior $\mathbf{g} \sim \text{GP}(m, k)$ with a prior mean m and covariance function k and defining a prior $\mathbf{f}$ for the integrand f by mapping $\mathbf{g}$ through φ :

$$\mathbf{f} = \varphi \circ \mathbf{g}. \quad (2.7)$$

Let the dataset consist of noiseless evaluations of the integrand f at nodes $\mathbf{x}_1, \dots, \mathbf{x}_N$:

$$\mathcal{D} = \{(\mathbf{x}_1, f(\mathbf{x}_1)), \dots, (\mathbf{x}_N, f(\mathbf{x}_N))\}.$$

We call any method that computes or approximates the posterior distribution over $I_P(\mathbf{f}) \mid \mathcal{D}$ a Bayesian quadrature (BQ) method.

Definition 2.1 (Bayesian quadrature). Let $\mathbf{g}$ be a Gaussian process and $\varphi: \mathbb{R} \rightarrow \mathbb{R}$ a function. Any method that computes or approximates the posterior distribution

$$I_P(\mathbf{f}) \mid \mathcal{D} = \int_D \mathbf{f}(\mathbf{x}) \, dP(\mathbf{x}) \mid \mathcal{D} \quad (2.8)$$

over the integral of the process $\mathbf{f} = \varphi \circ \mathbf{g}$ is a *Bayesian quadrature method*².

¹In fact, the square exponential covariance function is the *only* isotropic covariance function that is the product of its one-dimensional versions (Stein, 1999, pp. 55-56).

²The term *Bayesian quadrature* was popularised by O'Hagan (1991). The earliest

The integral posterior mean $\mathbb{E}[I_P(\mathbf{f}) \mid \mathcal{D}]$ or its approximation is used as a point estimate of the true integral $I_P(f)$. The integral posterior variance, $\text{Var}[I_P(\mathbf{f}) \mid \mathcal{D}]$, is typically used to summarise the spread of $I_P(\mathbf{f}) \mid \mathcal{D}$. We shall always tacitly assume that the process $\mathbf{f}$, the domain D , and the probability measure P are such that the posterior distribution in (2.8) is well-defined and has finite mean and variance. If the function φ is affine, it suffices that m and $k(\cdot, \mathbf{x})$ be integrable with respect to P for every $\mathbf{x} \in D$ and $\int_D \sqrt{k(\mathbf{x}, \mathbf{x})} dP(\mathbf{x}) < \infty$. For example, the integral is finite if the covariance function is stationary and P has a bounded density on a bounded D since in that case $k(\mathbf{x}, \mathbf{x}) = \Phi_k(\mathbf{0})$ for all $\mathbf{x} \in D$.

The selection of $\mathbf{g}$ and φ is supposed to be informed about any prior information one may have about the true integrand f . The prior information that is typically encoded includes different structural properties, such as stationarity, non-negativity or periodicity, and smoothness, described by the number of continuous derivatives that one expects f to possess. For example, when $\mathbf{f} = \mathbf{g}$ and the covariance function of $\mathbf{g}$ is a Matérn in (2.4) with smoothness ν , the samples of $\mathbf{f}$ essentially have smoothness ν in a Sobolev sense; see Scheuerer (2010), Steinwart (2019), and Henderson (2024), as well as Kanagawa *et al.* (2018, Corollary 4.15). However, *model misspecification*, to be understood here in a broad sense that f “looks” different than the samples from $\mathbf{f}$, is inevitable in anything more complicated than toy examples. Practical selection of the prior is a general challenge in Bayesian inference and not limited to Bayesian quadrature or inference with Gaussian processes. Prior selection, when simplified to mean the selection of a limited number of kernel parameters, such as σ and ℓ of the square exponential kernel in (2.3), is briefly reviewed in Section 4.3.

occurrence that we have found is in O’Hagan (1988). Diaconis (1988, p. 164) indicates an origin no later than 1985. In the early literature, *Bayes–Hermite quadrature* is used to refer to conjugate BQ when the GP prior has a square exponential covariance and P is a Gaussian distribution (O’Hagan, 1991; Kennedy and O’Hagan, 1996; Kennedy, 2000).

2.2.1 Conjugate Bayesian quadrature (affine φ)

The prior f is a GP if the function φ is affine. In this case Gaussian conjugate inference is possible and we accordingly call the resulting methods *conjugate Bayesian quadrature methods*. This class of methods is sometimes called *standard Bayesian quadrature* (Karvonen, 2019, Sec. 3.2) or *vanilla Bayesian quadrature* (Gessner et al., 2020, Sec. 2.1). Because an affine φ merely shifts and scales the mean and covariance of g , we can set φ to be the identity function without loss of generality. This gives the GP prior

$$f = \text{id} \circ g = g \sim \text{GP}(m, k).$$

Due to the linearity of integration, the posterior for $I_P(f)$ is a normal random variable that is obtained by simply integrating the posterior process $f \mid \mathcal{D} = g \mid \mathcal{D}$:

$$I_P(f) \mid \mathcal{D} = I_P(f \mid \mathcal{D}) = \int_D [f(\mathbf{x}) \mid \mathcal{D}] dP(\mathbf{x}) \sim N(\mu_{\mathcal{D}}, \Sigma_{\mathcal{D}}).$$

The integral mean

$$\begin{aligned} \mu_{\mathcal{D}} &= \mathbb{E}[I_P(f) \mid \mathcal{D}] = \int_D m_{\mathcal{D}}(\mathbf{x}) dP(\mathbf{x}) \\ &= m_P + \mathbf{k}_{PX}^{\top} \mathbf{K}_{XX}^{-1} (\mathbf{f}_X - \mathbf{m}_X), \end{aligned} \quad (2.9)$$

which is the integral of the posterior mean in (2.1), is expressed in terms of the integral $m_P = I_P(m)$ of the prior mean function m , the integrand evaluations $\mathbf{f}_X = (f(\mathbf{x}_1), \dots, f(\mathbf{x}_N)) \in \mathbb{R}^N$, and the evaluations $\mathbf{k}_{PX} = I_P(\mathbf{k}_X) = (k_P(\mathbf{x}_1), \dots, k_P(\mathbf{x}_N)) \in \mathbb{R}^N$ of the *kernel mean embedding*

$$k_P(\mathbf{x}) = \int_D k(\mathbf{x}', \mathbf{x}) dP(\mathbf{x}'). \quad (2.10)$$

The integral variance

$$\begin{aligned} \Sigma_{\mathcal{D}} &= \text{Var}[I_P(f) \mid \mathcal{D}] = \int_D \int_D k_{\mathcal{D}}(\mathbf{x}, \mathbf{x}') dP(\mathbf{x}) dP(\mathbf{x}') \\ &= k_{PP} - \mathbf{k}_{PX}^{\top} \mathbf{K}_{XX}^{-1} \mathbf{k}_{PX}, \end{aligned} \quad (2.11)$$

where

$$k_{PP} = \int_D \int_D k(\mathbf{x}, \mathbf{x}') dP(\mathbf{x}) dP(\mathbf{x}') = \int_D k_P(\mathbf{x}) dP(\mathbf{x}) \quad (2.12)$$

is the *initial variance*, is the integral of the posterior covariance in (2.2). We encapsulate conjugate Bayesian quadrature and its posterior equations in the following definition.

Definition 2.2 (Conjugate Bayesian quadrature). If $\varphi: \mathbb{R} \rightarrow \mathbb{R}$ is affine, so that $\mathbf{f} = \varphi \circ \mathbf{g}$ is a Gaussian process, Bayesian quadrature is *conjugate*. Without a loss of generality we can set $\mathbf{g} \sim \text{GP}(m, k)$ and $\varphi = \text{id}$. Then

$$I_P(\mathbf{f}) \mid \mathcal{D} = I_P(\mathbf{f} \mid \mathcal{D}) = \int_{\mathcal{D}} [\mathbf{f}(\mathbf{x}) \mid \mathcal{D}] dP(\mathbf{x}) \sim \text{N}(\mu_{\mathcal{D}}, \Sigma_{\mathcal{D}}),$$

where $\mu_{\mathcal{D}}$ and $\Sigma_{\mathcal{D}}$ given in (2.9) and (2.11) are the *integral mean* and *variance* of conjugate Bayesian quadrature.

Both the integral mean and variance can be expressed in terms of the weight vector $\mathbf{w} = \mathbf{K}_{XX}^{-1} \mathbf{k}_{PX}$ as

$$\mu_{\mathcal{D}} = m_P + \mathbf{w}^{\top}(\mathbf{f}_X - \mathbf{m}_X) \quad \text{and} \quad \Sigma_{\mathcal{D}} = k_{PP} - \mathbf{w}^{\top} \mathbf{k}_{PX}. \quad (2.13)$$

Some of the weights $\mathbf{w} = (w_1, \dots, w_N)$ can be negative (Huszár and Duvenaud, 2012; Karvonen *et al.*, 2019a). For a zero-mean prior the mean is simply $\mu_{\mathcal{D}} = \mathbf{w}^{\top} \mathbf{f}_X$, which means that it is a *quadrature rule*, an approximation of the integral in the form of a weighted sum of integrand evaluations. Although it is customary to use the term *cubature rule* to describe such an approximation when $d > 1$ (see Cools, 1997, for some discussion), this distinction is not consistently made in the literature on Bayesian quadrature. For articles where *Bayesian cubature* is used, see Karvonen *et al.* (2018), Jagadeeswaran and Hickernell (2019), Jagadeeswaran and Hickernell (2022), and Fisher *et al.* (2020).

2.2.2 Non-conjugate Bayesian quadrature (non-affine φ)

When φ is not affine, we speak of non-conjugate Bayesian quadrature.

Definition 2.3 (Non-conjugate Bayesian quadrature). If $\varphi: \mathbb{R} \rightarrow \mathbb{R}$ is non-affine and $\mathbf{f} = \varphi \circ \mathbf{g}$ is non-Gaussian³, Bayesian quadrature is *non-conjugate*. No general expression exists for the posterior $I_P(\mathbf{f}) \mid \mathcal{D}$.

³See Mase (1977) and Khatri and Mukerjee (1987) for some theorems on preservation of Gaussianity under transformations.

Non-conjugate Bayesian quadrature is usually motivated by a desire to model the non-negativity of the integrand, a property which cannot be encoded by a GP model (Osborne *et al.*, 2012a; Osborne *et al.*, 2012b; Hamrick and Griffiths, 2013; Gunter *et al.*, 2014). Functions used to produce a non-negative f include

$$\varphi(x) = \alpha + x^2 \quad \text{and} \quad \varphi(x) = \exp(x),$$

where $\alpha > 0$, and a general framework for encoding range constraints is described by Chai and Garnett (2019). See Zhou and Peng (2020) and Hamid *et al.* (2023) for applications to engineering and neural networks. However, when φ is non-affine (as the two functions above are), the prior $f = \varphi \circ g$ is usually not a Gaussian process and the posterior $I_P(f) \mid \mathcal{D}$ is not in general available in closed form, which precludes exact inference. Intractability of the posterior and its moments introduces a significant complication over conjugate Bayesian quadrature, forcing one to approximate the posterior. Gaussian approximations based on linearisation or moment-matching are typically used, but one can employ any posterior approximation available in the literature.

2.2.3 Bayesian probabilistic numerical integration

In the definition of Bayesian quadrature (Theorem 2.1) the integrand is modelled as a potentially non-linearly transformed Gaussian process $f = \varphi \circ g$. Nothing stops one from using *any* suitable stochastic process f . The following definition is adapted from Zhu *et al.* (2020, Sec. 1).

Definition 2.4 (Bayesian probabilistic numerical integration). Let f be a stochastic process. Any method that computes or approximates the posterior distribution

$$I_P(f) \mid \mathcal{D} = \int_D f(\mathbf{x}) \, dP(\mathbf{x}) \Big| \mathcal{D} \quad (2.14)$$

is a *Bayesian probabilistic numerical integration (BPNI) method*.

All Bayesian quadrature methods are BPNI methods. Due to computational tractability issues, few non-Gaussian BPNI methods exist. Student's t -processes, which differ little from Gaussian processes in practice, have been used in numerical integration by O'Hagan (1991, Sec. 2.2)

and Prüher *et al.* (2017). Integration methods based on Bayesian additive regression trees introduced by Zhu *et al.* (2020) appear to be the only non-Gaussian BPNI methods that substantially differ from Bayesian quadrature. There is much room for the development of new BPNI methods, particularly when integrand evaluations constitute the computational bottleneck. One can consider even more general and difficult integration problems in which the distribution P is unknown and accessible only via sampling or black-box evaluation of the density function (Oates *et al.*, 2017b; Llorento Fernández *et al.*, 2020).

2.3 Connections to approximation theory

Given the well known equivalences of Gaussian process regression to optimal approximation in a *reproducing kernel Hilbert space* (RKHS) and to *kernel interpolation* (Kimeldorf and Wahba, 1970; Scheuerer *et al.*, 2013; Kanagawa *et al.*, 2018), it is no surprise that these equivalences extend to conjugate Bayesian quadrature. We briefly describe the equivalences because of their historical value and because much of the theory for Bayesian quadrature, summarised in Sections 2.4 and 5, exploits them. See Briol *et al.* (2019, Sec. 2) and Kanagawa *et al.* (2020, Sec. 2) for earlier reviews on the topic.

2.3.1 Optimal integration in a reproducing kernel Hilbert space

Let $\mathcal{H}(k)$ denote the RKHS of k . This is a certain Hilbert space consisting of real-valued functions defined on D in which the kernel has the following *reproducing property*: $\langle g, k(\cdot, \mathbf{x}) \rangle_{\mathcal{H}(k)} = g(\mathbf{x})$ for every $g \in \mathcal{H}(k)$ and $\mathbf{x} \in D$. For example, the RKHS of a Matérn kernel (2.4) of order ν is, up to an equivalent norm, the Sobolev space $H^{\nu+d/2}(\mathbb{R}^d)$, a fact that is discussed in more detail in Section 5.1.1. We refer to Berlinet and Thomas-Agnan (2004), Steinwart and Christmann (2008), and Paulsen and Raghupathi (2016) for the theory of RKHSs. Section 7.1 contains a very concise review.

The *worst-case integration error* of any weights $\mathbf{v} \in \mathbb{R}^N$ and points

$X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subset D$ in $\mathcal{H}(k)$ is defined as

$$e_k(\mathbf{v}, X) = \sup_{\|g\|_{\mathcal{H}(k)} \leq 1} \left| \int_D g(\mathbf{x}) dP(\mathbf{x}) - \sum_{i=1}^N v_i g(\mathbf{x}_i) \right|. \quad (2.15)$$

It follows from the reproducing property that the worst-case error can be expressed in terms of the kernel.

Lemma 2.5. Suppose that $k(\cdot, \mathbf{x})$ is measurable with respect to P for every $\mathbf{x} \in D$ and that $\int_D \sqrt{k(\mathbf{x}, \mathbf{x})} dP(\mathbf{x}) < \infty$. Then

$$\begin{aligned} e_k(\mathbf{v}, X) &= \|k_P - \sum_{i=1}^N v_i k(\cdot, \mathbf{x}_i)\|_{\mathcal{H}(k)} \\ &= \sqrt{k_{PP} - 2\mathbf{v}^\top \mathbf{k}_{PX} + \mathbf{v}^\top \mathbf{K}_{XX} \mathbf{v}}. \end{aligned} \quad (2.16)$$

Proof. The measurability assumption implies that all $g \in \mathcal{H}(k)$ are measurable (Steinwart and Christmann, 2008, Lem. 4.24). Let $g \in \mathcal{H}(k)$. By the reproducing property and the Cauchy–Schwarz inequality,

$$\begin{aligned} \left| \int_D g(\mathbf{x}) dP(\mathbf{x}) \right| &= \left| \int_D \langle g, k(\cdot, \mathbf{x}) \rangle_{\mathcal{H}(k)} dP(\mathbf{x}) \right| \\ &\leq \|g\|_{\mathcal{H}(k)} \int_D \sqrt{k(\mathbf{x}, \mathbf{x})} dP(\mathbf{x}), \end{aligned}$$

where we used $\|k(\cdot, \mathbf{x})\|_{\mathcal{H}(k)}^2 = \langle k(\cdot, \mathbf{x}), k(\cdot, \mathbf{x}) \rangle_{\mathcal{H}(k)} = k(\mathbf{x}, \mathbf{x})$. It follows that every $g \in \mathcal{H}(k)$ is integrable with respect to P . As $k(\cdot, \mathbf{x}) \in \mathcal{H}(k)$ for every $\mathbf{x} \in D$, the mean embedding given by $k_P(\mathbf{x}) = \int_D k(\mathbf{x}', \mathbf{x}) dP(\mathbf{x}')$ is well-defined. From the Riesz representation theorem it follows that $k_P \in \mathcal{H}(k)$ and $I_P(g) = \langle g, k_P \rangle_{\mathcal{H}(k)}$ for every $g \in \mathcal{H}(k)$; see Section 7.1 for details. Therefore the RKHS norm in (2.16) is well-defined.

To prove the equality, use the Cauchy–Schwarz inequality to obtain

$$\begin{aligned} e_k(\mathbf{v}, X) &= \sup_{\|g\|_{\mathcal{H}(k)} \leq 1} |\langle g, k_P \rangle_{\mathcal{H}(k)} - \langle g, \sum_{i=1}^N v_i k(\cdot, \mathbf{x}_i) \rangle_{\mathcal{H}(k)}| \\ &\leq \|k_P - \sum_{i=1}^N v_i k(\cdot, \mathbf{x}_i)\|_{\mathcal{H}(k)}, \end{aligned}$$

which equals $\sqrt{k_{PP} - 2\mathbf{v}^\top \mathbf{k}_{PX} + \mathbf{v}^\top \mathbf{K}_{XX} \mathbf{v}}$ by the reproducing property and $I_P(g) = \langle g, k_P \rangle_{\mathcal{H}(k)}$. To verify that the upper bound is an equality, select $g_0 = (k_P - \sum_{i=1}^N v_i k(\cdot, \mathbf{x}_i)) / \|k_P - \sum_{i=1}^N v_i k(\cdot, \mathbf{x}_i)\|_{\mathcal{H}(k)}$. As this function has unit norm, $e_k(\mathbf{v}, X) \geq |\int_D g_0(\mathbf{x}) dP(\mathbf{x}) - \sum_{i=1}^N v_i g_0(\mathbf{x}_i)|$. A short and straightforward computation reveals that the lower bound equals $\|k_P - \sum_{i=1}^N v_i k(\cdot, \mathbf{x}_i)\|_{\mathcal{H}(k)}$. $\square$

See, for example, Novak and Woźniakowski (2010, Sec. 10.2) or Oettershagen (2017, Cor. 3.6) for generalisations of Lemma 2.5. The worst-case error measures the largest possible integration error by a quadrature rule of the form $\sum_{i=1}^N v_i g(\mathbf{x}_i)$ for functions $g \in \mathcal{H}(k)$ normalised in the RKHS norm. If the function f the quadrature rule is used to integrate is *not* an element of $\mathcal{H}(k)$, the worst-case error is still well-defined, as is evident from (2.16). However, in this case the worst-case error needs to bear no relation to the integration error $|\int_D f(\mathbf{x}) dP(\mathbf{x}) - \sum_{i=1}^N v_i f(\mathbf{x}_i)|$. As squared worst-case error is quadratic in $\mathbf{v}$, it is easy to see that, for fixed points, Equation (2.16) is minimised by the weights $\mathbf{v} = \mathbf{w} = \mathbf{K}_{XX}^{-1} \mathbf{k}_{PX}$, which are precisely the weights that define the integral mean and variance of conjugate Bayesian quadrature in (2.13). Moreover, for these weights the worst-case error simplifies to $e_k(\mathbf{w}, X) = \Sigma_D^{1/2} = \sqrt{k_{PP} - \mathbf{w}^\top \mathbf{k}_{PX}}$. The posterior mean of a conjugate Bayesian quadrature method is thus given by weights which minimise the worst-case error in the RKHS of the kernel k and the posterior variance equals the corresponding minimal squared worst-case error. These facts are summarised in the following proposition.

Proposition 2.6. Let $f \sim \text{GP}(m, k)$. Then $I_P(f) \mid \mathcal{D} = \text{N}(\mu_{\mathcal{D}}, \Sigma_{\mathcal{D}})$ for $\mu_{\mathcal{D}} = m_P + \mathbf{w}^\top (\mathbf{f}_X - \mathbf{m}_X)$ and $\Sigma_{\mathcal{D}} = k_{PP} - \mathbf{w}^\top \mathbf{k}_{PX} = \min_{\mathbf{v} \in \mathbb{R}^N} e_k(\mathbf{v}, X)^2$,

where

$$\mathbf{w} = \mathbf{K}_{XX}^{-1} \mathbf{k}_{PX} = \arg \min_{\mathbf{v} \in \mathbb{R}^N} e_k(\mathbf{v}, X). \quad (2.17)$$

If the prior mean m is zero and the integrand f is an element of $\mathcal{H}(k)$, we have the convenient error estimate

$$|I_P(f) - \mu_{\mathcal{D}}| = \langle f, k_P - \sum_{i=1}^N w_i k(\cdot, \mathbf{x}_i) \rangle_{\mathcal{H}(k)} \leq \|f\|_{\mathcal{H}(k)} \Sigma_D^{1/2}, \quad (2.18)$$

which follows from $\langle f, k_P \rangle_{\mathcal{H}(k)} = I_P(f)$ (see the proof of Lemma 2.5, Section 7.1, or Muandet *et al.*, 2017, Lem. 3.1) and the Cauchy–Schwarz inequality. This bound allows one to use the magnitude of $\Sigma_{\mathcal{D}}$ to assess the quality of $\mu_{\mathcal{D}}$ as a point estimator of the integral; its use is the reason behind the assumption $f \in \mathcal{H}(k)$ in several theorems in Section 5. If $f \notin \mathcal{H}(k)$, the bound (2.18) is not valid because the norm $\|f\|_{\mathcal{H}(k)}$ does not exist.

The construction of general worst-case optimal quadrature rules in RKHSs goes back at least to the work of Larkin in the 1970s (Larkin, 1970; Larkin, 1972; Larkin, 1974), while the related average-case framework was developed by Suldin (1959) and Suldin (1960).⁴ For early analyses, mostly in RKHSs of analytic functions, of these quadratures we refer to Richter (1970), Richter-Dyn (1971b), and Richter-Dyn (1971a). See also Traub *et al.* (1988, Sec. 2.3 in Ch. 5), Oettershagen (2017), Ehler *et al.* (2019), Gavrilov (1998), and Gavrilov (2007). A different tradition may be found in the quasi-Monte Carlo literature (Dick *et al.*, 2013), where spaces of periodic functions are often considered. The optimal weights in these spaces can be occasionally computed in closed form (Kaarnioja *et al.*, 2025). Certain quasi-Monte carlo rules can be interpreted as Bayesian quadratures (Jagadeeswaran and Hickernell, 2019; Jagadeeswaran and Hickernell, 2022; Sorokin *et al.*, 2025).

Whenever making use of the RKHS framework one should keep in mind that the samples from a Gaussian process are *not* in the RKHS of its kernel. Specifically, under fairly non-restrictive assumptions one can show that if $f \sim \text{GP}(m, k)$, then $\mathbb{P}[f \in \mathcal{H}(k)] = 0$; see Driscoll (1973) and Kanagawa *et al.* (2018, Sec. 4). That is, the RKHS is too small to contain the samples. One cannot therefore simply substitute the process f for the RKHS element f in results such as the error estimate (2.18).

2.3.2 Kernel interpolation

A different tradition may be found in the literature on *kernel interpolation* or, when the kernel is isotropic, *radial basis function interpolation* (Wendland, 2005). Given a positive-definite kernel k and evaluations of f at a collection $X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ of pairwise distinct points, the *kernel interpolant* $s_{f,X}$ is the unique function in the span of the *kernel translates* $\{k(\cdot, \mathbf{x}_i)\}_{i=1}^N$ that interpolates f at X , in the sense that $s_{f,X}(\mathbf{x}_i) = f(\mathbf{x}_i)$ for every $i = 1, \dots, N$. From the requirement that the interpolant be in the span of the translates it is easy to deduce

⁴Oates and Sullivan (2019, Sec. 2.2) provide a historical account on the pioneering role played by Suldin and Larkin in the development of probabilistic numerical computation. We note that earlier constructions of optimal quadrature rules that do not utilise RKHSs may be found in Sard (1949) and Nikolsky (1950).

that $s_{f,X}(\mathbf{x}) = \mathbf{k}_X(\mathbf{x})^\top \mathbf{K}_{XX}^{-1} \mathbf{f}_X$. We see that the kernel interpolant coincides with the GP conditional mean (2.1) under a zero-mean prior when $\mathbf{y} = \mathbf{f}_X$. The kernel interpolant may be alternatively defined as the minimum-norm interpolant in the RKHS of k :

$$s_{f,X} = \arg \min_{s \in \mathcal{H}(k)} \{ \|s\|_{\mathcal{H}(k)} : s(\mathbf{x}_i) = f(\mathbf{x}_i) \text{ for } i = 1, \dots, N \}. \quad (2.19)$$

See, for example, Theorem 3.5 in Kanagawa *et al.* (2018). Due to the minimum-norm property kernel interpolants are often called *spline interpolants* (Traub *et al.* 1988, Sec. 5.7; Wahba 1990). As the kernel interpolant $s_{f,X}$ depends on the value of f at a finite point set, it is well-defined whether f is in $\mathcal{H}(k)$ or not. However, the useful inequality $\|s_{f,X}\|_{\mathcal{H}(k)} \leq \|f\|_{\mathcal{H}(k)}$, a consequence of the minimum-norm interpolation property, holds only if $f \in \mathcal{H}(k)$.

Integrating the kernel interpolant yields the integral approximation $I_P(f) \approx I_P(s_{f,X}) = \mathbf{k}_{PX}^\top \mathbf{K}_{XX}^{-1} \mathbf{f}_X$, which is equal to the conjugate BQ integral mean (2.9) in the mean-zero case. Bezhaev (1991) appears to have been the first to suggest this approach. More recent work on integrated kernel interpolants includes Sommariva and Womersley (2005), Sommariva and Vianello (2006b), Sommariva and Vianello (2006a), Punzi *et al.* (2008), Siraj-ul-Islam *et al.* (2012), Fuselier *et al.* (2014), Reeger and Fornberg (2015), Reeger *et al.* (2016), Reeger and Fornberg (2018), Sommariva and Vianello (2021), and Glaubitz and Reeger (2023). Minka (2000, Sec. 6) discusses the interpretation of spline-based quadrature as BQ. The connection to kernel interpolation is important because many estimates for the integration error $|I_P(f) - \mu_{\mathcal{D}}|$ of Bayesian quadrature are derived from L^p -error estimates for kernel interpolation (Kanagawa *et al.*, 2020; Wynne *et al.*, 2021).

2.3.3 Kernel quadrature

Quadrature rules with the weights (2.17) that minimise the worst-case error in an RKHS are often called *kernel quadratures* (Briol *et al.*, 2017; Karvonen and Särkkä, 2018; Karvonen and Särkkä, 2019; Chen *et al.*, 2025). However, this nomenclature is more commonly (and somewhat ambiguously) applied to any quadrature rule that seeks to achieve small worst-case error in an RKHS and does this by making use of the

kernel or objects derived from it (Bach, 2017; Epperly and Moreno, 2023).⁵ For example, Belhadji *et al.* (2019), Belhadji *et al.* (2020), Belhadji (2020), Belhadji (2021), Hayakawa *et al.* (2022), Hayakawa *et al.* (2023a), Hayakawa *et al.* (2023b), Hayakawa (2023), and Roustant *et al.* (2024) use Mercer expansions of k to construct quadrature rules. See also Tanaka (2021), Tsuji *et al.* (2022), and Tsuji and Tanaka (2026).

2.3.4 Bayesian quadrature in disguise

Many existing and seemingly non-Bayesian integration methods are, or can be seen as, Bayesian quadrature methods.⁶ It is a well-known and established fact that the integral mean of conjugate Bayesian quadrature is equivalent to the classical trapezoidal rule if $k(x, x') = \min\{x, x'\}$ is the covariance function of the Brownian motion (Duc-Jacquet, 1973; Diaconis, 1988). Recall that the trapezoidal rule is the integral approximation

$$\int_0^1 f(x) dx \approx \sum_{i=1}^N \frac{f(x_i) + f(x_{i-1})}{2} (x_i - x_{i-1}),$$

where $0 = x_0 < x_1 < \dots < x_{N-1} < x_N = 1$. The geometric interpretation is that the integral is approximated with N trapezoids with base lengths $x_i - x_{i-1}$. Ritter (2000, Sec. 3 in Ch. II) is an accessible source for computations that constitute the proof of the following proposition.

Proposition 2.7. Let P be uniform on $D = [0, 1]$ and $f \sim \text{GP}(0, k)$ with $k(x, x') = \min\{x, x'\}$. If $0 < x_1 < \dots < x_{N-1} < x_N = 1$ and $f(0) = 0$,

⁵Kernel quadrature is occasionally used to refer *any* quadrature rule designed so as to minimize its worst-case error in an RKHS (e.g., Hayakawa, 2023, Sec. 3.4.3). Such unconditional usage of the term is somewhat ambitious: Every Hilbert space of functions that one can construct is an RKHS because such a Hilbert space is an RKHS if and only if the point evaluation functionals $f \mapsto f(\mathbf{x})$ are bounded, the existence of unbounded functionals depends on the axiom of choice (Wright, 1973; Alpay and Mills, 2003). For example, under this definition quasi-Monte Carlo integration would fall under the umbrella of kernel quadrature (see in particular Dick *et al.*, 2013, Sec. 3).

⁶Connections exist beyond mere numerical integration: Snell and Griffiths (2025) connect Bayesian quadrature and conformal prediction.

then

$$\mu_{\mathcal{D}} = \sum_{i=1}^N \frac{f(x_i) + f(x_{i-1})}{2} (x_i - x_{i-1}) \quad \text{and} \quad \Sigma_{\mathcal{D}} = \frac{1}{12} \sum_{i=1}^n (x_i - x_{i-1})^3,$$

where $x_0 = 0$.

We refer to Naslidnyk *et al.* (2025) for more explicit computations of this kind for the Brownian motion covariance. Higher-order versions of the trapezoidal rule and their connection to Bayesian quadrature with the p times integrated Brownian motion covariance

$$k(x, y) = \int_0^1 \frac{(x-t)_+^m (y-t)_+^m}{(m!)^2} dt,$$

where $(x)_+ = \max\{0, x\}$, are discussed in Schober *et al.* (2014) and Karvonen (2019, Sec. 5.5). See also Wahba (1990).

Many classical quadrature rules can be cast as conjugate Bayesian quadrature if the covariance function is selected appropriately (Schober *et al.*, 2014; Karvonen and Särkkä, 2017; Karvonen *et al.*, 2018; Belhadji, 2021). See also Hennig *et al.* (2022, Sec. 11.4). Recall that a quadrature rule defined by weights $\mathbf{v}$ and nodes X is the integral approximation $\mathbf{v}^\top \mathbf{f}_X = \sum_{i=1}^N v_i f(\mathbf{x}_i) \approx \int_D f(\mathbf{x}) dP(\mathbf{x})$. If $m \equiv 0$, the integral mean $\mu_{\mathcal{D}}$ is a quadrature rule with weights $\mathbf{v} = \mathbf{w} = \mathbf{K}_{XX}^{-1} \mathbf{k}_{PX}$.

Proposition 2.8. Let $D \subseteq \mathbb{R}$ and $f \sim \text{GP}(0, k)$ with covariance function $k(x, y) = \sum_{p=0}^{N-1} a_p x^p y^p$, where $a_0, \dots, a_{N-1} \neq 0$. Then $\mu_{\mathcal{D}}$ is the unique quadrature rule that integrates all polynomials of degree at most $N-1$ exactly. That is, $\mu_{\mathcal{D}} = I_P(f)$ whenever f is a polynomial of degree at most $N-1$. Moreover, $\Sigma_{\mathcal{D}} = 0$.

Proof. The kernel Gram matrix can be written as

$$\mathbf{K}_{XX} = \mathbf{V}_X \mathbf{A} \mathbf{V}_X^\top,$$

where $\mathbf{A} = \text{diag}(a_0, \dots, a_{N-1})$ is a diagonal matrix and $\mathbf{V}_X$ the invertible $N \times N$ Vandermonde matrix with elements $(\mathbf{V}_X)_{ij} = x_i^{j-1}$. Additionally, $\mathbf{k}_{PX} = \mathbf{V}_X \mathbf{A} \mathbf{p}$ and $k_{PP} = \mathbf{p}^\top \mathbf{A} \mathbf{p}$, where $\mathbf{p} = (\int_D x^i dP(x))_{i=0}^{N-1}$. By substituting these expressions in (2.9) and (2.11) we obtain

$$\mu_{\mathcal{D}} = (\mathbf{V}_X^{-\top} \mathbf{p})^\top \mathbf{f}_X \quad \text{and} \quad \Sigma_{\mathcal{D}} = 0.$$

By the definitions of V_X and $\mathbf{p}$, the weights $\mathbf{v} = V_X^{-\top} \mathbf{p}$ are the unique quadrature weights such that $\sum_{i=1}^N v_i x_i^p = \int_D x^p dP(x)$ for every $p \in \{0, \dots, N-1\}$. It follows that $\mu_{\mathcal{D}}$ is the unique quadrature rule that integrates all polynomials of degree at most $N-1$ exactly. $\square$

By selecting the nodes correctly one recovers from Proposition 2.8 every Gaussian quadrature rule (nodes are roots of an orthogonal polynomial) and other polynomial quadrature rules, such as the Newton–Cotes rule (nodes are equispaced on an interval). Proposition 2.8 does not provide means to attach uncertainty to classical quadratures because the integral variance is always zero. Another way to recover classical quadratures is by considering the length-scale limit $\ell \rightarrow \infty$ (O’Hagan, 1991; Minka, 2000; Särkkä *et al.*, 2016; Karvonen and Särkkä, 2020). See also Karvonen (2019, Sec. 5.4); Karvonen and Oates (2023, Thm. 2.6); and Barthelmé *et al.* (2023). This too results in zero integral variance.

Estimation of the prior mean (more on this in Section 4.5) via a flat prior can be used to “force” the mean $\mu_{\mathcal{D}}$ to assume almost any desired form (Karvonen *et al.*, 2018, Thm. 2.10) while keeping the variance $\Sigma_{\mathcal{D}}$ positive.

Proposition 2.9. Let $D \subseteq \mathbb{R}^d$ and suppose that $\varphi_1, \dots, \varphi_N$ are any functions such that the matrix $\Phi_X = (\varphi_i(\mathbf{x}_j))_{i,j=1}^N$ is invertible. If $\mathbf{f} \mid \gamma \sim \text{GP}(m_\gamma, k)$ for $m_\gamma(\mathbf{x}) = \sum_{i=1}^q \gamma_i \varphi_i(\mathbf{x})$ and $\gamma \sim \text{N}(\mathbf{0}, \Sigma)$, then

$$\mu_{\mathcal{D}} = \mathbf{v}^\top \mathbf{f}_X \quad \text{and} \quad \Sigma_{\mathcal{D}} = e_k(\mathbf{v}, X)^2$$

at the limit $\Sigma^{-1} \rightarrow \mathbf{0}$, where $\mathbf{v} = \Phi_X^{-1} \mathbf{a}$ and $\mathbf{a} = (I_P(\varphi_i))_{i=1}^N$.

An expression for the worst-case error $e_k(\mathbf{v}, X)$ is given in Lemma 2.5. This construction has an air of artificiality as the variance depends on the covariance function but the mean does not.

2.4 Fundamental theory for Bayesian quadrature

With the results and tools introduced above it is possible to establish some fundamental theory for conjugate Bayesian quadrature that holds regardless of the properties the covariance kernel has. The following proposition consists of the intuitively obvious fact that new data decrease the integral variance.

Proposition 2.10. Let $\mathcal{D}$ and $\mathcal{D}'$ be datasets such that $\mathcal{D} \subseteq \mathcal{D}'$. Then the integral variance in (2.11) satisfies $\Sigma_{\mathcal{D}'} \leq \Sigma_{\mathcal{D}}$.

Proof. Suppose that $\mathcal{D}$ uses nodes $X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ and $\mathcal{D}'$ uses nodes $X' = \{\mathbf{x}_1, \dots, \mathbf{x}_{N'}\}$ for $N' \geq N$. Let $\mathbf{w} \in \mathbb{R}^N$ and $\mathbf{w}' \in \mathbb{R}^{N'}$ be the Bayesian quadrature weights based on X and X' , respectively. Additionally, let $\mathbf{w}_0 = (\mathbf{w}, 0, \dots, 0) \in \mathbb{R}^{N'}$. Then Proposition 2.6 yields

$$\begin{aligned} \Sigma_{\mathcal{D}'} &= e_k(\mathbf{w}', X')^2 = \min_{\mathbf{v} \in \mathbb{R}^{N'}} e_k(\mathbf{v}, X')^2 \leq e_k(\mathbf{w}_0, X')^2 = e_k(\mathbf{w}, X)^2 \\ &= \Sigma_{\mathcal{D}}, \end{aligned}$$

which is the claim. $\square$

Note that both $\Sigma_{\mathcal{D}}$ and $\Sigma_{\mathcal{D}'}$ in Proposition 2.10 are based on the same covariance function. In practice it can happen that the variance increases as more data are obtained if the covariance parameters are being learned from the data. For example, new data could indicate that the integrand is much rougher than previously thought. We discuss parameter estimation in Section 4.3. The next proposition describes how the covariance function affects the variance via its RKHS.

Proposition 2.11. Let $\Sigma_{\mathcal{D},k}$ be the integral variance in (2.11) under the prior $f \sim \text{GP}(m, k)$. If k and r are covariance kernels such that their RKHSs satisfy $\mathcal{H}(k) \subseteq \mathcal{H}(r)$, then there is a constant $c > 0$ such that

$$\Sigma_{\mathcal{D},k} \leq c \cdot \Sigma_{\mathcal{D},r} \quad \text{for every dataset } \mathcal{D}.$$

Proof. The inclusion $\mathcal{H}(k) \subseteq \mathcal{H}(r)$ implies that there is $c > 0$ such that $\|g\|_{\mathcal{H}(r)} \leq \sqrt{c} \cdot \|g\|_{\mathcal{H}(k)}$ for all $g \in \mathcal{H}(k)$; see Paulsen and Raghupathi (2016, Thm. 5.1). It follows that

$$\{g \in \mathcal{H}(k) : \|g\|_{\mathcal{H}(k)} \leq 1\} \subseteq \{g \in \mathcal{H}(r) : \|g\|_{\mathcal{H}(r)} \leq \sqrt{c}\}.$$

By the definition of the worst-case error and Proposition 2.6,

$$\begin{aligned}
\sqrt{\Sigma_{\mathcal{D},k}} &= \min_{\mathbf{v} \in \mathbb{R}^n} \sup_{\|g\|_{\mathcal{H}(k)} \leq 1} \left| \int_D g(\mathbf{x}) dP(\mathbf{x}) - \sum_{i=1}^N v_i g(\mathbf{x}_i) \right| \\
&\leq \min_{\mathbf{v} \in \mathbb{R}^n} \sup_{\|g\|_{\mathcal{H}(r)} \leq \sqrt{c}} \left| \int_D g(\mathbf{x}) dP(\mathbf{x}) - \sum_{i=1}^N v_i g(\mathbf{x}_i) \right| \\
&= \sqrt{c} \cdot \min_{\mathbf{v} \in \mathbb{R}^n} \sup_{\|g\|_{\mathcal{H}(r)} \leq 1} \left| \int_D g(\mathbf{x}) dP(\mathbf{x}) - \sum_{i=1}^N v_i g(\mathbf{x}_i) \right| \\
&= \sqrt{c \cdot \Sigma_{\mathcal{D},r}},
\end{aligned}$$

which is the claim. $\square$

Inclusions between RKHSs of standard covariance functions are well-understood (Kanagawa *et al.*, 2018). For example, if k_ν is a Matérn covariance in (2.4) with regularity ν and k_{se} the square exponential in (2.3), then

$$\mathcal{H}(k_{\nu_1}) \subseteq \mathcal{H}(k_{\nu_2}) \subsetneq \mathcal{H}(k_{\text{se}})$$

if $\nu_1 \leq \nu_2$ and the domain D has non-empty interior. The kernels in this chain of inclusions are allowed to have different length-scales. The final proposition of this section says that conjugate Bayesian quadrature integrates the kernel translates $k(\cdot, \mathbf{x}_i)$ exactly.

Proposition 2.12. Let $f \sim \text{GP}(0, k)$. If $f = k(\cdot, \mathbf{x}_i)$ for $i \in \{1, \dots, N\}$, then $\mu_{\mathcal{D}} = I_P(f)$.

Proof. Let $f = k(\cdot, \mathbf{x}_i)$ for $i \in \{1, \dots, N\}$. When $m \equiv 0$, the integral mean is $\mu_{\mathcal{D}} = \sum_{i=1}^N \mathbf{w}^\top \mathbf{f}_X$ for $\mathbf{w} \in \mathbb{R}^N$ that satisfies $\mathbf{K}_{XX} \mathbf{w} = \mathbf{k}_{PX}$. The i th row of this linear system is $\sum_{j=1}^N k(\mathbf{x}_i, \mathbf{x}_j) w_j = \int_D k(\mathbf{x}, \mathbf{x}_i) dP(\mathbf{x})$, which we can write as $\sum_{j=1}^N f(\mathbf{x}_j) w_j = \int_D f(\mathbf{x}) dP(\mathbf{x})$. Equivalently, $\mu_{\mathcal{D}} = \mathbf{w}^\top \mathbf{f}_X = I_P(f)$. $\square$

2.5 Generalisations

We have elected to work in the setting described above to retain a degree of concreteness and to avoid introduction of overbearing notation. However, a number of conceptually straightforward (though not

necessarily so in practice) generalisations are possible and appear often in the literature. These include the following:

- *Domain.* There is nothing in Bayesian quadrature or Gaussian processes that ties them to subsets of $\mathbb{R}^d$. In general, one can let D be any set and P any measure on D . This generality of Bayesian quadrature has been recognised since the very beginning (Larkin, 1972; O’Hagan, 1991). A recurring example in the literature is the computation of *global illumination integrals* on the unit sphere $\mathbb{S}^2 = \{\mathbf{x} \in \mathbb{R}^3 : \|\mathbf{x}\| = 1\}$, to which Bayesian quadrature has been applied in Broullat *et al.* (2009), Marques *et al.* (2013), Marques *et al.* (2015), Xi *et al.* (2018), Briol *et al.* (2019), and Karvonen *et al.* (2019b). Other domains may be found in Fröhlich *et al.* (2021) and Kang *et al.* (2025). Integration on a sphere has also been considered in the literature on radial basis function integration (Sommariva and Womersley, 2005; Fuselier *et al.*, 2014; Reeger and Fornberg, 2015) whose connection to Bayesian quadrature we described in Section 2.3.2.
- *Information.* It is straightforward to incorporate any linear information (i.e., observations that have been obtained via application of a linear functional) in Gaussian process regression (Särkkä, 2011) and, consequently, Bayesian quadrature. We refer to Oettershagen (2017, Sec. 3) for a rigorous treatment of the topic in the context of approximation theory, the connection to Gaussian processes of which we reviewed in Section 2.3. Derivative observations, which occur frequently in Gaussian process literature (e.g., Solak *et al.*, 2002), have been used in Bayesian quadrature by O’Hagan (1992, Sec. 2.5) and Prüher and Särkkä (2016), while Weiland *et al.* (2026) use integral observations over sub-domains.
- *Noise.* Incorporating observations that are perturbed by Gaussian noise, such as when f is the output of a stochastic simulation, to Bayesian quadrature is straightforward. In the typical Gaussian conjugate setting this simply amounts to incrementing the diagonal of the covariance matrix with the noise variances (see the equations for the regularised mean and variance in Section 4.4); this is

supported by most open source implementations of Bayesian quadrature. Even though the setting of noisy observations is dominant in the Gaussian process regression literature, Bayesian quadrature with noisy observations appears to have received little explicit attention. The only substantial analysis the noisy case that we are aware of is by Cai *et al.* (2023). Ma *et al.* (2014) also incorporate noise in their method.

- *Prior.* In our definition of Bayesian quadrature the integrand needs to be modelled as a potentially non-linearly transformed Gaussian process $f = \varphi \circ g$. As discussed in Section 2.2.3, one is in principle free to use any suitable stochastic process f to model the integrand. Due to computational tractability issues, priors other than transformed Gaussian processes are rarely used. Student's t -processes, which differ little from Gaussian processes in practice, have been used in numerical integration by O'Hagan (1991, Sec. 2.2) and Prüher *et al.* (2017). What appears to be the most interesting example of non-Gaussian priors in numerical integration is due to Zhu *et al.* (2020), who use Bayesian additive regression trees.

3

Taxonomy of Bayesian quadrature

Definition 2.1 constructed Bayesian quadrature as a method for computing or approximating the distribution $I_P(\mathbf{f}) \mid \mathcal{D}$. This abstract formulation admits several concrete realizations, each corresponding to a particular Bayesian quadrature method or algorithm. In recent years, the literature has proposed a wide variety of approaches consistent with Definition 2.1, differing mainly in their methodological choices and practical implementation. This chapter offers a concise, high-level overview of these alternatives. It may be read as a taxonomy of Bayesian quadrature methods or used as a reference for terminology. The taxonomy is illustrated visually in Figure 3.1, and Table 3.1 classifies selected algorithms from the literature.

To motivate the taxonomy, we inspect a high-level pseudocode for Bayesian quadrature shown in Algorithm 1. The pseudocode reveals three core components: a model for $\mathbf{f}$ specified by $\mathbf{g}$ and φ (see (2.7)); a sampling procedure (line 1) for selecting the nodes X that constitute the dataset; and an inference procedure (line 4) that computes the return object *res* representing the distribution $I_P(\mathbf{f}) \mid \mathcal{D}$.¹

¹As $I_P(\mathbf{f}) \mid \mathcal{D}$ is a distribution, any datatype that can represent a distribution is valid. For Gaussians, a common way is to return the sufficient statistics: mean

Algorithm 1 Bayesian quadrature

Require:

- The integration task:
 - The integrand f , given by a function handle.
 - The integration measure P , usually given by the density p and domain D .
 - The model $\mathbf{f}$:
 - The GP $\mathbf{g}$, given by function handles of k and m .
 - The transformation φ and its inverse given by function handles.
 - The budget given by the dataset size N .
- 1: $X \leftarrow \text{SAMPLE}(N, \mathbf{f}, P)$ $\triangleright$ Sample N nodes.
 - 2: $\mathbf{f}_X \leftarrow f(\mathbf{x}_i)$ for $\mathbf{x}_i$ in X $\triangleright$ Evaluate integrand at nodes.
 - 3: $\mathcal{D} \leftarrow [X, \mathbf{f}_X]$ $\triangleright$ The dataset (for notational convenience).
 - 4: $\text{res} \leftarrow \text{INFER}(\mathbf{f}, P, \mathcal{D})$ $\triangleright$ The result res is a representation of $I_P(\mathbf{f}) \mid \mathcal{D}$.
 - 5: **return** res
-

This naturally leads to three main axes along which Bayesian quadrature methods can be organized:

- a *model axis*, which characterizes the Gaussian process prior $\mathbf{g}$ and the transformation φ (Section 3.1);
- an *inference axis*, which specifies how the result res is computed (Section 3.2);
- a *sampling axis*, which describes how the nodes X are selected (Section 3.3).

Along each axis, we introduce multi-label subcategories intended to reflect the current landscape of methods and algorithms. Here, “multi-label” indicates that categories are not necessarily mutually exclusive (when they are, this will be stated explicitly); consequently, a method may be assigned multiple labels per axis. An illustrative example is provided in Section 3.5.

Not all Bayesian quadrature methods fit neatly into the pseudocode outlined above. This primarily concerns approaches that actively manage data acquisition and computational budgets (Section 3.3). Inevitably, any taxonomy involves a degree of subjectivity; ours is guided by practical considerations motivated by applications in applied science.

$$\mathbb{E}[I_P(\mathbf{f}) \mid \mathcal{D}] \text{ and variance } \text{Var}[I_P(\mathbf{f}) \mid \mathcal{D}].$$

We occasionally use the big- $\mathcal{O}$ notation when referring to computational complexity or memory requirements. Accessible implementations of Bayesian quadrature algorithms are unfortunately still scarce and not on par with the richness of the literature. Some are available in the open source libraries EmuKit (for Python; Paleyes *et al.*, 2019; Paleyes *et al.*, 2023), GAIL (for MATLAB; Choi *et al.*, 2020), and ProbNum (for Python; Wenger *et al.*, 2021).

3.1 Model

The first axis of the taxonomy is concerned with the prior model $\mathbf{f} = \varphi \circ \mathbf{g}$, in particular the choice of the function φ . We differentiate between two mutually distinct classes: “Conjugate” and “non-conjugate” that in essence categorize the posterior $I_P(\mathbf{f}) \mid \mathcal{D}$ as Gaussian or non-Gaussian.

- *Conjugate:* Conjugate models are arguably the most well-known and the most straightforward Bayesian quadrature models. They are described in Section 2.2.1, where φ is affine and the Gaussian process $\mathbf{g}$ implies a Gaussian process $\mathbf{f}$ and a Gaussian posterior $I_P(\mathbf{f}) \mid \mathcal{D}$. Conjugate Bayesian quadrature models inherit the conveniences of GP regression models, and $I_P(\mathbf{f}) \mid \mathcal{D}$ can be computed exactly under certain well-understood circumstances (Section 3.2). For these reasons, conjugate Bayesian quadrature is sometimes also called *standard* Bayesian quadrature (Karvonen, 2019; Naslidnyk *et al.*, 2021) or *vanilla* Bayesian quadrature (Paleyes *et al.*, 2019; Gessner *et al.*, 2020; Adachi *et al.*, 2022). As $\mathbf{f}$ and $I_P(\mathbf{f}) \mid \mathcal{D}$ are confined to Gaussians, conjugate models are limited in their expressiveness.
- *Non-conjugate:* Non-conjugate models are often motivated by the desire to encode physically meaningful, non-negative integrands f and integral values $I_P(\mathbf{f}) \mid \mathcal{D}$ which Gaussians cannot provide. Steering away from Gaussianity is possible with an appropriate function φ . Non-conjugate models are described in Section 2.2.2 where φ is not affine, $\mathbf{f}$ is a random process other than a Gaussian process and the implied posterior $I_P(\mathbf{f}) \mid \mathcal{D}$ is non-Gaussian. For example, for $\varphi(x) = \alpha + x^2$ as in Gunter *et al.* (2014), $\mathbf{f}$ is a

process whose marginals are noncentral χ^2 -distributions of degree 1. Inference, however, is usually less straightforward and may require approximations (Section 3.2). Due to the presence of φ , non-conjugate Bayesian quadrature is sometimes also referred to as *warped*² Bayesian quadrature (Gunter *et al.*, 2014; Paleyes *et al.*, 2019). While the space of possible φ is rich in principle, only a limited number have been explored in the context of BQ.

Further, a kernel k and mean function m identify the Gaussian process g . As mentioned in Section 2.1, common choices for k are the square exponential kernel and kernels of the Matérn family, either isotropic or of product form. Any model may additionally be characterized by hyper-parameters (such as kernel parameters) that are either given or set by an appropriate method (Section 4.3).

3.2 Inference

Once the model f has been defined and a dataset $\mathcal{D}$ has been obtained (node sampling forms the third axis of the taxonomy and is the content of Section 3.3), one can attempt to compute the solution $I_P(f) \mid \mathcal{D}$. The second axis of the taxonomy—*inference*—distinguishes the quality of the solution as either “exact” or “approximate”. A third label “scalable” emphasizes computational complexity of the inference algorithm with respect to the dataset size, N .

- *Exact*: The inference method is called exact if $I_P(f) \mid \mathcal{D}$ can be computed without approximation error (*res* in Algorithm 1 is an exact representation). This is possible if $I_P(f) \mid \mathcal{D}$ has closed form and admits numerically exact computations. Exact inference can for example be realized under the conjugate BQ model if the measure P and kernel k pair yields closed form solutions of Eqs. (2.9) and (2.11); see Section 4.1. Since the expressions have closed

²The name is likely inspired by Snelson *et al.* (2003) who learn the transformation φ from data in the context of GP regression and call their method “warped Gaussian processes”. In the context of Bayesian quadrature, the transformation φ is usually fixed, but could in principle also be learned in a similar way.

form, exact inference in Bayesian quadrature is also called *analytic*. Section 4.4 discusses some common practical considerations concerning numerical precision.

- *Approximate*: The inference method is called approximate if the returned distribution (res in Algorithm 1) differs from $I_P(f) \mid \mathcal{D}$ due to methodological choices.³ Such approximations to $I_P(f) \mid \mathcal{D}$ are necessary if (i) the model $f \mid \mathcal{D}$ does not admit analytic integration, or (ii) if evaluating the analytic expressions to numerical precision is prohibitively expensive. In practice, inference methods for non-conjugate BQ models often entail approximations due to the former reason because the process f can be arbitrarily complicated. Some recent approaches use Gaussian approximations to f via linearisation or moment matching which are integrated instead of f . Conjugate Bayesian quadrature, where f is a Gaussian process, may also require approximations when the kernel k and integration measure P pair has no closed form kernel mean embedding (Section 4.1). Approximations of kernel embeddings have been scarcely studied in the context of BQ, but in principle, any means of approximation is admissible. Another reason arises when the closed form solution is too expensive to compute for large dataset sizes N because the computational complexity scales unfavorably. Ways to remedy this are discussed next.

Algorithm 2 shows a pseudocode of the exact inference method for a conjugate Bayesian quadrature model. From the pseudocode it is apparent that exact inference for the conjugate model is feasible if the kernel mean embedding k_P , the initial variance k_{PP} and the prior mean integral m_P are analytic. Algorithm 2 first computes k_{PP} and m_P (lines 2–3). Then, it evaluates the kernel k , mean function m and

³Bayesian quadrature is related to the field of *probabilistic numerics* (Hennig *et al.*, 2015; Cockayne *et al.*, 2019; Hennig *et al.*, 2022) which frames integration, among other numerical algorithms, as inference. If we were to follow Cockayne *et al.* (2019, Defs. 2.2 & 2.5) who characterize a probabilistic numerical method as “Bayesian” or “non-Bayesian”, *Bayesian quadrature* as in Definition 2.1 admitting *exact* or *approximate* inference according to our taxonomy would naturally be called *Bayesian Bayesian quadrature* or *non-Bayesian Bayesian quadrature*, respectively.

kernel mean embedding k_P at the given nodes (lines 4–5) and constructs the kernel Gram matrix $\mathbf{K}_{XX}$ (line 6). The bulk of the computation is performed in line 8 where the algorithm solves a linear system of size N in order to retrieve the weights $\mathbf{w} = \mathbf{K}_{XX}^{-1} \mathbf{k}_{PX}$. Lines 9–10 then perform simple vector multiplications and summations that yield the final solution.

Algorithm 2 Exact inference method for the conjugate BQ model

Require:

- Function handles to the kernel k and mean function m .
- Function handle to the kernel mean embedding k_P .
- Function handle to (or precomputed) initial variance k_{PP} and prior mean integral m_P .
- The dataset $\mathcal{D}$ with nodes X and observations $\mathbf{f}_X$.

```

1: function INFER( $k, m, k_P, m_P, k_{PP}, X, \mathbf{f}_X$ )
2:    $m_P \leftarrow m_P()$  ▷ Evaluate or assign prior mean integral  $I_P(m)$ .
3:    $k_{PP} \leftarrow k_{PP}()$  ▷ Evaluate or assign initial variance as in Eq. (2.12).
4:    $\mathbf{k}_{PX} \leftarrow k_P(\mathbf{x}_i)$  for  $\mathbf{x}_i$  in  $X$  ▷ Evaluate kernel mean embedding at nodes.
5:    $\mathbf{m}_X \leftarrow m(\mathbf{x}_i)$  for  $\mathbf{x}_i$  in  $X$  ▷ Evaluate prior mean at nodes.
6:    $\mathbf{K}_{XX} \leftarrow k(\mathbf{x}_i, \mathbf{x}_j)$  for  $\mathbf{x}_i$  in  $X$  for  $\mathbf{x}_j$  in  $X$  ▷ Evaluate kernel for all node combinations.
7:
8:    $\mathbf{w} \leftarrow \text{SOLVELINEAR}(\mathbf{K}_{XX}, \mathbf{k}_{PX})$  ▷ Solve linear system,  $\mathcal{O}(N^3)$ .
9:    $\mu_{\mathcal{D}} \leftarrow m_P + \mathbf{w}^\top (\mathbf{f}_X - \mathbf{m}_X)$  ▷ Compute mean of  $I_P(\mathbf{f}) \mid \mathcal{D}$ , Eq. (2.13).
10:   $\Sigma_{\mathcal{D}} \leftarrow k_{PP} - \mathbf{w}^\top \mathbf{k}_{PX}$  ▷ Compute variance of  $I_P(\mathbf{f}) \mid \mathcal{D}$ , Eq. (2.13).
11:   $res \leftarrow (\mu_{\mathcal{D}}, \Sigma_{\mathcal{D}})$  ▷  $I_P(\mathbf{f}) \mid \mathcal{D}$  is Gaussian and described by  $\mu_{\mathcal{D}}$  and  $\Sigma_{\mathcal{D}}$ .
12:  return  $res$ 
13: end function

```

Algorithm 2 shows arguably the most well-known BQ setting which inherits its quadratic memory $\mathcal{O}(N^2)$ (line 6) and cubic compute cost $\mathcal{O}(N^3)$ (line 8) from standard Gaussian process regression. Thus, the final label of the inference axis describes the scalability of the BQ inference method with respect to N compared to the standard cubic cost. This label is not mutually exclusive to the previous two; in fact, both an “exact” and “approximate” inference method can be either scalable or not.

- *Scalable:* The Bayesian quadrature inference method is called scalable if its computational complexity is less than cubic in the number of datapoints N and thus scales favorably compared

to standard Gaussian process regression. Interestingly, so far scalable methods have been proposed primarily for *exact* inference under the conjugate BQ model. For instance, Karvonen and Särkkä (2018) exploit structure in the posterior mean equation for certain kernels, measures and a custom node sampling scheme; their algorithm scales sub-cubically. Jagadeeswaran and Hickernell (2019) use fast Fourier transform to obtain computational cost $\mathcal{O}(N \log N)$. Scalable methods for *approximate* inference under the conjugate BQ model have not been explored systematically.⁴ Akella *et al.* (2021) use a scalable approximate BQ method based on structured kernel interpolation (Wilson and Nickisch, 2015) to estimate policy gradients and Li *et al.* (2025) use sparse GPs within variational BQ. We are not aware of scalable methods for non-conjugate BQ models. Scalable Bayesian quadrature is sometimes also referred to as *fast* Bayesian quadrature (Jagadeeswaran and Hickernell, 2019; Jagadeeswaran and Hickernell, 2022). This is not to be confused with the use of the term *fast* BQ as in Gunter *et al.* (2014) or Adachi *et al.* (2022) where the term highlights sample efficiency or end-to-end wall-clock time performance of the BQ method and data retrieval which our taxonomy does not address. Here we also mention a recent quantum algorithm for Bayesian quadrature by Galvis-Flores *et al.* (2025).

3.3 Sampling (node selection)

The first two axes described the BQ model and inference method when a dataset $\mathcal{D}$ is provided. The third and last axis of the taxonomy focuses on the dataset $\mathcal{D}$, particularly emphasizing the *selection of the nodes* X (as the corresponding function values $\mathbf{f}_X$ can be retrieved by evaluating f at X). We start by observing that traditionally, to solve an intractable integral, practitioners may refer to classic quadrature rules where nodes are predefined and cannot be chosen freely, or Monte Carlo estimators,

⁴There is a vast literature on scalable algorithms, such as random Fourier features, for approximate Gaussian process inference (Quiñero-Candela and Rasmussen, 2005; Bui *et al.*, 2017; Liu *et al.*, 2020). Such algorithms have seen little use in the context of Bayesian quadrature.

where nodes must be random. Bayesian quadrature is positioned in between these methods as it generally can use any nodes $\mathbf{x}_1, \dots, \mathbf{x}_N$ as long they are pairwise distinct. This fortunate property of BQ leads to a rich variety of practical sampling options wherefore this section is divided into several subsections.

3.3.1 Mode of sampling

We begin with three high-level categories that are fundamental to the mode in which the nodes are obtained: “Deterministic”, “random”, and “fixed”. The former two are mutually distinct, while the third is simply undetermined.

- *Deterministic*: Deterministic node selection schemes are those schemes that *ideally* do not use random or pseudo-random numbers, except potentially for an initialization step such as random selection of the generating vector for lattice nodes (Kaarnioja *et al.*, 2025; Jagadeeswaran and Hickernell, 2019, Sec. 4). In practice, deterministic schemes often comprise a degree of randomness, for example when the nodes are selected by maximizing an acquisition function and the optimizer makes use of randomness.⁵
- *Random*: Random sampling schemes are those that explicitly use random or pseudo-random numbers to generate the nodes. In contrast to “deterministic” sampling as described above, “random” sampling has a degree of randomness intrinsic to its design and even under ideal practical conditions. An early use of random node sampling for BQ was proposed as part of the Bayesian Monte Carlo method by Rasmussen and Ghahramani (2002) who sampled from the integration measure P . In the meantime, also other distributions have been used (Bach, 2017; Briol *et al.*, 2017).

⁵We are not aware of explicit use of stochastic optimizers in BQ, although stochastic optimizers have been used in the related field of Bayesian optimization (for an overview, see Balandat *et al.*, 2020). However, even deterministic optimizers usually contain a degree of randomness. For example, the initial point of the iteration needs to be chosen which may affect which maximizer is found and hence which node is sampled.

- *Fixed / unknown*: Fixed or unknown sampling describes a set of nodes whose generating process is either unknown or beyond the control of the Bayesian quadrature practitioner. A fixed set of nodes could for example be provided by a (black-box) service.

The above three categories describe the degree of control that the practitioner has (or rather needs to have) over the nodes, strictly within the context of Bayesian quadrature: When the nodes are fixed, one has *no control*; when they are random, one has only *distributional control*; and when they are deterministic, one has *full control* over the nodes (at least in principle).⁶ The remaining categories provide more granularity to node selection.

3.3.2 Contextual sampling

We now inspect line 1 of Algorithm 1 more closely and observe that the model f is an optional argument to the `SAMPLE` method which means that some sampling procedures may depend on their context. Thus the first granular label of the sampling axis is called “model-dependent”.

- *Model-dependent*: It is easy to fathom sampling schemes that are independent of the model f , such as grids or draws from the integration measure P . If the sampling method does not request the model f (the `SAMPLE` method in line 1 of Algorithm 1 does not require f as argument) we call it “model-independent” and if it does, “model-dependent”. As of now, this definition is purely algorithmic in the sense that the nodes X could be created either before or after f is chosen. As we will see soon, the distinction is somewhat more intricate than it may initially appear since it disentangles (more so than other labels) the method from the algorithm. This is best explained with an example: It is a well-known fact (e.g., Hennig *et al.*, 2022, Sec. 11.1.3) that for f a

⁶The mode of sampling at this point simply describes characteristics of the node selection option. Unlike data-dependent estimators that may inherit the randomness property from the mode of sampling, Bayesian models (the BQ model is a Bayesian model) do not. However, estimators derived from the BQ model such as the mean or MAP estimator may be analyzed in this way.

Brownian motion and P the uniform measure on the interval $[0, 1]$, equidistant nodes are the maximally-informative design in the sense that $\text{Var}[I_P(f) \mid \mathcal{D}]$ is minimized. The creation of equidistant nodes on $[0, 1]$ of course does not require f explicitly. However, since the practitioner promises maximally informative nodes, knowledge about f (i.e., it being a Brownian motion) is required before the nodes are created. In other words, if the model f changes, the sampling method will need to change as well. If the practitioner were to use the equidistant nodes no matter f (no “link” between f and the sampling method is claimed), the BQ method would be model-independent. To conclude the example, maximally informative nodes for processes f other than a Brownian motion are not generally analytic and their computation may require f explicitly in which case it being model-dependent is apparent. Model-dependent sampling often implies a notion of informativity of the nodes under the model. A recent example of model-dependent node selection is given in Adachi *et al.* (2022) who construct a sampling distribution that depends on f as well as P .⁷ Further, active sampling (Section 3.3.4) is model-dependent by definition as the acquisition function depends on f . It is highly unclear when model-dependent or independent sampling for Bayesian quadrature should be preferred and it may be contingent on the application at hand. Although model-independent node selection has no knowledge about f it often performs well, is generally fast as nodes can be pre-computed, and lends itself well to theoretical examination. Model-dependent node selection has the benefit of being tailored to the integration problem at hand.

Model-dependent sampling, as mentioned, usually implies that the nodes are chosen due to their informativity under the model f . Another type of dependence may occur if the sampler is interlaced with the model additionally or exclusively via the *inference method*. Such nodes may or may not be chosen due to their informative content, but most likely they are chosen for making the inference method “work”. An example

⁷See Adachi *et al.* (2023c) for an application of this method to the selection of lithium-ion battery models.

are inference methods that are only scalable (have computational cost less than cubic in N) if the nodes follow a specific sampling strategy. The inference method can even be connected to the sampler in such a way that choosing any other sampler would not lead to an executable algorithm. To highlight the association we will call these sampling schemes “inference-dependent”. This is a good point to reiterate that the taxonomy labels should be attached to specific instances of BQ methods that comprise a model, an inference and a sampling method. Hence labels need to be interpreted with respect to the BQ method as a whole and in the context of the practitioner’s aim. A context-free taxonomy with merit is almost impossible as BQ methods only allow to plug-and-play under restrictions.⁸

- *Inference-dependent:* Some BQ methods require specific node sampling schemes that are motivated by the unique properties of their inference method. We call those samplers “inference-dependent”. As mentioned, examples are the sampling schemes of the exact, scalable BQ methods in Karvonen and Särkkä (2018) and Jagadeeswaran and Hickernell (2019). The concept is similar in spirit (albeit not as strict) to how nodes in classical Gaussian quadrature rules are derived.

3.3.3 Sequential sampling

So far, we have worked with Algorithm 1 where the nodes X are sampled (line 1) as one large batch of size N . We are now ready to study “sequential” sampling where nodes are acquired repeatedly in a loop in smaller sized batches (and often even only one at a time). Algorithm 3 shows the general layout of a sequential method.

⁸The situation is slightly different in the related Bayesian optimization (BO) method where plug-and-play is often feasible. This is because the inference method in BO has a less prominent role and influences node sampling but not the return object, which is $\arg \min_{\mathbf{x}_i \in X} f(\mathbf{x}_i)$. For the same reason, BO methods always require model-dependent sampling in order to be meaningful.

Algorithm 3 Sequential Bayesian quadrature

Require:

- The integration task given by f and P .
 - The model f given by g and φ .
 - The stopping condition that may represent a budget.
 - The batch size n .
 - Optional, an initial design X_{init} .
- 1: $\mathbf{f}_{X_{\text{init}}} \leftarrow f(\mathbf{x}_i)$ for $\mathbf{x}_i$ in X_{init} $\triangleright$ Evaluate integrand at initial design.
 - 2: $\mathcal{D} \leftarrow [X_{\text{init}}, \mathbf{f}_{X_{\text{init}}}]$ $\triangleright$ The dataset is initialized with the initial design.
 - 3: **while** not STOPPING_CONDITION($\mathcal{D}, f, P$) **do**
 - 4: $X_{\text{batch}} \leftarrow \text{SAMPLE}(n, f, P, \mathcal{D})$ $\triangleright$ Sample a batch of n nodes.
 - 5: $\mathbf{f}_{X_{\text{batch}}} \leftarrow f(\mathbf{x}_i)$ for $\mathbf{x}_i$ in X_{batch} $\triangleright$ Evaluate integrand at batch.
 - 6: $\mathcal{D} \leftarrow \mathcal{D} \cup [X_{\text{batch}}, \mathbf{f}_{X_{\text{batch}}}]$ $\triangleright$ Augment the dataset.
 - 7: **end while**
 - 8: $\text{res} \leftarrow \text{INFER}(f, P, \mathcal{D})$ $\triangleright$ The result res is a representation of $I_P(f) \mid \mathcal{D}$.
 - 9: **return** res
-

We observe that Algorithm 3 is essentially a loop (lines 3–7). Before each step (one step equals executing lines 4–6) a STOPPING_CONDITION checks if the iteration should be halted (line 3). The stopping condition may use information of the dataset $\mathcal{D}$ containing the data collected thus far, the model f , and the measure P . During each step, a SAMPLER produces a batch of nodes X_{batch} of size n (line 4) at which the integrand is evaluated (line 5). Finally, the dataset $\mathcal{D}$ is augmented with the newly acquired nodes and observations $\mathbf{f}_{X_{\text{batch}}}$ (line 6). Once the stopping condition is fulfilled, the loop terminates (the while-statement in line 3 is FALSE and the loop breaks) and the solution res is computed and returned (lines 8–9).

Algorithm 3 not only has different structure than Algorithm 1 but also requires different inputs, most prominently a *stopping condition*, which alludes to one of the general motivations of sequential BQ methods: the sampling (and hence the evaluation of f) is paced in order to test for some user-defined stopping criterion. For example, the practitioner may want to terminate the BQ method when the variance $\text{Var}[I_P(f) \mid \mathcal{D}]$ falls below a tolerance value. The BQ method samples as many (small) batches as needed until the stopping condition is fulfilled and it is unknown a priori how many nodes the method eventually collects in total (this is elaborated further under *dynamic budget* in Section 3.4). A second, orthogonal motivation for sequential node acquisition emerges if

the sampling method in line 4 is model-dependent and changes with the collected dataset up to that point (since $\mathbf{f} \mid \mathcal{D}$ changes). A sequential approach is then able to gather informative nodes at each step of the loop depending on the current evidence and prior model. An example of the latter are some versions of ‘active’ sampling introduced in the next subsection.

- *Sequential*: Sequential sampling produces nodes in an iterative fashion in small batches of size n as sketched in Algorithm 3. Sequential sampling requires a stopping condition that indicates when the data acquisition should terminate. Often, nodes are sampled one at a time ($n = 1$); if $n > 1$ then sequential sampling is also called *batched*. Cook and Clayton (1998) were the first to focus on sequential BQ methods.

3.3.4 Active sampling

The last label of the sampling axis of the taxonomy is “active” sampling for BQ. The term “active” is used in various machine learning fields and generally describes that the model can query useful or informative data. The idea is that the model may perform better if it can actively select data rather than just passively accept data that is handed to it. Choosing the data thus constitutes a *decision problem* that the model needs to solve. In the context of BQ this means that the model $\mathbf{f}$ influences the sampling method and “requests” f to be evaluated at certain nodes. For probabilistic models (the BQ model is a probabilistic model), an explicit formulation of this decision problem is possible via Bayesian decision theory. There, node selection is cast as the solution of the following optimization problem (we will use this formulation throughout for active sampling):

$$\mathbf{x}_1, \dots, \mathbf{x}_N = \arg \max_{\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N} \mathbb{E}_{\mathbf{f}}[u(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N \mid \mathbf{f}, P)]. \quad (3.1)$$

In the above formula, the nodes $\mathbf{x}_1, \dots, \mathbf{x}_N$ are the maximizer of an expected utility called an *acquisition function*

$$a(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N \mid P) := \mathbb{E}_{\mathbf{f}}[u(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N \mid \mathbf{f}, P)]$$

that, together with the $\arg\max$ operator, defines the node selection policy and hence the sampling method of the active BQ method. The function $u(\cdot \mid \mathbf{f}, P)$ is a *utility function* that quantifies the usefulness of evaluating the integrand f at certain nodes under the model $\mathbf{f}$.⁹

While the utility function is arbitrary in the sense that it describes the practitioner’s conviction of usefulness, the active BQ method internally assumes the resulting sampling policy to provide the best course of action. Once again, this is best explained through an example. Let $\tilde{\mathcal{D}} = \{(\tilde{\mathbf{x}}_1, \mathbf{f}(\tilde{\mathbf{x}}_1)), \dots, (\tilde{\mathbf{x}}_N, \mathbf{f}(\tilde{\mathbf{x}}_N))\}$ denote a hypothetical dataset, meaning that the targets are the hypothetical values $\mathbf{f}(\tilde{\mathbf{x}}_i)$ under the model [instead of the integrand values $f(\tilde{\mathbf{x}})$], and let $\mathbf{f} \mid \tilde{\mathcal{D}}$ be the model conditioned on the hypothetical dataset. If we choose $u(\cdot \mid \mathbf{f}, P) = -(I_P(\mathbf{f} \mid \tilde{\mathcal{D}}) - \mu_{\tilde{\mathcal{D}}})^2$ to be the negative [Eq. (3.1) is a maximization problem] *square error* between the integral $I_P(\mathbf{f} \mid \tilde{\mathcal{D}})$ of the model and its expectation $\mu_{\tilde{\mathcal{D}}}$ (the square error is a natural and decent choice of utility), the resulting acquisition function $a(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N \mid P) = -\text{Var}[I_P(\mathbf{f}) \mid \tilde{\mathcal{D}}]$ equals the negative variance of the integral over the conditioned model and its maximization yields maximally informative nodes (e.g., Hennig *et al.*, 2022, Sec. 10.2). Likewise, other utility functions can be linked to well-known BQ acquisition functions, some of which will be discussed further in Section 4.2.3.

Eq. (3.1) assumes that the practitioner aims to sample a set of N nodes at once. Often, active node sampling is deployed in a sequential fashion due to the mentioned potential benefits of sequential node acquisition of model-dependent sampling schemes. In such cases, the utility function takes into account sampling decisions under the model in future steps of the iteration when making the sampling decision in the current step (e.g., Garnett, 2023, Section 5.2). In practice, for tractability reasons often only an isolated decision is considered, meaning that the

⁹The formulation of the active sampler is supported by the von Neumann–Morgenstern utility theorem (Neumann and Morgenstern, 1944) which shows, under some conditions, that the decision of a rational agent under uncertainty can be constructed as a policy that maximizes an expected utility function. For an in-depth derivation and motivation of active sampling in the related field of Bayesian optimization see, for example, Section 5.1 in Garnett (2023). The treatment in BQ is analogous and not repeated here. For Bayesian decision theory for GPs, see Section 2.4 in Rasmussen and Williams (2006).

utility function encodes the usefulness of sampling the current step (or even just a single next node) as if it were the last step/node that the model can ever request. Such an approach is called *one-step-lookahead* or (*maximally*) *myopic* sequential active sampling and is usually the default over more elaborate schemes due to its practicality and tractability (e.g., Garnett, 2023, Sec. 7.2).¹⁰ Due to its importance, Algorithm 4 shows this approach.

Algorithm 4 Myopic active sampling for Bayesian quadrature

Require:

- The integration measure P .
- The model f given by g and φ .
- The so-far collected data $\mathcal{D}$.

```

1: function SAMPLE( $f, P, \mathcal{D}$ )
2:    $u_f \leftarrow u(\cdot \mid f, P, \mathcal{D})$            ▷ Define the utility function (only for notation).
3:    $a \leftarrow \mathbb{E}_f[u_f(\cdot)]$                  ▷ Define the acquisition function.
4:    $x \leftarrow \text{POLICY}(a)$                  ▷ The policy maximizes the acquisition function.
5:   return  $x$ 
6: end function
7:
8: function POLICY( $a$ )
9:    $x \leftarrow \arg \max_{\tilde{x}} a(\tilde{x})$            ▷ Analytic or with off-the-shelf optimizers.
10:  return  $x$ 
11: end function

```

Let us inspect the return object (line 5) of Algorithm 4 which is a single node x since the active sampler is maximally myopic with batch size $n = 1$.¹¹ The sampled node is the return object of a maximum acquisition policy (line 4). The policy, which is highlighted in a separate

¹⁰Non-myopic sequential, active sampling has so far received little attention in BQ. The only approach we are aware of is due to Jiang *et al.* (2019). A simple example can illustrate the difference between myopic and non-myopic sampling: Consider a sampling horizon of $N = 2$ and let f be a Brownian motion as in the example of Section 3.3.2. A non-myopic sampler would place the first node at $x = 4/5$, anticipating a future node on $x = 2/5$, while the initial node of the myopic sampler would be placed sub-optimally at $x = 2/3$ (e.g., Hennig *et al.*, 2022, Sec. 11.1.3).

¹¹Note that non-myopic sampling and batched sampling are orthogonal concepts. In principle, there exist batched ($n > 1$) and non-batched ($n = 1$) methods that can be either myopic or not. Batched BQ methods that use maximally myopic acquisition functions are proposed in Wagstaff *et al.* (2018) and Hong *et al.* (2024). Further, batched and non-batched methods need of course not even be active.

function on lines 8–11, takes an acquisition function as input that is defined like a lambda function on lines 2 and 3 inside the sampling method based on f , P and $\mathcal{D}$. The dependence of $u(\cdot \mid f, P, \mathcal{D})$ on the current dataset $\mathcal{D}$ in Algorithm 4 is due to the sequential nature of the sampler and was not present in Eq. (3.1). The maximizer of the optimization problem on line 9 can either be implemented directly if an analytically solution is available, or be found by off-the-shelf optimizers, especially if gradients of f are accessible.¹²

We are finally ready to define the “active” label.

- *Active*: Actively sampled nodes are the solution to an optimization problem as defined in Eq. (3.1). They require the choice of a utility function u by the practitioner. Often active samplers are deployed in combination with an iterative, sequential sampling scheme. Similar concepts exist in *experimental design* (Fisher, 1936) and *Bayesian optimization* (Mockus, 1989; Garnett, 2023) where the utility functions are tailored to the respective quantity of interest. Section 4.2.3 discusses utility functions specific to Bayesian quadrature. If the resulting acquisition function depends on some previously collected dataset $\mathcal{D}$, we further distinguish *implicitly* and *explicitly active* sampling which describes the type of dependence.
 - *Implicitly active*: Active sampling is implicit if the acquisition function depends on the observations $\mathbf{f}_X$ only via a data-dependent model for f . This is for example the case when the kernel-parameters of a conjugate BQ model are estimated with empirical Bayes after each step, while performing sequential active sampling (Section 4.3 will discuss kernel-parameter estimation). Implicitly active sampling is sometimes also called *semi-active*, *non-adaptive*, *open-loop* (e.g., Hennig *et al.*, 2022) or simply *active* (e.g., Gessner *et al.*, 2020).

¹²It is not necessary to have access to gradients of the integrand f since the gradients of the acquisition function a depend only on gradients of the model f , which is under the control of the practitioner.

- *Explicitly active*: Active sampling is explicit if the acquisition function depends on $\mathbf{f}_X$ directly, beyond a data-dependent model. An example is uncertainty sampling in the WSABI method (Gunter *et al.*, 2014). In contrast to implicitly active sampling, explicitly active sampling schemes cannot be precomputed if data-dependency of the model is dropped. Explicitly active sampling thus naturally combines with sequential sampling. Explicitly active sampling is also referred to as *adaptive*¹³ or *closed-loop* (both, e.g., Hennig *et al.*, 2022) sampling.

The taxonomy has illustrated that *Bayesian quadrature* should be understood as an umbrella term (under Definition 2.1) and that Bayesian quadrature *methods* differ significantly in model assumption, inference method, sampling procedure and even algorithmic requirements and structure. They indeed range from methods that accept fixed datasets to agent-like methods that interact with the data collection mechanism. Figure 3.1 illustrates the taxonomy and Table 3.1 categorizes some existing BQ methods.

3.4 Other terms

This section lists terms that are occasionally used to describe some characteristics of Bayesian quadrature algorithms but are not part of our taxonomy. We list them for completeness.

- *Structured*: Bayesian quadrature is sometimes called structured¹⁴ when the model $\mathbf{f}$ encodes a known structural property, such

¹³The term *adaptive* has several additional meanings in numerical integration literature: in *adaptive quadrature* the integral is approximated by a sum of quadratures rules on sub-domains that are adaptively refined until a desired error tolerance is reached (Press *et al.*, 2007, Sec. 4.7), while *dimension-adaptive quadrature* attempts to find the dimensions which contribute most to the integral (Gerstner and Griebel, 2003). Although the modelling framework would be well-suited to the task, no dimension-adaptive Bayesian quadrature methods appear to have been developed so far.

¹⁴Our use of the term has nothing in common with *structured kernel interpolation* that is related to scalable GP regression (Wilson and Nickisch, 2015).

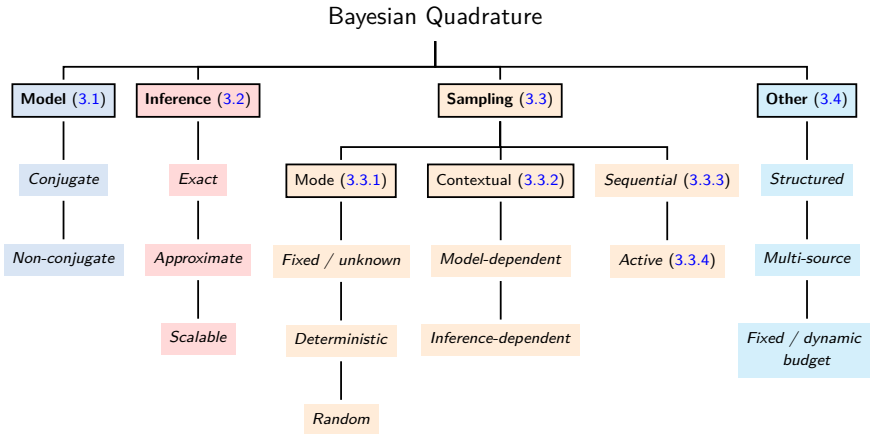


Figure 3.1: Taxonomy of Bayesian quadrature methods along three main axes: *Model*, *inference* and *sampling* as introduced in Chapter 3. The labels under *other* are listed for completeness. The labels in each axis do not follow a hierarchy and are not necessarily mutually exclusive. Section numbers are given in parentheses.

as an invariance or range constraint, of the integrand f via the transformation φ or otherwise. This is to highlight that the method is designed to address certain prior information that may be available about f . For example, Naslidnyk *et al.* (2021) encode input invariances of the integrand f in a conjugate BQ model. Chai and Garnett (2019) and Gunter *et al.* (2014) use exponential and square transforms φ , respectively, to encode non-negativity in a non-conjugate model (see Section 2.2.2).

- *Fixed & dynamic budget:* The computational budget of a Bayesian quadrature algorithm is *fixed* if the total number of integrand evaluations N is determined before the algorithm runs. This is the case if N is set by the practitioner by, for example, providing a fixed set of nodes $\mathbf{x}_1, \dots, \mathbf{x}_N$ or by providing N directly alongside a sampler. The budget is *dynamic* if the number of evaluations of f is not known before run time, and the algorithm determines an appropriate N automatically.¹⁵ Methods with a dynamic budget

¹⁵Apart from a safeguard condition that caps the number of f -evaluations at some large $N_{\max}$.

are sometimes called *automatic* as for example in Jagadeeswaran and Hickernell (2019), Jagadeeswaran and Hickernell (2022), and Jagadeeswaran (2019). A dynamic budget does not imply that node selection is active. Whether a fixed or dynamic budget is preferable in practice depends on the requirements and constraints of the application.

- *Multi-source*: Multi-source Bayesian quadrature is an umbrella term that implies that not only one, but multiple, usually related integrals are being considered simultaneously. Several settings exist: *Multi-fidelity* BQ assumes a natural ordering of the integrals (e.g., from cheap to expensive) and usually aims to infer the integral with the highest fidelity (e.g., the most expensive one) from integrand evaluations of all fidelity levels. The ordering can either be encoded in the model directly (Li *et al.*, 2023) or implicitly via the sampling strategy (Gessner *et al.*, 2020). *Multi-source* BQ (Gessner *et al.*, 2020) generalizes multi-fidelity BQ in that it aims to learn a primary integral but does not assume a natural ordering of the related integrals. Xi *et al.*, 2018 assume no primary integral and aim to learn all related integrals equally well. *Active* multi-source BQ (Gessner *et al.*, 2020) can, via its acquisition function, naturally be tailored towards aiming to learn a primary, some, or all related integrals. Other BQ methods that feature multiple integrals include *Bayesian quadrature optimization* methods which seek to optimise functions of the form $g(\mathbf{y}) = \int_D f(\mathbf{x}, \mathbf{y}) dP(\mathbf{x})$ (Nguyen *et al.*, 2020; Iwazaki *et al.*, 2021; Sadeghi *et al.*, 2022; Toscano-Palmerin and Frazier, 2022) and the conditional BQ method of Chen *et al.* (2024) that estimates integrals with respect to a parametric family of distributions $\{P_\theta\}_{\theta \in \Theta}$. While multi-source BQ does not strictly comply with Definition 2.1, resulting algorithms fit well into our taxonomy.

3.5 Example

Here we present an example of a BQ method and examine how modifying the method would affect its classification under the taxonomy.

Method	Model	Inference	Sampling	Reference
Bayesian Monte Carlo (BMC)	Conjugate	Exact	Random	Rasmussen and Ghahramani, 2002
Warped sequential active Bayesian integration (WSABI)	Non-conjug. Structured	Approx.	Deterministic Sequential Active (expl.)	Gunter <i>et al.</i> , 2014
BQ for multiple related integrals	Conjugate Multi-output	Exact	Random or Deterministic	Xi <i>et al.</i> , 2018
Variational BMC	Conjugate	Exact	Deterministic Sequential Active (expl.)	Acerbi, 2018a; Acerbi, 2018b
Fully symmetric kernel quadrature	Conjugate	Exact Scalable	Deterministic Infer-depend.	Karvonen and Särkkä, 2018
Fast automatic BQ	Conjugate	Exact Scalable	Deterministic Infer-depend. Sequential	Jagadeeswaran and Hickernell, 2019
Active multi-source BQ	Conjugate Multi-output	Exact	Deterministic Sequential Active (impl.)	Gessner <i>et al.</i> , 2020
Locally adaptive BQ	Conjugate	Exact	Deterministic Sequential Active (impl.)	Fisher <i>et al.</i> , 2020
Invariant BQ	Conjugate Structured	Exact	Deterministic Sequential Active (impl.)	Naslidnyk <i>et al.</i> , 2021
Bayesian alternately subsampled quadrature	Non-conjug.	Approx. Scalable	Random Model-depend.	Adachi <i>et al.</i> , 2022

Table 3.1: Some well-known BQ methods categorized according to the taxonomy of Chapter 3.

The method is only a conceptual example. We have not implemented it and there may be practical issues that render it impractical. Proper numerical experiments are found in Chapter 6.

Let $D = [0, 1]^d$ and suppose that integration problem is to compute $I_P(f) = \int_D f(\mathbf{x}) d\mathbf{x}$ for some integrand function $f: D \rightarrow \mathbb{R}$. Our illustrative BQ method is the following:

Model: To model f , we select the zero-mean GP model $\mathbf{f} \sim \text{GP}(0, k_\theta)$ whose covariance function is the square exponential kernel in (2.3) and $\theta = (\sigma, \ell)$ are the kernel parameters. This choice means that the method is *conjugate* (Section 3.1).

Sampling: Let $\{x_1, \dots, x_n\}^d \subset [0, 1]^d$ be the d -ary Cartesian product of a set points $\{x_1, \dots, x_n\} \subset [0, 1]$. Select the number of iterations $i_{\max} \geq 1$ and sample the nodes by iterating the following steps for $i = 1, \dots, i_{\max}$:

1. Denote $X(x) = \{x_1, \dots, x_{i-1}, x\}^d \subset [0, 1]^d$ and solve the optimization problem

$$x_i = \arg \max_{x \in [0, 1]} \rho^2(x) \quad (3.2)$$

for

$$\rho^2(x) = \mathbf{k}_{\boldsymbol{\theta}, PX(x)}^\top \mathbf{K}_{\boldsymbol{\theta}, X(x)X(x)}^{-1} \mathbf{k}_{\boldsymbol{\theta}, PX(x)} / k_{\boldsymbol{\theta}, PP}.$$

That the quantities defined in Section 2.2.1 depend on $\boldsymbol{\theta}$ is made explicit here.

2. Set $X = X(x_i)$ and $\mathcal{D} = \{(\mathbf{x}, f(\mathbf{x}))\}_{\mathbf{x} \in X}$ and note that X contains $N_i = i^d$ points. In the notation of Algorithm 3, $X_{\text{batch}} = \{x_1, \dots, x_i\}^d \setminus \{x_1, \dots, x_{i-1}\}^d$ and the batch size $n = i^d - (i-1)^d = N_i - N_{i-1}$ depends on the step.
3. Use maximum likelihood to update the kernel parameters (see Section 4.3):

$$\boldsymbol{\theta} = \arg \max_{\boldsymbol{\theta} \in \Theta} \left\{ -\frac{1}{2} \mathbf{f}_X^\top \mathbf{K}_{\boldsymbol{\theta}, XX}^{-1} \mathbf{f}_X - \frac{1}{2} \log \det \mathbf{K}_{\boldsymbol{\theta}, XX} \right\},$$

where Θ is a suitable subset of $[0, \infty) \times (0, \infty)$.

The method is *sequential* and *batched* (Section 3.3.3). It is also *deterministic* (Section 3.3.1) because a perfect implementation would contain no randomness, though in practice there may be a degree of randomness in the optimization performed during Steps 1 and 3. Because x_i in Step 1 depends on the model $\mathbf{f}$ via the covariance function k , the method is *model-dependent* (Section 3.3.2). In fact, as we shall see in Section 4.2.3, on each step the optimization problem (3.2) can be written in the form (3.1), which makes the method *active* (and, in this case, *myopic*; see Section 3.3.3). Since there is no “direct” dependency on the observations $\mathbf{f}_X$ in Step 1, the method is not explicitly active. But because ρ^2 depends on

the kernel parameters, which in turn depend on $\mathbf{f}_X$ via Step 3, the method is *implicitly active*.

Inference: After the nodes have been sampled, the method returns the posterior $I_P(\mathbf{f}) \mid \mathcal{D} \sim \mathcal{N}(\mu_{\mathcal{D}}, \Sigma_{\mathcal{D}})$ by computing $\mu_{\mathcal{D}}$ and $\Sigma_{\mathcal{D}}$ from (2.9) and (2.11). Because both $k_{\theta,P}(\mathbf{x})$ and $k_{\theta,PP}$ are available in closed form for the square exponential kernel and the uniform measure on $[0, 1]^d$ (see Section 4.1), inference is *exact* (Section 3.2) potential numerical problems are ignored. The method becomes *scalable* (Section 3.2) but remains exact if one does not naively solve (2.9) and (2.11) but instead uses the well-known Kronecker decompositions of $\mathbf{K}_{\theta,XX}$ and $\mathbf{k}_{\theta,PX}$ for Cartesian product nodes and product kernels, such as the square exponential (O’Hagan, 1991, Sec. 4). Sampling would no longer be model-dependent, active, or sequential should the method be modified so that X were simply some pre-specified product design, such as the d -ary product of equispaced points on $[0, 1]$. However, this sampling could be considered *inference-dependent* (Section 3.3.2) as a product design is needed for scalable inference.

Next we turn to practical issues, many of them present in the method above, that one needs to negotiate in order to successfully design and implement a BQ method.

4

Practical issues

In this chapter we describe a number of practical issues that one encounters and how to solve them when implementing Bayesian quadrature algorithms. Some, such as numerical ill-conditioning and the need for kernel parameter estimation, need to be taken into account in all applications of Gaussian process regression and interpolation, while other, in particular the computation of kernel means and variances, are intrinsic to Bayesian quadrature. We conclude the section with an overview of what we view as the main limitations of Bayesian quadrature and how these have affected—and continue to affect—algorithmic development and the range of integration problems that Bayesian quadrature can or should solve.

4.1 Kernel means and variances

Conjugate Bayesian quadrature requires the computation of $\mu_{\mathcal{D}}$ and $\Sigma_{\mathcal{D}}$ as in Eq. (2.13) in order to characterize $I_P(f) \mid \mathcal{D}$. Eq. (2.13) poses the challenge of computing the vector $\mathbf{k}_{PX} = (k_P(\mathbf{x}_1), \dots, k_P(\mathbf{x}_N)) \in \mathbb{R}^N$ of kernel means in Eq. (2.10) and the scalar kernel variance k_{PP} in Eq. (2.12). A straightforward approach is to use a kernel k that admits an analytic solution of both k_{PP} and $k_P(\mathbf{x})$. A well-known example is

the square exponential kernel in Eq. (2.3) integrated with respect to the standard Lebesgue measure on the domain $D = [0, 1]^d$, which, for $d = 1$ yields

$$k_P(x) = \left(\frac{\pi\ell^2}{2}\right)^{\frac{1}{2}} \left[\operatorname{erf}\left(\frac{1-x}{\ell\sqrt{2}}\right) + \operatorname{erf}\left(\frac{x}{\ell\sqrt{2}}\right) \right],$$

$$k_{PP} = 2\ell^2(e^{-\frac{1}{2\ell^2}} - 1) + (2\pi\ell^2)^{\frac{1}{2}} \operatorname{erf}\left(\frac{1}{\ell\sqrt{2}}\right).$$

Comprehensive lists of closed form kernel embeddings can be found in Briol *et al.* (2025) and Hennig *et al.* (2022, Section 10.1). The former is accompanied with a kernel mean embedding library in Python¹. A variety of kernel embeddings are also implemented in the Python libraries EmuKit (Paleyes *et al.*, 2019; Paleyes *et al.*, 2023) and ProNum (Wenger *et al.*, 2021). Approximations to non-analytic kernel embeddings for use in BQ have been less explored in the literature. Notable exceptions are Warren *et al.* (2022) and Warren and Ramos (2024), who use fast Fourier transforms to approximate kernel means of stationary kernels. See also Tronarp *et al.* (2018) for a different approximation for isotropic Matérns.

Non-conjugate BQ often requires the computation of further quantities, such as $\int_D \int_D k^2(\mathbf{x}, \mathbf{x}') dP(\mathbf{x}) dP(\mathbf{x}')$, the integral of the kernel squared, and other combinations of kernel products (e.g., WSABI in Gunter *et al.*, 2014). Hence, methods based on non-conjugate BQ models are often specific to measure-kernel combinations which meet these requirements. Such integrals are also required in application of Bayesian quadrature to non-linear Kalman filtering (Deisenroth *et al.*, 2009; Deisenroth *et al.*, 2012; Särkkä *et al.*, 2014; Kersting and Hennig, 2016; Prüher and Šimandl, 2016; Prüher and Straka, 2018; Prüher *et al.*, 2021).

4.2 Sampling and node selection

Section 3.3 described how the nodes may be sampled (i.e., selected). This section details practical choices and issues one is faced with when designing a sampling procedure for BQ.

¹https://github.com/mmahsereci/kernel_embedding_dictionary

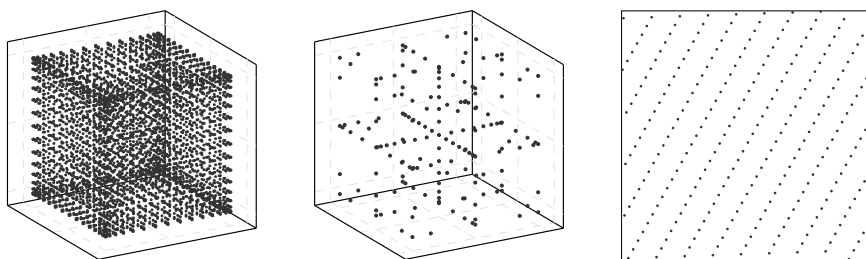


Figure 4.1: Three deterministic sampling designs mentioned in Section 4.2.1: Tensor grid ($d = 3$), sparse grid ($d = 3$), and a lattice ($d = 2$).

4.2.1 Model-independent deterministic designs

If computational or other reasons prevent active sampling and better coverage of the domain than what is achievable via random sampling is desired, it is common to use deterministic designs that are model-independent² (see Section 3.3.2). If the domain is a hypercube (say, $D = [0, 1]^d$), arguably the simplest deterministic design is the regular *tensor grid*

$$\begin{aligned} X_{N_1}^d &= X_{N_1} \times \cdots \times X_{N_1} \\ &= \left\{ \frac{1}{N_1 + 1}, \dots, \frac{N_1}{N_1 + 1} \right\} \times \cdots \times \left\{ \frac{1}{N_1 + 1}, \dots, \frac{N_1}{N_1 + 1} \right\}, \end{aligned} \quad (4.1)$$

which consists of $N = N_1^d$ nodes. More advanced designs are obtained by replacing each one-dimensional node sets X_{N_1} with the nodes of an appropriate Gaussian quadrature rule, such as Gauss–Hermite if P is standard Gaussian on $\mathbb{R}^d$ or Gauss–Legendre if P is uniform on $[0, 1]^d$. The resulting designs are sometimes called *classical* in the BQ literature. The computation of these nodes is efficiently implemented by most scientific computing libraries, including SciPy for Python. As the number of points depends exponentially on the dimension, tensor grids are inflexible and difficult to use in high dimensions. *Sparse grids* (Karvonen and Särkkä, 2018, Sec. 4.2) as well as *lattice nodes* (Jagadeeswaran and

²However, it is always good to keep in mind some properties of the prior when selecting the nodes. For example, equispaced nodes are not well suited for approximation of infinitely differentiable functions (Platte *et al.*, 2011). One should avoid equispaced nodes if the kernel is, for example, the square exponential.

Hickernell, 2019) and other designs from quasi-Monte Carlo literature may be used in high dimensions. Some of the above designs are depicted in Figure 4.1 for $d \in \{2, 3\}$. It is a great advantage of Bayesian quadrature that one may use any nodes that have been proposed in the literature.

In using *non-nested* nodes, many deterministic designs implicitly assume that the computational budget is fixed (see Section 3.4). For example, $X_{N_1} \not\subset X_{N_1+1}$ if X_{N_1} are the equispaced nodes in (4.1) or any classical Gaussian quadrature nodes (nested modifications of which exist; see Davis and Rabinowitz, 1984, Sec. 2.7.1.1). A non-nested design can be used as an initial design in sequential Bayesian quadrature where more nodes are sampled (e.g., actively) until some stopping criterion is met. *Nested* deterministic designs include the extensible integration lattices used in Jagadeeswaran and Hickernell (2019) and the Clenshaw–Curtis nodes (Trefethen, 2008) when N is a power of two (i.e., $X_{2^n} \subset X_{2^{n+1}}$).

4.2.2 Random nodes

An early instance of a BQ method—Bayesian Monte Carlo (Rasmussen and Ghahramani, 2002)—uses nodes that are random samples from the integration measure P . As such it puts higher value on observing the integrand in areas where the density p of P is large than where it is small. This sampling method is model-uninformed. Adachi *et al.* (2022) recently proposed a sampling distribution for BQ that is inspired by importance sampling and also covers areas where f is supposedly large (even if the density p is small). This sampling method, in contrast, is model-dependent. Purposely sampling the nodes from a distribution different from P has also been proposed and studied (Briol *et al.*, 2017; Bach, 2017).

The space-filling Latin hypercube design (McKay *et al.*, 1979) is a random, scalable equivalent to a grid. Being model-independent, it is used as initial design in sequential active BQ methods. Determinantal point processes, used in BQ by Belhadji *et al.* (2019), provide model-dependent random designs constructed using the kernel function. Nodes sampled from a determinantal point process are repulsive and space-filling.

4.2.3 Actively sampled nodes

As promised in Section 3.3.4, this section reviews some commonly used utility functions for active BQ and their corresponding acquisition functions. For the conjugate model we give explicit expressions whenever possible. Recall from Section 3.3.4 that active nodes maximize an expected utility according to Eq. (3.1). The equation is repeated here for convenience:

$$\mathbf{x}_1, \dots, \mathbf{x}_N \in \arg \max_{\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N} \mathbb{E}_{\mathbf{f}}[u(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N \mid \mathbf{f}, P)].$$

Again, the function u is the utility function and

$$a(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N \mid P) := \mathbb{E}_{\mathbf{f}}[u(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N \mid \mathbf{f}, P)]$$

the corresponding acquisition function. The above equation, at heart, considers what would happen under the model $\mathbf{f}$ if the integrand f were hypothetically evaluated at some yet unseen nodes $\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N$, expressed quantitatively as an average utility a . The active sampler returns the *maximizer* of a . In other words, the sampler produces nodes that yield the maximal average utility under the model $\mathbf{f}$. A question of course arises: How to choose u ?

From an information theoretic perspective, a meaningful acquisition function is the *mutual information* (MI) between the integral $I_P(\mathbf{f})$ and yet unseen integrand values $\mathbf{f}_{\tilde{X}} := (f(\tilde{\mathbf{x}}_1), \dots, f(\tilde{\mathbf{x}}_N)) \in \mathbb{R}^N$ at nodes $\tilde{X} = \{\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N\}$. Let $\tilde{\mathcal{D}} = \{(\tilde{\mathbf{x}}_1, f(\tilde{\mathbf{x}}_1)), \dots, (\tilde{\mathbf{x}}_N, f(\tilde{\mathbf{x}}_N))\}$ be the corresponding hypothetical dataset. The MI acquisition function is

$$\begin{aligned} a(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N \mid P) &= \text{MI}(I_P(\mathbf{f}); \mathbf{f}_{\tilde{X}}) \\ &:= \mathbb{E}_{\mathbf{f}}[\text{KL}(\mathbf{f}_{\tilde{X}} \mid I_P(\mathbf{f}), \tilde{X} \parallel \mathbf{f}_{\tilde{X}} \mid \tilde{X})] \\ &= \mathbb{E}_{\mathbf{f}_{\tilde{X}}}[\text{KL}(I_P(\mathbf{f}) \mid \tilde{\mathcal{D}} \parallel I_P(\mathbf{f}))], \end{aligned}$$

where $\text{KL}(Q \parallel Q')$ is the Kullback–Leibler (KL) divergence between two distributions Q and Q' . The MI acquisition function measures the expected change in distribution between the integral $I_P(\mathbf{f})$ over the prior model and the integral $I_P(\mathbf{f}) \mid \tilde{\mathcal{D}}$ over the model that is condition on the hypothetical dataset $\tilde{\mathcal{D}}$. Maximizing it yields maximally informative nodes (in the Shannon sense) about the integral value under the model $\mathbf{f}$.

For conjugate Bayesian quadrature, the mutual information is analytic and has the form

$$\text{MI}(I_P(\mathbf{f}); \mathbf{f}_{\tilde{X}}) = -\frac{1}{2} \log\left(1 - \rho^2(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N)\right), \quad (4.2)$$

where

$$\rho^2(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N) := (\mathbf{k}_{P\tilde{X}}^\top \mathbf{K}_{\tilde{X}\tilde{X}}^{-1} \mathbf{k}_{P\tilde{X}}) / k_{PP} \in [0, 1] \quad (4.3)$$

is the scalar squared correction. Here k_{PP} is the initial variance defined in Eq. (2.12), $\mathbf{k}_{P\tilde{X}} = (k_P(\tilde{\mathbf{x}}_1), \dots, k_P(\tilde{\mathbf{x}}_N)) \in \mathbb{R}^N$ is a vector, and $\mathbf{K}_{\tilde{X}\tilde{X}} = (k(\tilde{\mathbf{x}}_i, \tilde{\mathbf{x}}_j))_{i,j=1}^N \in \mathbb{R}^{N \times N}$ a matrix. For example Chai *et al.* (2019) and Hamid *et al.* (2022) use information-theoretic acquisition functions.

Other frequently used BQ acquisition functions include the *integral variance reduction* (IVR) and the *negative integral variance* (NIV) mentioned in Section 3.3.4 that arise from the square error utility:

$$\begin{aligned} \text{IVR}(I_P(\mathbf{f}); I_P(\mathbf{f}) \mid \tilde{\mathcal{D}}) \\ &:= s^{-1} \mathbb{E}_{\mathbf{f}} \left[(I_P(\mathbf{f}) - \mathbb{E}_{\mathbf{f}}[I_P(\mathbf{f})])^2 - (I_P(\mathbf{f}) \mid \tilde{\mathcal{D}} - \mathbb{E}_{\mathbf{f}}[I_P(\mathbf{f}) \mid \tilde{\mathcal{D}}])^2 \right] \\ &= s^{-1} (s - \text{Var}[I_P(\mathbf{f}) \mid \tilde{\mathcal{D}}]) \end{aligned}$$

and

$$\begin{aligned} \text{NIV}(I_P(\mathbf{f}) \mid \tilde{\mathcal{D}}) &:= -s^{-1} \mathbb{E}_{\mathbf{f}} \left[(I_P(\mathbf{f}) \mid \tilde{\mathcal{D}} - \mathbb{E}_{\mathbf{f}}[I_P(\mathbf{f}) \mid \tilde{\mathcal{D}}])^2 \right] \\ &= -s^{-1} \text{Var}[I_P(\mathbf{f}) \mid \tilde{\mathcal{D}}]. \end{aligned}$$

The inverse of the factor $s = \text{Var}[I_P(\mathbf{f})]$ (not necessarily equal to k_{PP} for arbitrary $\mathbf{f}$) is a positive constant with respect to $\tilde{X}$ and serves as normalization. The IVR is simply the (normalized) difference between the variance s of the integral over the prior model and the variance of the integral over the model conditioned on the hypothetical dataset $\tilde{\mathcal{D}}$. Hence IVR selects nodes that shrink the variance of the integral maximally. Analogously, NIV chooses nodes that minimize the variance of the integral. The IVR and NIV yield the same maximizer and hence the same active nodes as both functions are strictly increasing transformations of each other. For the conjugate model, these two

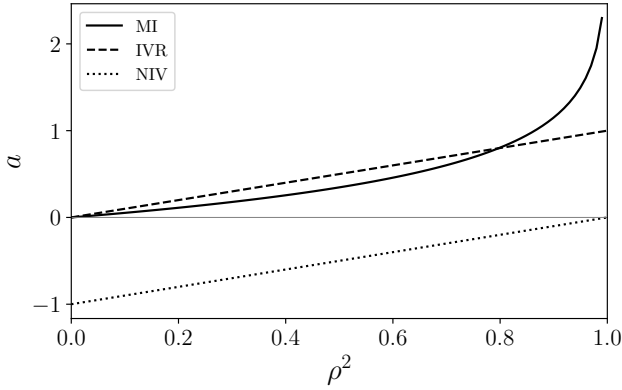


Figure 4.2: BQ acquisition functions for the conjugate model as functions of ρ^2 . The three acquisition functions are given in Eqs. (4.2), (4.4), and (4.5).

acquisition functions are analytic and simplify to

$$\text{IVR}(I_P(f); I_P(f) \mid \tilde{\mathcal{D}}) = \rho^2(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N) \in [0, 1], \quad (4.4)$$

$$\text{NIV}(I_P(f) \mid \tilde{\mathcal{D}}) = \rho^2(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N) - 1 \in [-1, 0]. \quad (4.5)$$

By comparing Eqs. (4.2), (4.4), and (4.5) it is apparent that, in the conjugate case, all three acquisition functions (IVR, NIV, MI) can be expressed as functions of the squared correlation ρ^2 and as strictly increasing transformations of each other. Hence all three yield the same nodes (Gessner *et al.*, 2020). Figure 4.2 shows the acquisitions as functions of ρ^2 .³

For *sequential* active sampling as introduced in Algorithms 3 and 4 in Sections 3.3.3 and 3.3.4, the formulas need only slight modification.

³Active sampling typically assumes that the cost of evaluating $f(\mathbf{x})$ is the same for all $\mathbf{x}$ making maximizing the acquisition function reasonable. However, if the cost varies with $\mathbf{x}$, an alternative approach may be preferable. The only one we are aware of in the context of BQ is due to Gessner *et al.* (2020) who maximize acquisition rates $r(\tilde{\mathbf{x}}) = a(\tilde{\mathbf{x}})/c(\tilde{\mathbf{x}})$ based on some positive cost function $c(\tilde{\mathbf{x}})$. For example, if a is MI and c measures the cost in time or monetary value, then r would yield nodes that maximize bits per time or bits per dollar. In this cost-sensitive scenario, even if $\arg \max a_1(\tilde{\mathbf{x}}) = \arg \max a_2(\tilde{\mathbf{x}})$ for two different acquisition functions a_1 and a_2 (such as IVR and NIV), generally $\arg \max r_1(\tilde{\mathbf{x}}) \neq \arg \max r_2(\tilde{\mathbf{x}})$, even if they share the same cost function c . In fact, certain acquisition functions, like NIV, cannot be converted to meaningful acquisition rates at all. Cost-sensitive evaluations further play a role in multi-source and multi-fidelity active BQ where the acquisition strategy requires careful consideration.

Recall that a sequential sampler produces a small batch of nodes of size n at every step of the iteration which is then added to the current dataset. Hence, denote the current dataset as $\mathcal{D}$ and the hypothetical batch dataset as $\tilde{\mathcal{D}} = \{(\tilde{\mathbf{x}}_1, \mathbf{f}(\tilde{\mathbf{x}}_1)), \dots, (\tilde{\mathbf{x}}_n, \mathbf{f}(\tilde{\mathbf{x}}_n))\}$, which overloads notation in this paragraph. Further, denote the current posterior as $\mathbf{f}_{\mathcal{D}} = \mathbf{f} \mid \mathcal{D}$. The acquisition function can be defined autoregressively (meaning that $\mathbf{f}_{\mathcal{D}}$ acts as the prior model in the current step) according to

$$\begin{aligned} \text{MI}(I_P(\mathbf{f}_{\mathcal{D}}); \mathbf{f}_{\tilde{X}}) &= \mathbb{E}_{\mathbf{f}_{\tilde{X}}} \left[\text{KL}(I_P(\mathbf{f}_{\mathcal{D}}) \mid \tilde{\mathcal{D}} \parallel I_P(\mathbf{f}_{\mathcal{D}})) \right], \\ \text{IVR}(I_P(\mathbf{f}_{\mathcal{D}}); I_P(\mathbf{f}_{\mathcal{D}}) \mid \tilde{\mathcal{D}}) &= s_{\mathcal{D}}^{-1} (s_{\mathcal{D}} - \text{Var}[I_P(\mathbf{f}_{\mathcal{D}}) \mid \tilde{\mathcal{D}}]), \\ \text{NIV}(I_P(\mathbf{f}_{\mathcal{D}}) \mid \tilde{\mathcal{D}}) &= -s_{\mathcal{D}}^{-1} \text{Var}[I_P(\mathbf{f}_{\mathcal{D}}) \mid \tilde{\mathcal{D}}], \end{aligned}$$

with $s_{\mathcal{D}} = \text{Var}[I_P(\mathbf{f}_{\mathcal{D}})]$ again a constant with respect to $\tilde{X}$ that only serves to normalize the acquisition functions. The acquisition functions are myopic by definition as they do not consider the impact of future steps. For the conjugate model the autoregressive squared correlation is

$$\rho^2(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_n) = (\mathbf{k}_{\mathcal{D}; P\tilde{X}}^{\top} \mathbf{K}_{\mathcal{D}; \tilde{X}\tilde{X}}^{-1} \mathbf{k}_{\mathcal{D}; P\tilde{X}}) / \Sigma_{\mathcal{D}},$$

where $\mathbf{k}_{\mathcal{D}; P\tilde{X}} = I_P(\mathbf{k}_{\mathcal{D}; \tilde{X}}) = (k_{\mathcal{D}; P}(\tilde{\mathbf{x}}_1), \dots, k_{\mathcal{D}; P}(\tilde{\mathbf{x}}_n))$ is the kernel mean embedding of the posterior kernel function as in Eq. (2.2) as a function of the n hypothetical nodes, $\mathbf{K}_{\mathcal{D}; \tilde{X}\tilde{X}}$ is the corresponding Gram matrix, and $\Sigma_{\mathcal{D}}$ is the integral variance in Eq. (2.11). For the prevalent case $n = 1$, the sequential squared correlation simplifies to

$$\rho^2(\tilde{\mathbf{x}}) = \frac{k_{\mathcal{D}; P}(\tilde{\mathbf{x}})^2}{\Sigma_{\mathcal{D}} k_{\mathcal{D}}(\tilde{\mathbf{x}}, \tilde{\mathbf{x}})}.$$

We mention two acquisition functions that are only indirectly related to the integral. *Uncertainty sampling* (US), popular in experimental design, is defined for $n = 1$ only by

$$\text{US}(\mathbf{f}_{\mathcal{D}}) := \mathbb{E}_{\mathbf{f}} \left[(\mathbf{f}_{\mathcal{D}}(\tilde{\mathbf{x}}) - \mathbb{E}_{\mathbf{f}_{\mathcal{D}}}[\mathbf{f}_{\mathcal{D}}(\tilde{\mathbf{x}})])^2 p^2(\tilde{\mathbf{x}}) \right] = \text{Var}[\mathbf{f}_{\mathcal{D}}(\tilde{\mathbf{x}})] p^2(\tilde{\mathbf{x}}).$$

For the conjugate model we get $\text{US}(\mathbf{f}_{\mathcal{D}}) = k_{\mathcal{D}}(\tilde{\mathbf{x}}, \tilde{\mathbf{x}}) p^2(\tilde{\mathbf{x}})$. Uncertainty sampling yields nodes at locations where the variance of $\mathbf{f}_{\mathcal{D}}$ and the density of the integration measure $p(\tilde{\mathbf{x}})$ are both large. By focusing on the variance of the model $\mathbf{f}$ instead of its integral $I_P(\mathbf{f})$, US in a

sense disregards the integral as the quantity of interest and may hence be more tractable or accessible than many other acquisition functions. Uncertainty sampling is for example used in Gunter *et al.* (2014) and in a related (non-active) context in Adachi *et al.* (2022). The *posterior variance contribution* (PVC) acquisition function (Wei *et al.*, 2020)

$$\text{PVC}(f_{\mathcal{D}}) := k_{\mathcal{D};P}(\tilde{\mathbf{x}})p(\tilde{\mathbf{x}})$$

is a heuristic defined only for the conjugate model. Like US, PVC yields nodes likely to contribute much to the integral variance, but ignores spatial correlations between candidate nodes. One can also proceed to the opposite direction and use an acquisition function developed for Bayesian quadrature to select evaluation locations for a purpose other than integration (e.g., Paul *et al.*, 2018; Adachi *et al.*, 2023a; Adachi *et al.*, 2023b; Adachi *et al.*, 2024).

Implementations of sequential active sampling loops and BQ acquisition functions for $n = 1$ can for example be found in the Python library EmuKit (Paleyes *et al.*, 2019; Paleyes *et al.*, 2023) which we use in the experiments in Chapter 6.

Recall from Section 2.3 that BQ is equivalent to worst-case optimal integration in the RKHS $\mathcal{H}(k)$ of the covariance function. This and related equivalences allow expressing all acquisition functions above as quantities involving different worst-case errors. For example, we saw that the MI, IVR and NIV acquisition functions yield nodes that maximize the squared correlation in Eq. (4.3). Equivalently, they minimize the integral variance $\Sigma_{\tilde{\mathcal{D}}} = k_{PP} - \mathbf{k}_{P\tilde{X}}^{\top} \mathbf{K}_{\tilde{X}\tilde{X}}^{-1} \mathbf{k}_{P\tilde{X}}$. By Proposition 2.6, for given nodes the integral variance equals the smallest possible squared worst-case error in $\mathcal{H}(k)$ obtainable by a quadrature rule using these nodes. Nodes $\mathbf{x}_1, \dots, \mathbf{x}_N$ sampled by MI, IVR or NIV are thus worst-case optimal:

$$\mathbf{x}_1, \dots, \mathbf{x}_N \in \arg \min_{\mathbf{x}_1, \dots, \tilde{\mathbf{x}}_N} \min_{\mathbf{v} \in \mathbb{R}^N} e_k(\mathbf{v}, \tilde{X}), \quad (4.6)$$

where

$$e_k(\mathbf{v}, \tilde{X}) = \sup_{\|g\|_{\mathcal{H}(k)} \leq 1} \left| \int_D g(\mathbf{x}) \, dP(\mathbf{x}) - \sum_{i=1}^N v_i g(\tilde{\mathbf{x}}_i) \right|.$$

These nodes provide maximal information for integration in $\mathcal{H}(k)$. Accordingly, *optimal* Bayesian quadrature is sometimes used to refer to BQ with the nodes in Eq. (4.6). These nodes are called optimal also in numerical analysis and approximation theory, where the corresponding worst-case error is known as the N th minimal error; see Chapters 2 and 3 in Novak and Woźniakowski (2008) and Chapter 5 in Oettershagen (2017).

4.3 Kernel parameter estimation

In Bayesian inference one is confronted with the challenging task of selecting an appropriate prior. While other options exist, in Gaussian process modelling the prevalent approach is to select a parametrised kernel, such as the square exponential in (2.3) or a Matérn kernel in (2.4) with a fixed smoothness parameter ν , and estimate its parameters from the data (Rasmussen and Williams, 2006, Ch. 5). Most Bayesian quadrature methods use maximum likelihood estimation to set the kernel parameters. Marginalisation and cross-validation are also occasionally used (Briol *et al.*, 2019; Jagadeeswaran and Hickernell, 2019). As our experiments in Chapter 6 use maximum likelihood estimation, we briefly review it in the context of conjugate Bayesian quadrature; that is, when $\mathbf{f} = \mathbf{g} \sim \text{GP}(m, k_{\theta})$ for a kernel with parameters θ . As the conjugate Bayesian quadrature integral variance in (2.12) does not directly depend on the integrand evaluations $\mathbf{f}_X$, estimation of the kernel or its parameters from the data is essential if the variance is to measure integration uncertainty.

Let θ be the kernel parameters and let the zeroth parameter $\theta_0 = \sigma^2$ denote the kernel scale. Let $\mathcal{D}$ be the dataset consisting of the nodes $X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ and corresponding observations $\mathbf{f}_X$. *Maximum likelihood* (ML) estimation is a popular way to fit kernel parameters. The ML estimate of θ is

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} L(\theta; \mathcal{D}) = \arg \max_{\theta} p(\mathbf{f}_X \mid X, \theta),$$

where $L(\theta; \mathcal{D})$ is the likelihood of the parameters. From a probabilistic perspective, the ML estimator maximizes the probability $p(\mathbf{f}_X \mid X, \theta)$

that the given dataset is observed under the marginalized model $\mathbf{f}$. Equivalently, the estimator $\hat{\boldsymbol{\theta}}_{ML}$ maximizes the log-likelihood $\log L(\boldsymbol{\theta}; \mathcal{D})$. For the conjugate BQ model the logarithm admits the closed form

$$\begin{aligned} \log L(\boldsymbol{\theta}; \mathcal{D}) = & -\frac{1}{2}(\mathbf{f}_X - \mathbf{m}_X)^\top \mathbf{K}_{XX}^{-1}(\boldsymbol{\theta})(\mathbf{f}_X - \mathbf{m}_X) \\ & -\frac{1}{2} \log \det(\mathbf{K}_{XX}(\boldsymbol{\theta})) - \frac{N}{2} \log 2\pi, \end{aligned} \quad (4.7)$$

where $\det(\mathbf{K}_{XX}(\boldsymbol{\theta}))$ is the determinant of the Gram matrix $\mathbf{K}_{XX}(\boldsymbol{\theta})$ and the dependence on $\boldsymbol{\theta}$ has been made explicit. Combining the two equations yields

$$\begin{aligned} \hat{\boldsymbol{\theta}}_{ML} = \arg \max_{\boldsymbol{\theta}} \left\{ & -\frac{1}{2}(\mathbf{f}_X - \mathbf{m}_X)^\top \mathbf{K}_{XX}^{-1}(\boldsymbol{\theta})(\mathbf{f}_X - \mathbf{m}_X) \right. \\ & \left. -\frac{1}{2} \log \det(\mathbf{K}_{XX}(\boldsymbol{\theta})) \right\}. \end{aligned} \quad (4.8)$$

In general, an analytic solution of Eq. (4.8) only exists for the scale parameter

$$\hat{\sigma}_{ML}^2 = \frac{1}{N}(\mathbf{f}_X - \mathbf{m}_X)^\top \mathbf{K}_{XX}^{-1}(\boldsymbol{\theta}_0)(\mathbf{f}_X - \mathbf{m}_X),$$

where $\boldsymbol{\theta}_0 = (\theta_0 = 1, \theta_1, \dots)$, so that $\mathbf{K}_{XX}(\boldsymbol{\theta}_0)$ is the unscaled kernel Gram matrix. If all parameters $\boldsymbol{\theta}$ need to be estimated, Eq. (4.8) can be solved by an off-the-shelf optimizer as gradients of $\log L(\boldsymbol{\theta}; \mathcal{D})$ with respect to $\boldsymbol{\theta}$ are usually available.

The kernel parameters $\boldsymbol{\theta}$ are also called *hyper-parameters* to distinguish them from the (possibly infinite) parameters of the model $\mathbf{f}$. Thus, maximizing $L(\boldsymbol{\theta}; \mathcal{D})$ is also called *maximum likelihood type II* (instead of the standard type I). Other common descriptors are *maximum marginal likelihood* [since $p(\mathbf{f}_X | X, \boldsymbol{\theta})$ describes a marginal probability], or *evidence maximization* [as $p(\mathbf{f}_X | X, \boldsymbol{\theta})$ is the evidence of the probabilistic model].

As $\hat{\boldsymbol{\theta}}_{ML}$ is an estimator based on a limited dataset $\mathcal{D}$ overfitting can occur. As a consequence, BQ methods with sequential active node selection as in Algorithms 3 and 4 that compute the ML-estimate $\hat{\boldsymbol{\theta}}_{ML}$ at every step anew require initial designs of appropriate size that are independent of the model $\mathbf{f}$. This is conceptually analogous to initial

designs that are required by sequential active experimental design algorithms and Bayesian optimization.

4.4 Numerical ill-conditioning

In practically all variants of Bayesian quadrature the computation of the posterior $I_P(f) \mid \mathcal{D}$ and the fitting of the kernel parameters require one to solve—repeatedly in the case of parameter fitting—the linear system $\mathbf{K}_{XX}\mathbf{z} = \mathbf{a}$ for some vector $\mathbf{a} \in \mathbb{R}^N$. Unless the kernel or the nodes have properties that can be exploited to simplify this task, one has to use some standard linear solver, the accuracy of which depends on floating-point accuracy and the magnitude of the condition number of the kernel Gram matrix,

$$\text{cond}(\mathbf{K}_{XX}) = \frac{\lambda_{\max}(\mathbf{K}_{XX})}{\lambda_{\min}(\mathbf{K}_{XX})}.$$

Here $\lambda_{\max}(\mathbf{A})$ and $\lambda_{\min}(\mathbf{A})$ denote the maximal and minimal eigenvalues of a symmetric positive-definite matrix $\mathbf{A}$. When the condition number is large, solving the linear system is prone to numerical errors and hence numerically unstable. The condition numbers of commonly used kernels can grow fast when the nodes fill the domain. Let $D = [0, 1]^d$ and

$$h = \min_{i=1,\dots,N} \sup_{\mathbf{x} \in [0,1]^d} \min_{j \neq i} \|\mathbf{x} - \mathbf{x}_j\|,$$

which quantifies how well the nodes fill the domain, being the radius of the largest open ball in $[0, 1]^d$ that intersects no subset of X containing $N - 1$ nodes. This quantity is closely related to the fill-distance that we shall use in Chapter 5; see Eq. (5.2). The following theorem that links h to $\text{cond}(\mathbf{K}_{XX})$ may be found in Schaback (1995). See also Wendland (2005, Ch. 12), Bachoc and Furrer (2016), and Diederichs and Iske (2019). The general principle is that the smoother the kernel is, the faster the condition number grows as h tends to zero, which is to say that the nodes fill the domain.

Theorem 4.1. Let $D = [0, 1]^d$. If k is a Matérn kernel in (2.4) with smoothness $\nu > 0$, then

$$\text{cond}(\mathbf{K}_{XX}) \geq Ch^{-\nu},$$

where $C > 0$ depends only on σ , ℓ and ν . If k is a square exponential kernel in (2.3), then there is a constant $c > 0$, which depends only on σ and ℓ , such that

$$\text{cond}(\mathbf{K}_{XX}) \geq \exp(-ch \log h).$$

The assumption $D = [0, 1]^d$ can be relaxed as described in Remark 5.2. Numerical ill-conditioning issues are likewise present in Gaussian process interpolation and kriging, where they are usually dealt with by introducing a parameter $\lambda > 0$ called the *nugget* (alternatively, *regularisation parameter* or *jitter*) and replacing each occurrence of the Gram matrix $\mathbf{K}_{XX}$ with $\mathbf{K}_{XX} + \lambda \mathbf{I}_N$, where $\mathbf{I}_N$ is the $N \times N$ identity matrix (Neil, 1999; Rasmussen and Williams, 2006, Sec. 3.4.3; Ranjan *et al.*, 2011; Adrianakis and Challenor, 2012). Gramacy and Lee (2012) are strongly in favour of nuggets. If the kernel is stationary and $k(\mathbf{x}, \mathbf{x}) = 1$ for every $\mathbf{x} \in D$, it is straightforward to compute that

$$\text{cond}(\mathbf{K}_{XX} + \lambda \mathbf{I}_N) = \frac{\lambda_{\max}(\mathbf{K}_{XX}) + \lambda}{\lambda_{\min}(\mathbf{K}_{XX}) + \lambda} \leq \frac{N + \lambda}{\lambda},$$

which means that the condition number may increase at most linearly in the number of nodes. In the context of conjugate Bayesian quadrature, the use of a nugget corresponds to replacing the mean $\mu_{\mathcal{D}}$ and variance $\Sigma_{\mathcal{D}}$ in (2.9) and (2.11) with their regularised versions

$$\begin{aligned} \mu_{\mathcal{D}, \lambda} &= m_P + I_P(\mathbf{k}_X)^\top (\mathbf{K}_{XX} + \lambda \mathbf{I}_N)^{-1} (\mathbf{f}_X - \mathbf{m}_X), \\ \Sigma_{\mathcal{D}, \lambda} &= k_{PP} - \mathbf{k}_{PX}^\top (\mathbf{K}_{XX} + \lambda \mathbf{I}_N)^{-1} \mathbf{k}_{PX}. \end{aligned} \quad (4.9)$$

The nugget can be seen as an additional hyper-parameter and fitted alongside other parameters as described in Section 4.3.

Theorem 4.1 shows that nuggets are necessary when N is sufficiently large. However, in Chapter 5 we shall see that the nugget has a limiting effect on the accuracy of conjugate Bayesian quadrature, which suggests that its use ought to be avoided when very high accuracy is being sought. That is, there is a trade-off between numerical stability and accuracy known as *Schaback's uncertainty principle* (Schaback, 1995).⁴

⁴This principle only applies when one uses the basis $\{k(\cdot, \mathbf{x}_i)\}_{i=1}^N$ of kernel translates. Numerically more stable bases exist (Müller and Schaback, 2009; Fasshauer, 2011, Sec. 7.1).

Whether to use a nugget in Bayesian quadrature therefore depends on the modelling choices, the desired accuracy of the particular method, and the required level of code robustness. Most GP libraries hardcode a minimal *default nugget* that may depend on the type (at the time of writing GPy uses $\lambda = 10^{-8}$ while GPytorch uses $\lambda = 10^{-6}$ for floats and $\lambda = 10^{-8}$ for doubles) plus possibly a *dynamic nugget* that depends on the Gram matrix $\mathbf{K}_{XX}$ and is increased successively if solving the linear system fails. Section 6.1.3 describes how nuggets are used in our experiments.

4.5 Prior mean estimation

The prevalent approach in the literature on BQ is to use a zero-mean prior $\mathbf{g} \sim \text{GP}(0, k)$. However, like the kernel or its parameters, also the prior mean m can be estimated from the data. Suppose that we are in the conjugate setting described in Section 2.2.1 where $\mathbf{f} = \mathbf{g}$. The prior mean is typically estimated by postulating a parametric form

$$m(\mathbf{x}) = \sum_{i=1}^q \beta_i g_i(\mathbf{x})$$

for $q \in \mathbb{N}$ fixed basis functions $g_i: D \rightarrow \mathbb{R}$ and coefficients $\boldsymbol{\beta} = (\beta_1, \dots, \beta_q) \in \mathbb{R}^q$ which are assigned some prior.⁵ A common approach is to place a non-informative prior $p(\boldsymbol{\beta}, \sigma^2) \propto \sigma^{-2}$ on $\boldsymbol{\beta}$ and the kernel scale parameter σ^2 and subsequently marginalize out these parameters (O’Hagan, 1991). Doing so yields an integral posterior which is Student’s t with $N - q$ degrees of freedom. The mean $\tilde{\mu}_{\mathcal{D}}$ and variance $\tilde{\Sigma}_{\mathcal{D}}$ of the posterior have closed form expressions similar to those of $\mu_{\mathcal{D}}$ and $\Sigma_{\mathcal{D}}$ in Section 2.2.1, but they also involve evaluations of the basis functions g_i at the nodes and their integrals (O’Hagan, 1991, Sec. 2.2).

When only an unknown constant trend is postulated (i.e., $m \equiv \beta_1$) the posterior mean is $\tilde{\mu}_{\mathcal{D}} = \mathbf{w}_1^\top \mathbf{f}_X$, where

$$\mathbf{w}_1 = \left(\mathbf{I}_N - \frac{\mathbf{K}_{XX}^{-1} \mathbf{1}_N \mathbf{1}_N^\top}{\mathbf{1}_N^\top \mathbf{K}_{XX}^{-1} \mathbf{1}_N} \right) \mathbf{K}_{XX}^{-1} \mathbf{k}_{PX} + \frac{\mathbf{K}_{XX}^{-1} \mathbf{1}_N}{\mathbf{1}_N^\top \mathbf{K}_{XX}^{-1} \mathbf{1}_N} \quad (4.10)$$

⁵The approach that we describe is closely related to ordinary and universal kriging (Cressie, 1993, Ch. 3).

and $\mathbf{1}_N$ is the N -vector of ones, and one obtains *normalized* Bayesian quadrature. The terminology is motivated by the weights $\mathbf{w}_1$ in (4.10) summing up to one, which does not generally happen in conjugate Bayesian quadrature with a fixed prior mean. Weight normalization is particularly useful as a precaution against poorly set length-scale in high dimensions. If the length-scale ℓ of a stationary kernel, such as square exponential in (2.3) or Matérn in (2.4), is very small relative to the typical distance $\|\mathbf{x}_i - \mathbf{x}_j\|$ of the nodes, the covariance matrix is almost diagonal and $\mathbf{w}_1 \approx (\frac{1}{N}, \dots, \frac{1}{N})$. In contrast, the unnormalized weights in (2.13) are very close to zero in this case, which means that the posterior is close to the prior.

Prior mean estimation can be alternatively achieved via the use of a flat prior limit (Karvonen *et al.*, 2018; we have already seen this technique in action in Proposition 2.9) or maximum likelihood estimation (Jagadeeswaran and Hickernell, 2019). When maximum likelihood estimation is used to set the covariance scale parameter σ^2 (see Section 4.3), the resulting posteriors, except for being Gaussians, differ little from the Student's t posterior described above; many relevant posteriors are collected in Santner *et al.* (2003, Ch. 4). In addition to the works already mentioned, prior mean estimation or normalized Bayesian cubature has been used in O'Hagan (1992), Kennedy (1998), and Pronzato and Zhigljavsky (2020). We also refer to Bezhaev (1991) and DeVore *et al.* (2019) for non-probabilistic versions, whose relation to Bayesian quadrature has been described in Section 2.3, of prior mean estimation. Finally, we note that the theoretical guarantees available in the literature usually assume that the prior mean is fixed.

4.6 Main limitations

Several factors that limit the applicability of Bayesian quadrature emerge from the practical issues reviewed in Sections 4.1 to 4.5. The main limitations that have restricted Bayesian quadrature to a relatively narrow range of applications are (i) the need to have access to kernel means and variances and (ii) the cubic computational complexity in the number of nodes associated to Gaussian process methods. Mainly due to these two challenges a typical application of Bayesian quadrature

features a small number of evaluations of a low-dimensional integrand and a simple integration measure.

4.6.1 Kernel means and variances

That the kernel mean k_P and variance k_{PP} are available in closed forms for a limited collection of kernel-measure pairs is a severe limitation. Most Bayesian quadrature applications involve integration with respect to the uniform or Gaussian measure, and sometimes Bayesian quadrature is interpreted in a narrow sense to mean an algorithm or a class of algorithms for integration with respect to Gaussian measures. In this respect Bayesian quadrature resembles Gaussian quadratures and other classical approaches that are designed for integration with respect to particularly tractable measures.

While Oates *et al.* (2017b) have used Dirichlet process mixtures to extend Bayesian quadrature for intractable integration distributions that can only be sampled from, their approach has not caught on and is bound to sacrifice many desirable properties of Bayesian quadrature, such as a straightforward theoretical guarantees. An approach that is in principle general and powerful is to use the measure to design a kernel for which k_P and k_{PP} are easy to compute. Oates *et al.* (2017a) and South *et al.* (2022) use Stein's method to design such kernels. However, this simply transfers the trouble from computing integrals to computing a kernel and results in a model that is not easy to interpret. We believe that no satisfactory solution can exist that would obviate the need to compute kernel means or other typically intractable integrals. To construct a quadrature rule that encodes some properties of the integrand one needs access to some functions that effect the encoding. When N is small (e.g., the integrand is a computationally expensive simulation), it may in some cases make sense to approximate k_P and k_{PP} with, for example, Monte Carlo, if doing so has negligible computational cost in comparison to the integrand evaluations.

4.6.2 Cubic computational complexity

Computational cost is the second major limitation. As discussed in Section 4.4, to implement Bayesian quadrature one has to solve linear

systems defined by the typically dense $N \times N$ kernel matrix $\mathbf{K}_{XX}$. The cubic computational complexity in the number of datapoints that this gives rise to is a well-known computational bottleneck in all Gaussian process methods. Over the past 20 years much effort has gone into the development of computationally scalable Gaussian process methods (for a recent review, see Liu *et al.*, 2020). These methods are predominantly *approximate* in the sense of the definition given in Section 3.2 and developed under the assumption that the data are noisy, so that the linear systems that need to be solved are defined by the regularised kernel matrix $\mathbf{K}_{XX} + \lambda \mathbf{I}_N$ rather than $\mathbf{K}_{XX}$. Because the nugget λ is typically either zero or very small in Bayesian quadrature, many scalable Gaussian process methods cannot be imported to Bayesian quadrature as such. For example, methods based on low-rank approximations of the kernel matrix require a positive nugget. It is nevertheless surprising that no systematic study is available on the feasibility of applying existing approaches to scalable Gaussian process regression to Bayesian quadrature.

There exist a number of scalable approaches specifically to Bayesian quadrature (see the *scalable* label in Section 3.2). While some have been highly successful, all are tied to certain measure-kernel pairs and do not generalise easily or at all. Except for some special cases (e.g., Jagadeeswaran and Hickernell, 2019), Bayesian quadrature is presently not a serious competitor to Monte Carlo, quasi-Monte Carlo, and related approaches to numerical integration in high dimensions or in other situations in which a large number of integrand evaluations are necessary.⁶ We direct the reader to Hennig *et al.* (2022, Sec. 42.3) for an assessment of computational challenges that is in alignment with ours.

Because they incorporate prior information about the object being modelled, Bayesian methods can be expected to perform better than other methods when the amount of data is limited, a setting in which solving linear systems is not a computational problem. This has resulted

⁶This is not because Bayesian quadrature converges slower than the alternatives but because constants hidden by asymptotic notation tend to grow with dimension and to implement Bayesian quadrature one must solve a linear system. Note that, despite its classical dimension-independent convergence rate $\mathcal{O}(N^{-1/2})$, even Monte Carlo suffers from the curse of dimensionality in this sense (Tang, 2024).

in a marked emphasis on applications in which integrand evaluations are expensive (Kennedy, 2000; Briol *et al.*, 2019). For example, Briol *et al.* (2019, Sec. 1.2) “focus on numerical integrals where the cost of evaluating the integrand forms a computational bottleneck, and for which numerical error is hence more likely to be non-negligible.” Model selection naturally plays all the more important a role in the low-data regime in which non-conjugate Bayesian quadrature methods that would become intractable even for moderately large datasets may shine.

5

Theoretical guarantees

This chapter contains a number of mathematical results that provide theoretical guarantees on the accuracy of conjugate Bayesian quadrature under certain assumptions on the covariance function k , the integrand f and the coverage of the domain D by the nodes. We shall work with a sequence $(X_N)_{N=1}^\infty$ of sets $X_N = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subset D$ consisting of N pairwise distinct nodes. In spite of our notation, the node sets need not be nested (i.e., despite both being denoted $\mathbf{x}_1$, the first nodes in X_N and X_{N+1} need not be equal). To further simplify notation, we often replace subscripted X_N and $\mathcal{D}$ with N to emphasise the dependency on the number of nodes. For example, under this convention μ_N stands for $\mu_{\mathcal{D}}$ in (2.9) with $X = X_N$ and $\mu_{N,\lambda}$ for its regularized version in (4.9). Most theoretical guarantees we give state that

$$|I_P(f) - \mu_{N,\lambda}| \leq C(r_N + \sqrt{\lambda}) \quad (5.1)$$

for a positive constant C and a decreasing positive sequence $(r_N)_{N=1}^\infty$ which tends to zero. The rate with which r_N tends to zero reflects the smoothness of k and f . A large collection of theoretical guarantees for Bayesian quadrature may be found in references such as Kanagawa *et al.* (2016), Briol *et al.* (2019), Karvonen (2019), Kanagawa and Hennig (2019), Karvonen *et al.* (2020), and Kanagawa *et al.* (2020). Proofs are

relegated to Chapter 7, which also contains a more detailed comparison of our results to those in the literature.

We prioritize simplicity and ease of presentation over generality. Accordingly, throughout this section we assume that the prior mean m is identically zero, an assumption that could be relaxed to m being an element of the RKHS of k . Furthermore, in most results of this section we make use of the following assumption on the domain and the integration distribution.

Assumption 5.1. Assume that (i) $D = [0, 1]^d \subset \mathbb{R}^d$ and (ii) the probability measure P has a density function p such that $p_{\max} = \sup_{\mathbf{x} \in [0, 1]^d} p(\mathbf{x}) < \infty$.

Remark 5.2. Basic translation and scaling arguments can be used to allow for any hyperrectangular domain $[a_1, b_1] \times \cdots \times [a_d, b_d]$. More generally, the results below hold when D satisfies an *interior cone condition* and has a *Lipschitz boundary*. See Kanagawa *et al.* (2020, Sec. 3) or Wynne *et al.* (2021, App. A) for details in the context of Gaussian processes.

5.1 Guarantees for generic nodes

We first provide guarantees for *generic* nodes, by which we mean that no reference is made to the process by which the nodes are obtained and upper bounds of the form (5.1) are given in terms of the computable *fill-distance* h_X and *separation radius* q_X , which are defined as

$$h_X = \sup_{\mathbf{x} \in [0, 1]^d} \min_{i=1, \dots, N} \|\mathbf{x} - \mathbf{x}_i\| \quad \text{and} \quad q_X = \frac{1}{2} \min_{i \neq j} \|\mathbf{x}_i - \mathbf{x}_j\| \quad (5.2)$$

for the domain $D = [0, 1]^d$. Fill-distance equals the radius of the largest ball in $[0, 1]^d$ that does not contain any of the points in X while the separation radius is half the smallest distance between any two points in X . Because the fill-distance and separation radius are, at least in principle, computable for any set of nodes, error bounds expressed in terms of these quantities apply to all relevant Bayesian quadrature methods regardless of how the nodes have been sampled (Section 3.3).

Choices made in the sampling axis of taxonomy affect the bounds through fill-distance and separation radius.

If, for a sequence of node sets $(X_N)_{N=1}^\infty$, the fill-distance and separation radius are asymptotically comparable as $N \rightarrow \infty$, the point design is said to be *quasi-uniform*. This means that there exists a constant $c_{\text{qu}} > 0$ such that

$$q_N \leq h_N \leq c_{\text{qu}} q_N$$

for every $N \geq 1$. This implies that there are positive constants c_1 and c_2 such that

$$c_1 N^{-1/d} \leq h_N \leq c_2 N^{-1/d} \quad \text{and} \quad c_1 N^{-1/d} \leq q_N \leq c_2 N^{-1/d} \quad (5.3)$$

for every $N \geq 1$ (Wendland, 2005, Sec. 14.1). Regular grids are quasi-uniform. For example, the equispaced nodes $X_N = \{1/N, 2/N, \dots, (N-1)/N, 1\}$ on $D = [0, 1]$ have $q_N = h_N = 1/N$ for every $N \geq 1$. How the fill-distance and certain related quantities behave for random points is discussed in Krieg and Sonnleitner (2024).

We begin with two generic propositions that guarantee the convergence of Bayesian quadrature under very general conditions. Recall from Section 2.3.1 that $\mathcal{H}(k)$ is the RKHS of k on the domain D . The RKHS consists of certain functions defined on D .

Proposition 5.3. Suppose that Assumption 5.1 holds, the function $k(\cdot, \mathbf{x})$ is continuous for every $\mathbf{x} \in D$, and $\sup_{\mathbf{x} \in D} k(\mathbf{x}, \mathbf{x}) < \infty$. If $f \in \mathcal{H}(k)$, the node sets are nested (i.e., $X_N \subset X_{N+1}$) and $h_N \rightarrow 0$ as $N \rightarrow \infty$, then

$$|I_P(f) - \mu_N| \rightarrow 0 \quad \text{as} \quad N \rightarrow \infty.$$

Proof. See Section 7.3. □

Proposition 5.3 guarantees that the integral mean of Bayesian quadrature tends to the true integral if the point design is space-filling in the sense that $h_N \rightarrow 0$. However, the proposition gives us no inkling on the rate of this convergence. The next proposition ensures that *at least* the familiar square-root rate of Monte Carlo can be attained. That is, conjugate Bayesian quadrature can always beat Monte Carlo.

Proposition 5.4. Suppose that Assumption 5.1 holds and the nodes are independent samples from P . If $f \in \mathcal{H}(k)$, then there is $C > 0$, which depends only on k and P , such that

$$\mathbb{E} \left[|I_P(f) - \mu_N| \right] \leq CN^{-1/2} \quad (5.4)$$

for every $N \geq 1$, where the expectation is with respect to the random nodes.

Proof. See Section 7.3. □

Propositions 5.3 and 5.4 provide slow rates because they make few assumptions about the integrand, covariance function, and the nodes. The rest of this section shows how smoothness assumptions on k and f improve the square-root rate in (5.4).

5.1.1 Isotropic Matérn kernels

The *Sobolev space* $H^\alpha([0, 1]^d)$ of order $\alpha \in \mathbb{N}$ is a Hilbert space that consist of those functions g for which the *weak derivative* $D^n g$ exists and is square-integrable for every non-negative multi-index such that $|\mathbf{n}| = n_1 + \dots + n_d \leq \alpha$. That is, $g \in H^\alpha([0, 1]^d)$ if and only if

$$\left(\sum_{|\mathbf{n}| \leq \alpha} \int_{[0, 1]^d} [D^n g(\mathbf{x})]^2 d\mathbf{x} \right)^{1/2} < \infty. \quad (5.5)$$

We use the Fourier transform to define the norm $\|\cdot\|_{H^\alpha([0, 1]^d)}$ of $H^\alpha([0, 1]^d)$; see (7.3) in Section 7.2. The resulting norm is equivalent to the norm defined by (5.5). Any function whose classical derivatives up to order α exist and are square-integrable on $[0, 1]^d$ is an element of $H^\alpha([0, 1]^d)$. Conversely, if $\alpha > r + d/2$ for $r \in \mathbb{N}$, then all functions in $H^\alpha([0, 1]^d)$ are r times differentiable in the classical sense. In particular, all functions in $H^\alpha([0, 1]^d)$ are continuous if $\alpha > d/2$. Sobolev spaces $H^\alpha([0, 1]^d)$ of *fractional smoothness* $\alpha > 0$ may be easily defined via Fourier transforms; details are again given in Section 7.2. Theorems 5.5 and 5.6 provide theoretical guarantees for isotropic Matérn kernels based on the fill-distance. For the purposes of these theorems, define the *mesh ratio*

$$\rho_X = \frac{h_X}{q_X} \geq 1. \quad (5.6)$$

Theorem 5.5 (Generic nodes for Matérns). Suppose that Assumption 5.1 holds and let k be a Matérn kernel (2.4) with smoothness parameter $\nu > 0$. Let $\alpha = \nu + d/2 \geq \beta > d/2$. If $f \in H^\beta([0, 1]^d)$ and $\lambda \geq 0$, then there is a positive constant C , which does not depend on f , X , λ , ℓ or σ , such that

$$|I_P(f) - \mu_{\mathcal{D}, \lambda}| \leq C \left(c_{1, \ell, \nu} h_X^\beta \rho_X^{\alpha-\beta} + \frac{c_{2, \ell, \nu}}{\sigma \ell^{d/2}} \sqrt{\lambda} h_X^{d/2} q_X^{-(\alpha-\beta)} \right) \|f\|_{H^\beta([0, 1]^d)}, \quad (5.7)$$

where $c_{1, \ell, \nu} = \max\{\ell/\sqrt{2\nu}, \sqrt{2\nu}/\ell\}$ and $c_{2, \ell, \nu} = \max\{1, \ell/\sqrt{2\nu}\}$.

Proof. See Section 7.3.2. □

Theorem 5.5 provides an error estimate for any given set of nodes X in terms of h_X , q_X , and ρ_X . Using (5.3) to control the fill-distance and separation radius and making sure that constants that depend on kernel parameters do not blow up yields the following variant of Theorem 5.5, which applies to a *sequence* of node sets $(X_N)_{N=1}^\infty$ produced by a hypothetical Bayesian quadrature method that is run indefinitely without a stopping criterion.

Theorem 5.6 (Quasi-uniform nodes for Matérns). Suppose that Assumption 5.1 holds and let $(X_N)_{N=1}^\infty$ be a sequence of quasi-uniform node sets in $[0, 1]^d$. Let k be a Matérn kernel (2.4) with fixed smoothness parameter $\nu > 0$ and scale and length-scale parameters σ_N and ℓ_N which may vary with N but satisfy $\sigma_N \geq \sigma_{\min}$ and $\ell_{\min} \leq \ell_N \leq \ell_{\max}$ for some positive $\sigma_{\min}$, $\ell_{\min}$ and $\ell_{\max}$. Let $\alpha = \nu + d/2 \geq \beta > d/2$. If $f \in H^\beta([0, 1]^d)$ and $\lambda = \lambda_N \geq 0$, then there is a positive constant C , which does not depend on f , N or λ_N , such that

$$|I_P(f) - \mu_{N, \lambda_N}| \leq C \left(N^{-\beta/d} + \sqrt{\lambda_N} N^{-1/2} N^{(\alpha-\beta)/d} \right) \|f\|_{H^\beta([0, 1]^d)} \quad (5.8)$$

for every $N \geq 1$.

Proof. It follows from (5.3) that both h_N and q_N are of order $N^{-1/d}$ and that $\rho_N \geq 1$ is bounded from above. The claim follows by inserting these rates and bounds in Theorem 5.5 and using the assumed bounds

on the scale and length-scale parameters to bound the constants $c_{1,\ell,\nu}$ and $c_{2,\ell,\nu}/(\sigma\ell^{d/2})$ and incorporate them into C . $\square$

Theorems 5.5 and 5.6 comprise two somewhat different cases. *Firstly*, if $\beta = \alpha$, then f is an element of the RKHS of k (see Section 2.3.1). In this case the terms $\rho_X^{\alpha-\beta}$ and $q_X^{-(\alpha-\beta)}$ in (5.7) and $N^{(\alpha-\beta)/d}$ in (5.8), which slow down convergence, vanish and the bounds depend on the nodes only via the fill-distance. The theorems do not yield improved rates of convergence if $f \in H^\gamma([0, 1]^d)$ for $\gamma > \alpha$ (i.e., when the model *undersmooths*). Since $H^\gamma([0, 1]^d) \subset H^\alpha([0, 1]^d)$, in this case one simply applies the theorems with $\beta = \alpha$.¹ *Secondly*, if $\beta < \alpha$ and $f \notin H^\alpha([0, 1]^d)$, then f is not in $\mathcal{H}(k)$, so that the model *oversmooths*. The bound (2.18) links integration error to the BQ variance but holds only for functions in the RKHS. This gives rise to the distinction between the cases $f \in \mathcal{H}(k)$ and $f \notin \mathcal{H}(k)$.

When $\lambda_N = 0$, Theorem 5.6 gives the rate of convergence $N^{-\beta/d}$ that is worst-case optimal in the Sobolev space $H^\beta([0, 1]^d)$ (Novak and Triebel, 2006). Bayesian quadrature therefore *adapts to misspecified smoothness*, being capable of attaining the optimal rate of convergence even when $\alpha > \beta$. This is convenient, as eliciting the smoothness of f tends to be difficult. As evidenced by the term $N^{-1/2+(\alpha-\beta)/d}$, for a positive nugget the rate given by Theorem 5.6 depends on the degree of smoothness misspecification, $\alpha - \beta \geq 0$. In particular, if $\alpha \geq \beta + d/2$, Theorem 5.6 does not guarantee convergence unless λ_N tends to zero sufficiently fast. This may be due to sub-optimality of some estimates the proof uses.

For a machine learning practitioner or a statistician, randomly sampled nodes are undoubtedly the most relevant ones. The following theorem, which has not appeared in the context of Bayesian quadrature before, guarantees that for Matérns the expected error decays with the worst-case optimal rate.

¹However, bounds of order $h_X^{2\alpha}$ or $N^{-2\alpha}$ are available if f has smoothness 2α and satisfies certain boundary conditions (Schaback, 1999; Schaback, 2018; Tuo *et al.*, 2020; Karvonen *et al.*, 2020; Sloan and Kaarnioja, 2025; Karvonen *et al.*, 2025). This case is known as *superconvergence* or *improved rate* of kernel-based interpolation.

Theorem 5.7 (Random nodes for Matérns). Suppose that P is uniform on $D = [0, 1]^d$. Let k be a Matérn kernel (2.4) with fixed smoothness parameter $\nu > 0$ and scale and length-scale parameters σ_N and ℓ_N which may vary with N but satisfy $\sigma_N \geq \sigma_{\min}$ and $\ell_{\min} \leq \ell_N \leq \ell_{\max}$ for some positive $\sigma_{\min}$, $\ell_{\min}$ and $\ell_{\max}$. If $f \in H^\alpha([0, 1]^d)$ for $\alpha = \nu + d/2$ and the nodes are sampled independently from P , then there is a positive constant C , which does not depend on f or N , such that

$$\mathbb{E}[|I_P(f) - \mu_N|] \leq CN^{-\alpha/d}$$

for every $N \geq 1$. The expectation is with respect to the random nodes.

Proof. See Section 7.3.2. □

5.1.2 Product Matérn kernels

For a product Matérn kernel in (2.5), theoretical guarantees can be derived from the results in Teckentrup (2020) if the nodes form a sparse grid on $D = [0, 1]^d$ and the integrand is in a *mixed-order Sobolev space* (see also Rieger and Wendland, 2017; Nobile *et al.*, 2018; Rieger and Wendland, 2020). For $\alpha \in \mathbb{N}$, the mixed-order Sobolev space $H_{\text{mix}}^\alpha([0, 1]^d)$ is the tensor product of univariate Sobolev spaces $H^\alpha([0, 1])$ and consists of $g \in L^2([0, 1]^d)$ whose weak derivative $D^n g$ exists and is in $L^2([0, 1]^d)$ for every multi-index $\mathbf{n} = (n_1, \dots, n_d)$ such that $n_i \leq \alpha$ for every i . If k is a product Matérn kernel (2.5) with smoothness $\nu > 0$ such that $\alpha = \nu + 1/2$ is a positive integer $f \in H_{\text{mix}}^\beta([0, 1]^d)$ for an integer β such that $1 \leq \beta \leq \alpha$, it follows from Teckentrup (2020, Thm. 3.11) that there is $C > 0$, which does not depend on f or N , such that

$$|I_P(f) - \mu_N| \leq CN^{-\beta}(\log N)^{(\beta+1)(d-1)} \|f\|_{H_{\text{mix}}^\beta([0, 1]^d)}$$

when the nodes form a certain sparse grid (see Figure 4.1 for an example). Due to the requirement that the nodes form a sparse grid, this result is much more restrictive than Theorems 5.5 to 5.7. We do not go into further details so as to avoid having to review the construction sparse grids. In general, error analysis for product Matérns is much less mature than for isotropic Matérns.

5.1.3 Square exponential kernel

Here we provide theoretical guarantees for conjugate Bayesian quadrature based on the square exponential kernel in (2.3). Unlike those for Matérn kernels, these guarantees apply only to functions in the RKHS of the kernel, $\mathcal{H}(k)$. One can show (Minh, 2010) that the RKHS on any $D \subseteq \mathbb{R}^d$ with non-empty interior consists precisely of the functions

$$g(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x}\|^2}{2\ell^2}\right) \sum_{\mathbf{n} \in \mathbb{N}_0^d} c_{\mathbf{n}} \mathbf{x}^{\mathbf{n}} \quad \text{such that} \quad \sum_{\mathbf{n} \in \mathbb{N}_0^d} \ell^{2|\mathbf{n}|} \mathbf{n}! c_{\mathbf{n}}^2 < \infty,$$

where the sums are over all non-negative d -dimensional multi-indices $\mathbf{n} = (n_1, \dots, n_d)$ and we denote $\mathbf{x}^{\mathbf{n}} = x_1^{n_1} \cdots x_d^{n_d}$ and $\mathbf{n}! = n_1! \cdots n_d!$. The RKHS is minuscule: it consists of infinitely differentiable functions but contains no polynomials except the zero function. The current state of the theory is inadequate to provide theoretical guarantees for $f \notin \mathcal{H}(k)$.

For Matérns, the length-scale ℓ only affects the norm of the RKHS, not which functions are contained therein; see (7.10) and (7.11). However, for the square exponential kernel ℓ does affect the size of the RKHS and thus plays a role similar to the smoothness parameter ν in Matérn kernels. We therefore require that ℓ be kept fixed in the results below.

Theorem 5.8 (Generic nodes for SE). Suppose that Assumption 5.1 holds and let k be a square exponential kernel (2.4) with fixed scale $\sigma > 0$ and length-scale $\ell > 0$. If $f \in \mathcal{H}(k)$ and $\lambda \geq 0$, then there are positive c_1 and c_2 , which do not depend on f , X or λ , such that

$$|I_P(f) - \mu_{\mathcal{D}, \lambda}| \leq \left(2 \exp(c_1 h_X^{-1} \log h_X) + \sqrt{\lambda} \exp(c_2 h_X^{-1})\right) \|f\|_{\mathcal{H}(k)}.$$

Proof. See Section 7.3.3. □

Like Theorem 5.5, the above theorem applies to any given set of nodes. By instead considering a quasi-uniform sequence of node sets generated by some sampler we obtain an explicit rate of convergence given by the following theorem.

Theorem 5.9 (Quasi-uniform nodes for SE). Suppose that Assumption 5.1 holds and let k be a square exponential kernel (2.4) with fixed scale

$\sigma > 0$ and length-scale $\ell > 0$. If $f \in \mathcal{H}(k)$ and $\lambda = \lambda_N \geq 0$, then there are positive c_1 and c_2 , which do not depend on f , N or λ_N , such that

$$|I_P(f) - \mu_{\mathcal{D},\lambda}| \leq \left(2e^{-c_1 N^{1/d} \log N} + \sqrt{\lambda_N} e^{c_2 N^{1/d}} \right) \|f\|_{\mathcal{H}(k)}$$

for every $N \geq 1$.

Proof. The claim follows from Theorem 5.8 and (5.3). $\square$

Note that the terms dependent on the nugget in Theorems 5.8 and 5.9 blow up very fast as $h_X \rightarrow 0$ or $N \rightarrow \infty$. These theorems are useful only when the nugget is close to zero (or tends to zero sufficiently fast). When $\lambda_N = 0$, the convergence in Theorem 5.9 is super-exponential and hence much faster than in Theorems 5.6 to 5.7, which yield polynomial rates for Matérns. This is because the class of functions these theorems apply to, the RKHS of the square exponential kernel, is significantly smaller than any Sobolev space. Theorems 5.8 and 5.9 do not therefore demonstrate that the square exponential kernel is in any way superior to a Matérn (or any other) kernel unless one is certain that f is in $\mathcal{H}(k)$. We note that on dimension one and for product designs in higher dimensions it is possible to obtain slightly improved rates with explicit constants c_1 and c_2 by using results in Yarotsky (2013) and Karvonen and Suzuki (2025).

5.2 Guarantees for sequential Bayesian quadrature

Consider a sequential setting where the nodes are obtained by sequential maximization of an acquisition function that is a strictly increasing transformation $\psi: [0, \infty) \rightarrow \mathbb{R}$ of the scalar squared correction in (4.3). That is, for each $n \geq 1$, the $(n+1)$ th node $\mathbf{x}_{n+1}$ satisfies

$$\begin{aligned} \mathbf{x}_{n+1} &\in \arg \max_{\tilde{\mathbf{x}}} \psi(\rho^2(\tilde{\mathbf{x}})) \\ &= \arg \max_{\tilde{\mathbf{x}}} \psi\left(\mathbf{k}_{PX(\tilde{\mathbf{x}})}^\top \mathbf{K}_{X(\tilde{\mathbf{x}})X(\tilde{\mathbf{x}})}^{-1} \mathbf{k}_{PX(\tilde{\mathbf{x}})}\right), \end{aligned} \tag{5.9}$$

where $X(\tilde{\mathbf{x}}) = X \cup \{\tilde{\mathbf{x}}\} = \{\mathbf{x}_1, \dots, \mathbf{x}_n, \tilde{\mathbf{x}}\}$. Examples include the MI, IVR, and NIV acquisition functions from Section 4.2.3. The following theorem demonstrates that sequential conjugate Bayesian quadrature converges at least as fast as Monte Carlo.

Theorem 5.10 (Sequential nodes). Suppose that Assumption 5.1 holds, the function $k(\cdot, \mathbf{x})$ is continuous for every $\mathbf{x} \in D$, and $\sup_{\mathbf{x} \in D} k(\mathbf{x}, \mathbf{x}) < \infty$. If $f \in \mathcal{H}(k)$ and N nodes are selected sequentially as in (5.9), then there is a positive constant C , which depends only on D , P and k , such that

$$|I_P(f) - \mu_N| \leq CN^{-1/2} \|f\|_{\mathcal{H}(k)}. \quad (5.10)$$

Proof. See Section 7.4. □

The rate (5.10) is likely conservative (Santin *et al.*, 2022, Secs. 5 & 7). See Kanagawa and Hennig (2019), Wenzel *et al.* (2021), Wenzel *et al.* (2022), and Wenzel *et al.* (2025) for results on the convergence of Bayesian quadrature and related methods with acquisition functions other than (5.9).

5.3 Other guarantees

Various other guarantees for Bayesian quadrature are available in the literature. These include Bayesian quadrature with (i) nodes having strong symmetry properties (Karvonen and Särkkä, 2018, Sec. 4.4); (ii) nodes obtained by optimization (Briol *et al.*, 2015; Kanagawa and Hennig, 2019; Pronzato, 2023); (iii) nodes sampled from distributions different from P (Briol *et al.* 2017; see also Bach 2017); (iv) nodes sampled from a determinantal point process (Belhadji *et al.*, 2019; Belhadji, 2020; Belhadji, 2021); and (v) nodes selected using a certain recombination algorithm (Adachi *et al.*, 2022, Sec. 6). Cai and Scarlett (2024) have proved error bounds for BQ applied to the computation of normalizing constants of the form $\int_D \exp(-\lambda f(\mathbf{x})) d\mathbf{x}$ with $\lambda > 0$. Convergence guarantees for multi-output and multilevel Bayesian quadrature are provided in Xi *et al.* (2018, Sec. 3) and Li *et al.* (2023, Sec. 4), respectively. Chatalic *et al.* (2025) consider subsampling kernel mean embeddings approximated with Monte Carlo. The equivalences reviewed in Section 2.3.1 allow one to apply the wealth of results collected in Oettershagen (2017) to Bayesian quadrature.

6

Empirical illustrations

As mentioned in the introduction, the aim of our experiments is not to re-produce the performance of Bayesian quadrature methods, this having been done in the literature we have cited, nor to propose and evaluate new methods. Rather, the aim is to illustrate the taxonomy of Chapter 3 and the qualitative influence of its axes, *sampling*, *model* and *inference*. In particular, the experiments show ablation curves that disentangle the effect of the axes for several integrands f that differ in their degree of smoothness or (non-)stationarity. Where possible, we tie the qualitative or quantitative behavior of the BQ methods to the theoretical results from Chapter 5. We hope the illustrations provide additional intuition on BQ methods for practitioners.

The available open source code for BQ is unfortunately still scarce in comparison to the diversity of methods proposed in the literature. Under this constraint, we have opted to implement the experiments in the open source Python library EmuKit (Paleyes *et al.*, 2019; Paleyes *et al.*, 2023) since, at the time of writing, it provided the highest diversity of BQ algorithms. It further enabled us to code in a single language. In particular, EmuKit contains a variety of kernel embeddings for the prevalent conjugate BQ model and a variety of node selection schemes.

Another contender, the Python library ProbNum (Wenger *et al.*, 2021), did not contain adequate hyper-parameter tuning capability, nor state-of-the-art active sampling loops at the time of writing.¹ Unfortunately, scalable BQ methods are currently not available in EmuKit and are thus omitted here; some are implemented in the MATLAB library GAIL (Choi *et al.*, 2020). All experiments use exact inference. Section 6.1 describes the experimental setup and integrands (called test functions) in more detail. Section 6.2 illustrates and discusses results.

6.1 Experimental setup

This section describes the experimental setup.

6.1.1 Kernels (model axis)

As all the experiments use the conjugate model, the models only differ by their choice of the kernel k . We select 5 different kernels to denote 5 different models. Three of the kernels are Matérn kernels in (2.4) with smoothness parameter $\nu \in \{\frac{1}{2}, \frac{3}{2}, \frac{5}{2}\}$. As described in Sections 2.3.1 and 5.1.1, these kernels are linked to Sobolev spaces. Matérn kernels are stationary. Further, we use the non-stationary and non-smooth Brownian motion kernel as in (2.6) and the stationary and infinitely smooth square exponential kernel in (2.3). All selected kernels have analytic kernel embeddings with respect to P that are available in EmuKit. Table 6.1 summarizes the kernels and their parameters.

6.1.2 Node selection (sampling axis)

Along the sampling axis we use 5 sampling schemes: two random ones and three deterministic ones to cover the mode of sampling. The two random schemes are random sampling from P and the space-filling Latin hypercube sampling (Section 4.2.2). Two of the three deterministic samplers produce model-independent designs: the space-filling Sobol sequence and the classically motivated Legendre roots (Section 4.2.1). The final deterministic node selection scheme uses a sequential, active

¹As the library seems to be under active development, this may have changed.

Kernel	Kernel params.	Stationary	Smooth.
Matérn ($\nu = \frac{1}{2}$)	$\theta = \{\sigma^2, \ell_1, \dots, \ell_d\}$	✓	0
Matérn ($\nu = \frac{3}{2}$)	$\theta = \{\sigma^2, \ell_1, \dots, \ell_d\}$	✓	1
Matérn ($\nu = \frac{5}{2}$)	$\theta = \{\sigma^2, \ell_1, \dots, \ell_d\}$	✓	2
Brownian motion	$\theta = \{\sigma^2\}$	✗	0
Square exponential	$\theta = \{\sigma^2, \ell_1, \dots, \ell_d\}$	✓	∞

Table 6.1: Summary of the kernels used in the experiments. Here smoothness (smooth.) refers to the classical differentiability of the sample paths of the GP prior.

sampler and the IVR acquisition function (Section 4.2.3). The samplers are summarized in Table 6.2. Further details can be found in Section 6.3.

Sampler	Mode	Granular label	Code
Random from P	random	model-indep.	NumPy
Latin hypercube	random	model-indep.	SciPy
Sobol sequence	deterministic	model-indep.	SciPy
Legendre roots	deterministic	model-indep.	SciPy
IVR	deterministic	model-dep., seq., act.	EmuKit

Table 6.2: Summary of the samplers used in the experiments. The “code” column indicates which library was used to create the nodes (more details in Section 6.3).

6.1.3 Implementation details

As the experiments use the conjugate model throughout, $f = g$ is a Gaussian process. To represent f we use the GPy Python library (GPy authors, 2012) and for the kernel means and variances k_P and k_{PP} we use the Python library EmuKit (Paley et al., 2019; Paley et al., 2023) and its existing GPy quadrature wrappers. The kernel parameters θ (Table 6.1) are fitted with maximum marginal likelihood (Section 4.3),

which GPy defaults to.

We use the default settings for the nugget of both EmuKit and GPy. The nugget λ (Section 4.4) is composed of three terms:

$$\lambda = \lambda_{\text{Emu}} + \lambda_{\text{GPy}} + \lambda_{\text{GPyDyn}}.$$

The first two terms, $\lambda_{\text{Emu}} = 10^{-10}$ and $\lambda_{\text{GPy}} = 10^{-8}$, are the fixed default nuggets of EmuKit and GPy respectively. While λ_{Emu} can be adjusted via the EmuKit interface, λ_{GPy} is not directly accessible to the user. The dynamic nugget term λ_{GPyDyn} is computed as follows. Let $\mathbf{A} = \mathbf{K}_{XX} + (\lambda_{\text{Emu}} + \lambda_{\text{GPy}})\mathbf{I}_N$ be the kernel Gram matrix augmented with the fixed nugget terms and let $\bar{a} = \frac{1}{N} \sum_{i=1}^N A_{ii}$ be the mean of its diagonal elements. The third summand, λ_{GPyDyn} , is a dynamic nugget that GPy adds automatically when the Cholesky decomposition of $\mathbf{A}$ (used to solve the linear system of the kernel Gram matrix) fails. λ_{GPyDyn} is initialized to zero and then increased up to 5 times, taking the successive values $0, \bar{a}10^{-6}, \bar{a}10^{-5}, \bar{a}10^{-4}, \bar{a}10^{-3}$, and $\bar{a}10^{-2}$ until the Cholesky decomposition is successful. If the Cholesky decomposition fails with the largest dynamic nugget $\bar{a}10^{-2}$, the method finally gives up and throws an error. In contrast to the fixed nugget terms, the dynamic nugget depends on $\mathbf{K}_{XX}$ and is reset to zero after each successful Cholesky decomposition. Hence every Gram matrix in the experiments incorporates a nugget that is at least 10^{-8} but *may* vary from one matrix to the next.

All nodes, except the ones obtained actively via IVR, can be pre-computed as their sampling is model-independent. In the settings where we pre-compute nodes we (i) first sample the nodes, (ii) then hand them to the EmuKit BQ wrapper as a fixed dataset, (iii) fit the hyper-parameters θ , and (iv) finally compute $I_P(f) \mid \mathcal{D}$ using EmuKit's kernel embeddings. When the nodes are random, we use the same NumPy seed of 0 across all experiments. This means that random, model-independent nodes are identical across models and integrands f for a given N , which may be reflected in some curves showing similar fluctuations. We elaborate on this further when describing the results below. For active node selection with IVR, we fit the hyper-parameters successively after every newly obtained node. IVR as an active node acquisition method further requires an initial design which we choose to be $N_{\text{init}} =$

$\min\{N, 5\}$ random nodes from P . This ensures that hyper-parameter optimization by maximum marginal likelihood does not fail. The initial design uses the same NumPy seed of 0.

6.1.4 Integrands (test functions)

An integration problem is composed of an integrand or, *test function*, f and a measure P . For the experiments we choose P to be the standard Lebesgue (i.e., uniform) measure on the unit interval $D = [0, 1]$. The test functions f cover a smooth and non-smooth as well as stationary and non-stationary² integrands. All test functions are continuous and do not exhibit “jumps”.

In order to create distinct test functions with similar properties, we choose parameterized families of functions and sample the parameters for an individual test function from some distribution. That is, f is deterministic for each fixed set of parameters. In total, for every parameterized family we produce $T = 50$ deterministic integrands for evaluation. The two families are:

- *Univariate, smooth, stationary integrands:* We use smooth test functions as in Filip *et al.* (2019, Sections 2 & 3). The integrand is defined as a finite Fourier series

$$f(x) = \sqrt{2} \sum_{j=0}^J [a_j \cos(2\pi jx/L) + u_j \sin(2\pi jx/L)] \quad (6.1)$$

with length-scale $L = 5$ and $J = 25$ terms and scalar coefficients a_j and u_j for $j = 0, \dots, J$. The chosen values for L and J ensure that the function is reasonably variable in the domain D yet not too easy to learn. All functions are infinitely smooth. Each of the $T = 50$ sets of coefficients $\{a_j, u_j\}_{j=1}^J$ is produced according to $a_j, u_j \sim N(0, 2(J+1))$. We set $a_0 = u_0 = 0$ to define an offset.

- *Univariate, non-smooth, continuous, non-stationary integrands:* These are approximate Brownian motion paths represented by a

²When referring to functions we use the word “stationary” as in Rasmussen and Williams (2006, p. 4) who write “*informally, stationarity means that the functions look similar at all x locations*”.

truncated Karhunen–Loève expansion of the Brownian motion:

$$f(x) = \sqrt{2} \sum_{j=1}^J a_j \frac{\sin((j - \frac{1}{2})\pi x)}{(j - \frac{1}{2})\pi}. \quad (6.2)$$

We choose $J = 500$ terms. Each of the $T = 50$ sets of sets of coefficients $\{a_j\}_{j=1}^J$ is sampled according to $a_j \sim \mathcal{N}(0, 1)$. The constructed functions resemble non-smooth, continuous functions quite well.

All of the above integrands can be represented as function handle $f(\cdot)$ in code as opposed to some fixed vector of observations $\mathbf{f}_X$. This is achieved by constructing $f(\cdot)$ as a finite sum of basis functions. In our experiments, an explicit function handle is only really needed by active IVR sampling; the BQ model and inference do not require a function handle per se.

The ground truth integrals are compute with SciPy’s `integrate.quad` method to high precision. Table 6.3 summarizes the tests functions. Illustrations of some of the $T = 50$ integrands for each family are shown in Figure 6.1.

6.1.5 Description of individual experiments

We pair each of the five conjugate BQ models of Section 6.1.1 with the five node selection schemes of Section 6.1.2. This yields a total of 25 BQ methods, which we identify by the kernel name and sampling method. Then we evaluate each of the methods on the $T = 50$ test integrands of each family of integrands of Section 6.1.4, which amounts to 1250 combinations of test integrands and BQ methods per family.

Name	Smooth	Stationary	d
Fourier [Eq. (6.1)]	✓	✓	1
Brownian motion [Eq. (6.2)]	✗	✗	1

Table 6.3: Summary of the test functions used in the experiments.

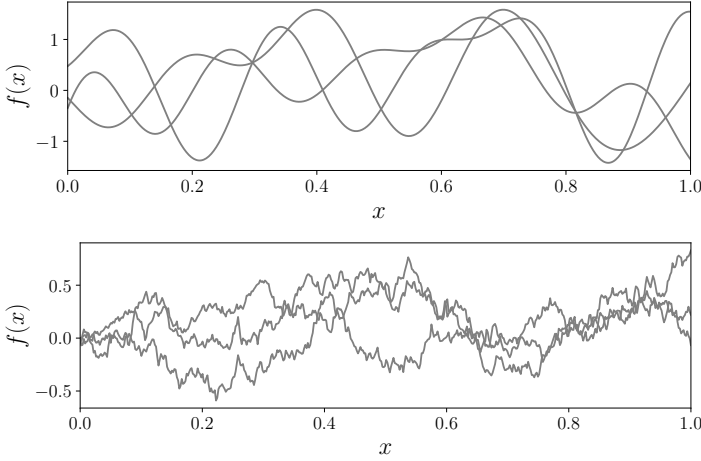


Figure 6.1: Examples of integrands. *Top:* Three examples of smooth univariate integrands in (6.1). *Bottom:* Three example of approximate Brownian motion paths in (6.2).

Furthermore, we evaluate each of those 1250 combinations on several fixed budgets of size N ranging from $N = 1$ to some maximal N_f in increments of 1. The maximal budget N_f depends on the integrand family. We choose $N_f = 30$ and $N_f = 50$ for the smooth and non-smooth univariate problems, respectively. These numbers are chosen to illustrate the performance change of the BQ methods across a relevant range of N . In total, we execute 100 000 BQ runs, which were run sequentially on a general purpose `m1.xlarge` instance with 4 vCPUs.

6.1.6 Scores

Recall from Eqs. (2.9) and (2.11) that $\mu_{\mathcal{D}}$ and $\Sigma_{\mathcal{D}}$ stand for the posterior mean and variance, respectively, of the conjugate BQ model. The mean $\mu_{\mathcal{D}}$ serves as an estimator for the true integral value $I_P(f)$ and $\Sigma_{\mathcal{D}}$ describes the spread of the posterior distribution $I_P(f) \mid \mathcal{D}$.

In the experiments, we evaluate the behavior of the BQ methods with two empirical scores. The first score is the *error score*

$$\frac{1}{T} \sum_{t=1}^T \left| \frac{I_P(f_t) - \mu_{\mathcal{D},t}}{I_P(f_t)} \right|.$$

The error score quantifies the performance of $\mu_{\mathcal{D}}$ as an estimator for the ground truth $I_P(f)$. More precisely, the error score is the average relative error between $\mu_{\mathcal{D}}$ and $I_P(f)$ over the test functions. The subscript t indicates the t^{th} test integrand; the average is computed over the $T = 50$ integrands for each family separately.³

The second score we use is the *calibration score*

$$\frac{1}{T} \sum_{t=1}^T \frac{|I_P(f_t) - \mu_{\mathcal{D},t}|}{\Sigma_{\mathcal{D},t}^{1/2}}.$$

The calibration score quantifies the calibration of the BQ method, or how well the credible interval of the posterior describes the error between the estimator $\mu_{\mathcal{D}}$ and the true integral value $I_P(f)$. The calibration score equals the average absolute error in units of the corresponding credible intervals. For each test function, the 68% credible interval centered around $I_P(f) \mid \mathcal{D} = I_P(f)$ and 1 standard deviation wide to each side is $[\mu_{\mathcal{D}} - \Sigma_{\mathcal{D}}^{1/2}, \mu_{\mathcal{D}} + \Sigma_{\mathcal{D}}^{1/2}]$, or equivalently $[-\Sigma_{\mathcal{D}}^{1/2}, \Sigma_{\mathcal{D}}^{1/2}]$ for the shifted value $I_P(f) \mid \mathcal{D} - \mu_{\mathcal{D}} = I_P(f) - \mu_{\mathcal{D}}$. Taking the absolute value lets us compare the size of $|I_P(f) - \mu_{\mathcal{D}}|$ (enumerator) to $\Sigma_{\mathcal{D}}^{1/2}$ (denominator). For an individual test function f , the score is smaller than 1 if the true value $I_P(f)$ lies inside the credible interval and larger than 1 if it lies outside. Hence, a somewhat *constant* calibration score across different N (e.g., hovering around 1) suggests that the BQ method is well calibrated. An increasing or decaying score suggests that the method is prone to over- or underconfidence, respectively (Karvonen *et al.*, 2020; Wang, 2021). It is usually not as harmful for a method to be underconfident (too uncertain) than overconfident (too certain).⁴

³Occasional outliers, mainly from failed runs, have been discarded when computing the averages. Outliers are defined as scores that lie more than $1.5 \times \Delta_{1-3}$ outside of the interquartile range Δ_{1-3} of all scores. This is the same definition that SciPy uses for its default boxplots.

⁴The calibration score is a simple proxy to get a general idea of the calibration and provides by no means a complete picture. We refer to Cockayne *et al.* (2022) for a formal treatment of calibration (see in particular Remark 9).

6.2 Results

This section describes the qualitative behavior of the BQ methods on the families of test functions.

6.2.1 Smooth test functions (sampling axis)

Figure 6.2 depicts the performance of the 5 models along the *sampling* axis for the family of univariate smooth test functions. Each row shows a single model with kernels of increasing smoothness from top to bottom. The first 4 rows show models of finite, and hence of lesser, smoothness than the test functions (Brownian motion, Matérn $1/2$, Matérn $3/2$, and Matérn $5/2$), while the last row shows a model with matching (infinite) smoothness (square exponential). The left and right columns show the error score and the calibration score, respectively. From the plots we can make several qualitative observations.

First, it is apparent that the sampling choice has a major effect on the error score (left column). In this particular example, Legendre roots seem to outperform all other sampling methods considered, followed by IVR. Random samples from P perform the worst across all models. The Sobol sequence and Latin hypercube perform similarly, and generally better than random samples from P . While not covered by the theory in Section 5, the superiority of Legendre roots is likely related to the well-known phenomenon in numerical analysis that nodes which are denser near the boundary of the domain than in its interior are good for approximating and integrating infinitely smooth functions (Trefethen, 2008; Rieger and Zwicknagl, 2014). In Figure 6.4 we observe that the performance of Legendre nodes differs little from other sampling choices when the integrands are non-smooth.

Second, from the right column we see that all models are well calibrated or underconfident for $N \gtrsim 10$, as the calibration scores hover around a constant value or tend to zero. There is a “kink” in the calibration score for random nodes (red) at $N \approx 18$ for nodes sampled from P that vanishes at $N = 20$ (coinciding with a drop of the corresponding error scores). This is an artifact of a “gap” in the randomly sampled nodes that gets filled at $N = 20$. The other, space-filling designs do not

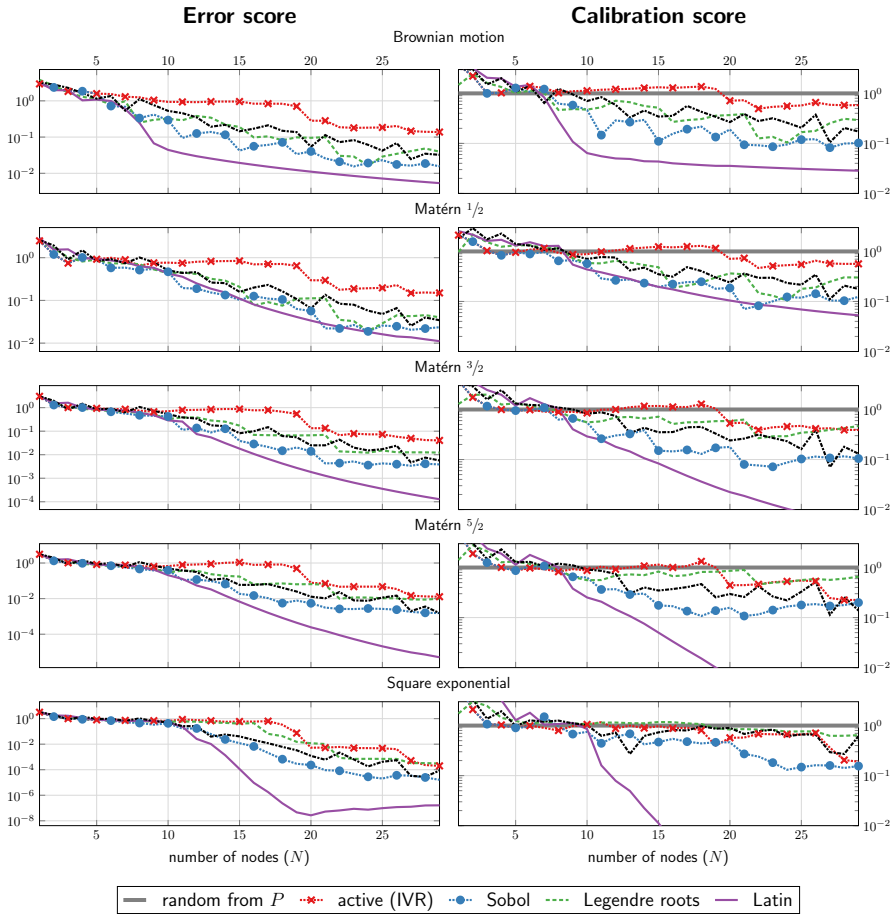


Figure 6.2: Performance of the models along the *sampling* axis on *univariate smooth functions*. Left column: error scores. Right column: calibration scores. Each row shows a single model with kernels of increasing smoothness from top to bottom. The first 4 rows show models of finite, and hence of lesser, smoothness than the test functions (Brownian motion, Matérn $1/2$, Matérn $3/2$, and Matérn $5/2$), while the last row shows a model with matching (infinite) smoothness (square exponential). The left and right columns show the error score and the calibration score, respectively. From the plots we can make several qualitative observations. It is apparent that the sampling choice has a major effect on the error score (left column). In this particular example, Legendre roots seem to outperform all other sampling methods considered, followed by IVR, while random samples from P perform the worst across all models. Further, all models are well calibrated or under-confident for $N \gtrapprox 10$. See main text for more details.

exhibit this fluctuation. The “kink” is less pronounced for more well-specified (i.e., smoother) models. The “kinks” do not average out across the $T = 50$ integrands because the random nodes X are identical for a given N and across all experiments as the same random seed was used (see Section 6.1.3). When the model does not oversmooth a univariate stationary integrand, the sampling choice therefore appears to mostly affect the performance of the mean estimator $\mu_{\mathcal{D}}$. Calibration is decent and largely unaffected by sampling. This makes the BQ methods usable in this particular setting.

The third observation we make is the flattening of the performance score in the bottom left plot of Figure 6.2 for the Legendre nodes at around 10^{-7} . This is likely due to the positive nugget used in the experiments (see Section 6.1.3). Theorems 5.5, 5.6, 5.8 and 5.9 suggest that a positive nugget may either greatly slow the rate of convergence or altogether prevent convergence after a certain accuracy has been reached.

6.2.2 Smooth test functions (model axis)

Figure 6.3 shows the same curves as Figure 6.2 but this time along the *model* axis. That is, rows are now grouped according to the sampling choice such that the effect of the kernel can be observed in each subplot. The nodes X are identical on each row and N across all test functions if sampling is model-independent (rows 1–4). For deterministic nodes this is fulfilled trivially; for random nodes this is because the same random seed was used across experiments (see Section 6.1.3). Hence, solely the effect of the model can be observed in each plot. The nodes on row 5 differ across N , as well as models and test functions, as the IVR sampler is active and model-dependent. Again we make some qualitative observations.

First, the right column shows that all models are well calibrated or underconfident. The “kink” mentioned in Section 6.2.1 at $N \approx 18$ for random nodes from P can be observed in the top right plot. There are no substantial differences between the models, except that the infinitely smooth square exponential model is generally the least underconfident. Because the test integrands are infinitely smooth, this is not surprising.

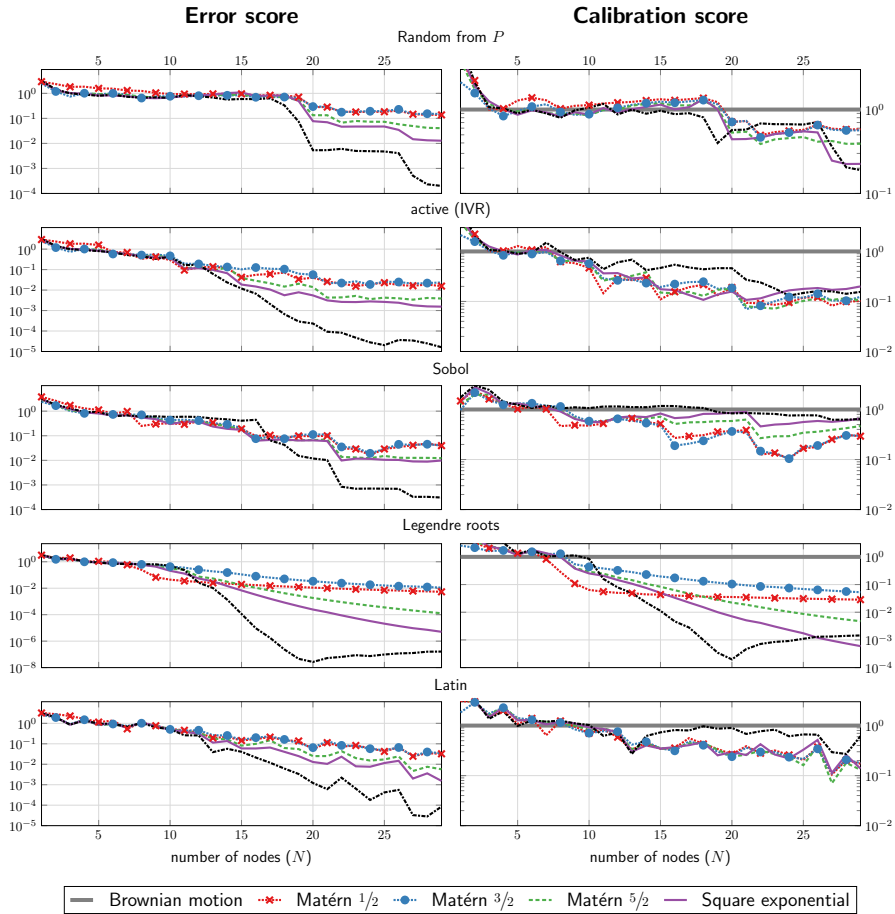


Figure 6.3: Performance of the models along the *model* axis on the family of *univariate smooth functions*. Left column: error scores. Right column: calibration scores. Rows are now grouped according to the sampling choice such that the effect of the kernel can be observed in each subplot. The nodes X are identical on each row and N across all test functions if sampling is model-independent (rows 1–4). For deterministic nodes this is fulfilled trivially; for random nodes this is because the same random seed was used across experiments (see Section 6.1.3). Hence, solely the effect of the model can be observed in each plot. The nodes on row 5 differ across N , as well as models and test functions, as the IVR sampler is active and model-dependent. The models perform better with increasing smoothness; the infinitely differentiable square exponential kernel that matches the smoothness of the test functions performs best across all sampling choices. All models are well calibrated or underconfident. See Section 6.2.2 for more details.

Second, the error score (left column) decreases as the kernel becomes smoother. In particular, the infinitely smooth square exponential kernel performs better than other kernels across all sampling choices. This is consistent with the theoretical guarantees of Section 5, which indicate that smoother kernels can attain faster rates of convergence provided that the integrand is sufficiently smooth. In conclusion, models that undersmooth the truth are decently calibrated but have larger integration errors than models that do not undersmooth.

6.2.3 Non-smooth test functions (sampling axis)

Figure 6.4 depicts the performance of the 5 models along the *sampling* axis for the family of approximate Brownian motion paths. Again, each row shows a single model with kernels of increasing smoothness from top to bottom. The first two rows show models that are continuous but non-differentiable and match the smoothness (or roughness) of the test functions. The models on rows 3–5 are increasingly smooth and thus oversmooth the truth. In particular, the square exponential kernel on row 5 is “infinitely smoother” than the test functions. Again, the left and right columns show the error score and the calibration score, respectively. From the plots we can make the following qualitative observations.

First, the sampling choice has no apparent effect on the error score for the models with matching smoothness (rows 1–2); solely random sampling from P performs slightly poorer. This is quite surprising and unlike the behavior we observed in Section 6.2.1 for the family of smooth test functions. As expected, calibration is good for all sampling schemes.

Second, models that oversmooth (rows 3–5) perform successively worse in error score. Interestingly, random sampling from P reinforces the adverse effect of oversmoothing. This may be explained by the presence of the mesh ratio term $\rho_X^{\alpha-\beta}$ [mesh ratio is define in Eq. (5.6)] in Theorem 5.5: when there is significant oversmoothing ($\alpha - \beta$ is large), this term is large. Moreover, the mesh ratio is large if the nodes exhibit clustering, which explains the behavior of random sampling. The calibration score is less stable and indicates increasing overconfidence across sampling schemes for models that oversmooth.

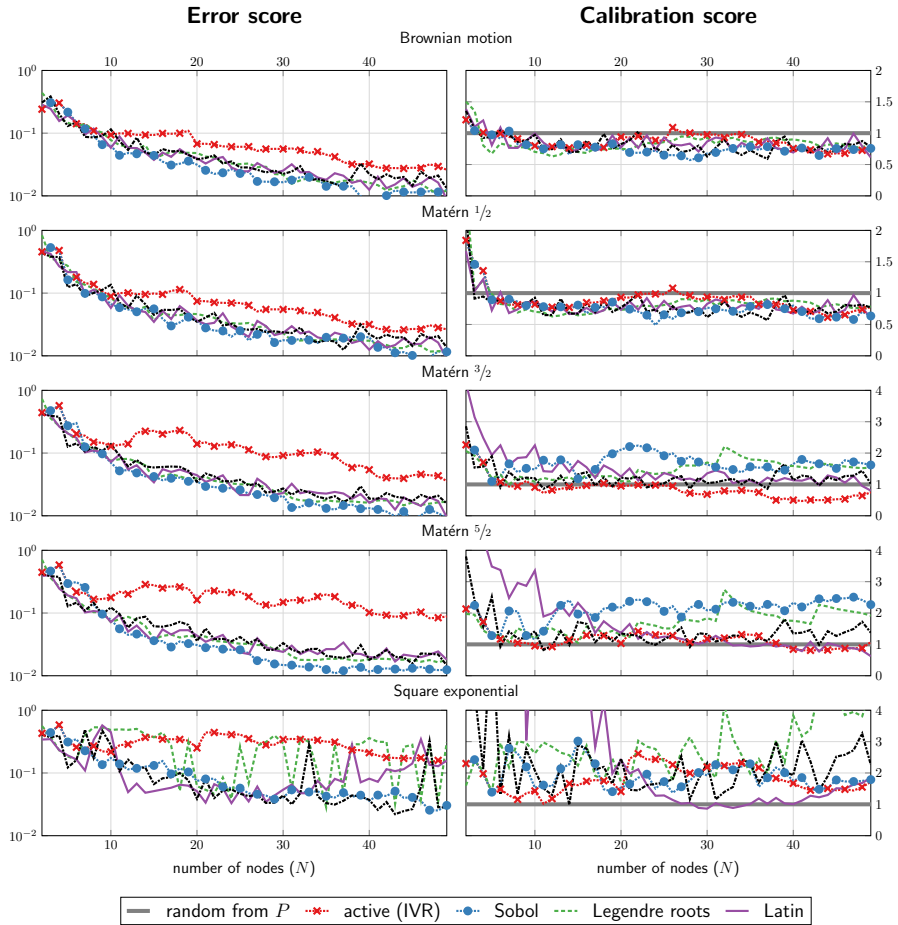


Figure 6.4: Performance of the models along the *sampling* axis on the family of *approximate Brownian motion paths*. Left column: error scores. Right column: calibration scores. Each row shows a single model with kernels of increasing smoothness from top to bottom. The first two rows show models that are continuous but non-differentiable and match the smoothness (or roughness) of the test functions. The models on rows 3–5 are increasingly smooth and thus oversmooth the truth. In particular, the square exponential kernel on row 5 is “infinitely smoother” than the test functions. The sampling scheme has no visible effect on the error score for models with matching smoothness. Random sampling from P is slightly worse than other schemes. All sampling schemes yield well calibrated models. The smoother then model, the worse it performs in both scores when nodes are sampled from P . See Section 6.2.3 for more details.

Unlike undersmoothing, oversmoothing the truth can have a detrimental effect on error and calibration scores across all sampling choices. Oversmoothing the truth slightly can have an undesirable effect on the calibration score while the error score is mostly unaffected unless random sampling from P is used. At the same time, the choice of sampler is less important for models with matching smoothness. Random sampling seems to be the poorest choice for all models, a fact that may be explained by the observation above on the magnitude of the mesh ratio for random nodes.

6.2.4 Non-smooth test functions (model axis)

Figure 6.5 shows the same curves as Figure 6.4 but this time along the *model* axis. That is, rows are now grouped according to the sampling choice such that the effect of the kernel can be observed in each subplot. Again, the nodes X are identical on each row and N across all test functions if sampling is model-independent (rows 1–4). Hence, solely the effect of the model can be observed in each plot. Again, the nodes in the bottom row 5 differ across N , as well as models and test functions, as the IVR sampler is active and model-dependent. We observe the following qualitative behavior.

Unsurprisingly, models that match the roughness of the integrand (Brownian and Matérn $1/2$) perform best on both error and calibration scores for all sampling schemes. Models that oversmooth slightly (Matérn $3/2$ and Matérn $5/2$) have equally good error scores (exception described below) but appear to be somewhat overconfident. The square exponential kernel that oversmooths heavily performs poorly across all sampling schemes. Random sampling from P (row 1) is the only sampling scheme where even slight oversmoothing has an adverse effect on the error score. This effect was given theoretical explanation in Section 6.2.3. Random sampling should therefore be avoided whenever there is a risk of oversmoothing. However, random sampling can be expected to perform well and be well-calibrated if smoothness is correctly specified.

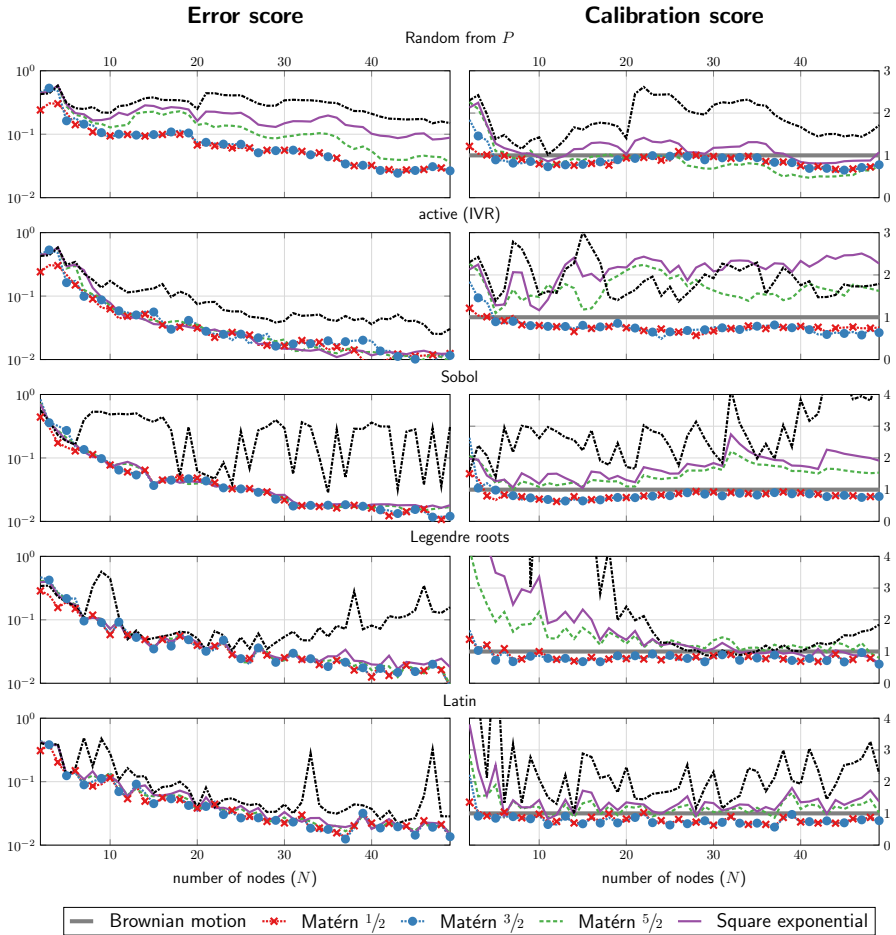


Figure 6.5: Performance of the models along the *model* axis on the family of *approximate Brownian motion paths*. Left column: error scores. Right column: calibration scores. Each row shows a single sampling choice for models with increasing smoothness. The nodes X are identical on each row and N across all test functions if sampling is model-independent (rows 1–4). Hence, solely the effect of the model can be observed in each plot. The nodes in the bottom row 5 differ across N , as well as models and test functions, as the IVR sampler is active and model-dependent. Models that match the smoothness of the integrand perform best on both scores for all sampling schemes. Models perform poorer in error score and can be increasingly overconfident the more they oversmooth the truth; unsurprisingly, the square exponential kernel that oversmooths the truth infinitely performs much poorer than all other models. See Section 6.2.4 for more details.

6.3 Further experimental details

This section contains further details on the implementation of the experiments. The samplers of Table 6.2 were implemented as follows:

- *Random from P (random)*: Nodes are random samples from the integration measure P . We use the method

`uniform` of NumPy's `random.default_rng`

when P is the uniform measure on the unit interval $[0, 1]$.

- *Latin hypercube (random)*: Nodes are samples of a Latin hypercube (McKay *et al.*, 1979), a distribution of space-filling pseudo hypergrids. We use the method

`sample` of SciPy's `stats.qmc.LatinHypercube`

with default arguments.

- *Sobol sequence (deterministic)*: Nodes are the Sobol sequence in dimension d . We use the method

`random_base2` of SciPy's `stats.qmc.Sobol`

with `scramble=False`. A Sobol sequence produces points for N , an integer power of 2. For N which are not powers of 2, we use the next larger sequence and truncate it.

- *Legendre roots (deterministic)*: Nodes are the roots of some Legendre polynomial. We use the method

`roots_legendre` of SciPy's `special`

module.

- *Integral variance reduction (sequential, active, model-dependent)*: Nodes are collected sequentially and myopically with batch size $n = 1$. The acquisition function $a(\tilde{\mathbf{x}} | P)$ is the integral variance reduction (IVR) acquisition function from Section 4.2.3. We use

`IntegralVarianceReduction`

of EmuKit's `quadrature.acquisitions`

module as well as its sequential acquisition loop.

7

Proofs for Chapter 5

This chapter contains proofs for the results in Chapter 5. We begin with some preliminaries.

7.1 Reproducing kernel Hilbert spaces

Some basic properties of RKHSs have appeared already in Section 2.3. A kernel $k: D \times D \rightarrow \mathbb{R}$ is positive-semidefinite if the kernel Gram matrix $\mathbf{K}_{XX} = (k(\mathbf{x}_i, \mathbf{x}_j))_{i,j=1}^N$ is positive-semidefinite for any $N \in \mathbb{N}$ and any set $X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subseteq D$ of points. The classical Moore–Aronszajn theorem (Aronszajn, 1950) states that for every positive-semidefinite kernel there is a unique Hilbert space $\mathcal{H}(k)$, the RKHS of k , which consists of real-valued functions defined on D and is equipped with an inner product $\langle \cdot, \cdot \rangle_{\mathcal{H}(k)}$ and the corresponding norm $\|\cdot\|_{\mathcal{H}(k)}$ such that the kernel has the following reproducing property:

$$\langle g, k(\cdot, \mathbf{x}) \rangle_{\mathcal{H}(k)} = g(\mathbf{x}) \quad \text{for all } g \in \mathcal{H}(k) \text{ and } \mathbf{x} \in D. \quad (7.1)$$

Whenever it is necessary to be precise about the domain of the functions in the RKHS, we say that $\mathcal{H}(D)$ is an RKHS *on* D . From the Riesz representation theorem it follows that the reproducing property extends to any continuous linear functional on $\mathcal{H}(k)$. Namely, if $L: \mathcal{H}(k) \rightarrow \mathbb{R}$

is a continuous linear functional, then

$$L(g) = \langle g, k_L \rangle_{\mathcal{H}(k)} \quad \text{for all } g \in \mathcal{H}(k),$$

where k_L , the representer of L , is defined as $k_L(\mathbf{x}) = L(k(\cdot, \mathbf{x}))$. Selecting $L = I_P$ yields

$$I_P(g) = \langle g, k_P \rangle_{\mathcal{H}(k)},$$

where $k_P(\mathbf{x}) = \int_D k(\mathbf{x}', \mathbf{x}) dP(\mathbf{x}')$ is the kernel mean embedding in (2.10).

It is usually not straightforward to decide whether or not a given function is an element of the RKHS of a given kernel. There are two characterisations of the RKHS that are occasionally helpful. If there are functions $\{\varphi_i\}_{i=1}^\infty$ such that

$$k(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^\infty \varphi_i(\mathbf{x}) \varphi_i(\mathbf{x}') \quad \text{for all } \mathbf{x}, \mathbf{x}' \in D,$$

then every $g \in \mathcal{H}(k)$ may be written as $g = \sum_{i=1}^\infty c_i \varphi_i$ (the convergence is both in $\|\cdot\|_{\mathcal{H}(k)}$ and pointwise) for some $c_i \in \mathbb{R}$ such that $\sum_{i=1}^\infty c_i^2 < \infty$ (Paulsen and Raghupathi, 2016, Sec. 2.1). Conversely, if no such expansion for square-summable c_i exists, then $g \notin \mathcal{H}(k)$.

If the kernel is stationary on $\mathbb{R}^d$, in that there is a function $\Phi_k: \mathbb{R}^d \rightarrow \mathbb{R}$ such that $k(\mathbf{x}, \mathbf{x}') = \Phi_k(\mathbf{x} - \mathbf{x}')$ for all $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$, then the RKHS of k on $\mathbb{R}^d$ may be expressed in terms of Fourier transforms (Kimeldorf and Wahba, 1970; Wendland, 2005, Thm. 10.12). Namely, if Φ_k is continuous and integrable, then $\mathcal{H}(k)$ is

$$\left\{ g \in C(\mathbb{R}^d) \cap L^2(\mathbb{R}^d) : \|g\|_{\mathcal{H}(k)}^2 = \frac{1}{(2\pi)^{d/2}} \int_{\mathbb{R}^d} \frac{|\widehat{g}(\boldsymbol{\xi})|^2}{\widehat{\Phi}_k(\boldsymbol{\xi})} d\boldsymbol{\xi} < \infty \right\}, \quad (7.2)$$

where $\widehat{g}(\boldsymbol{\xi}) = (2\pi)^{-d/2} \int_{\mathbb{R}^d} g(\mathbf{x}) e^{-i\boldsymbol{\xi}^\top \mathbf{x}} d\mathbf{x}$ denotes the Fourier transform. To use the Fourier characterisation on a subset of $\mathbb{R}^d$ we use a standard result on restrictions of RKHSs (Berlinet and Thomas-Agnan, 2004, Sec. 1.4.2). Let $\mathcal{H}(k, \mathbb{R}^d)$ denote the RKHS in (7.2) and let $\mathcal{H}(k)$ be the RKHS on $D \subset \mathbb{R}^d$. Then $\mathcal{H}(k)$ consists of functions $g: D \rightarrow \mathbb{R}$ for which there exists an *extension* $g_e \in \mathcal{H}(k, \mathbb{R}^d)$ such that $g_e(\mathbf{x}) = g(\mathbf{x})$ for all $\mathbf{x} \in D$ (i.e., $g_e|_D = g$). Moreover,

$$\|g\|_{\mathcal{H}(k)} = \min\{\|g_e\|_{\mathcal{H}(k, \mathbb{R}^d)} : g_e \in \mathcal{H}(k, \mathbb{R}^d) \text{ is s.t. } g_e|_D = g\}.$$

7.2 Sobolev spaces

In Section 5.1.1, we defined the Sobolev space $H^\alpha([0, 1]^d)$ of integer smoothness α as the space of functions whose weak derivatives up to order α exist and are in $L^2([0, 1]^d)$. To allow fractional smoothness we may turn to Fourier transforms. For any $\alpha > 0$, we let $H^\alpha(\mathbb{R}^d)$ be the Hilbert space of functions $g \in L^2(\mathbb{R}^d)$ such that

$$\|g\|_{H^\alpha(\mathbb{R}^d)}^2 = \int_{\mathbb{R}^d} (1 + \|\boldsymbol{\xi}\|^2)^\alpha |\widehat{g}(\boldsymbol{\xi})|^2 d\boldsymbol{\xi} < \infty. \quad (7.3)$$

To illustrate the relationship of this norm to condition in (5.5) expressed using weak derivatives, consider $\alpha \in \mathbb{N}$ and $d = 1$. We may then use the binomial theorem, the properties of the Fourier transform, and Parseval's identity to compute

$$\begin{aligned} \|g\|_{H^\alpha(\mathbb{R})}^2 &= \int_{\mathbb{R}} (1 + \xi^2)^\alpha |\widehat{g}(\xi)|^2 d\xi = \sum_{n=0}^{\alpha} \binom{\alpha}{n} \int_{\mathbb{R}} |\xi^n \widehat{g}(\xi)|^2 d\xi \\ &= \sum_{n=0}^{\alpha} \binom{\alpha}{n} \int_{\mathbb{R}} [D^n g(x)]^2 dx, \end{aligned}$$

which is, up to binomial coefficient factors for each integral, equal to the left-hand side of (5.5) defined on $\mathbb{R}$ instead of on $[0, 1]$. A fractional Sobolev space and its norm on $[0, 1]^d$ can be defined via restrictions in a way analogous to how we related $\mathcal{H}(k)$ to $\mathcal{H}(k, \mathbb{R}^d)$ in Section 7.1. One can then show that g is in the resulting space if and only if it satisfies (5.5).

7.3 Proofs for Section 5.1

This section contains proofs for the results in Section 5.1.

7.3.1 Proofs for generic nodes

It is a basic result (e.g., Kanagawa *et al.*, 2018, Sec. 3.2) that the regularised posterior mean

$$m_{\mathcal{D}, \lambda}(\mathbf{x}) = \mathbf{k}_X(\mathbf{x})^\top (\mathbf{K}_{XX} + \lambda \mathbf{I}_N)^{-1} \mathbf{y}$$

is the unique solution to a regularised least-squares problem:

$$m_{\mathcal{D},\lambda} = \arg \min_{s \in \mathcal{H}(k)} \left\{ \sum_{i=1}^N [f(\mathbf{x}_i) - s(\mathbf{x}_i)]^2 + \lambda \|s\|_{\mathcal{H}(k)}^2 \right\}. \quad (7.4)$$

Because $m_{\mathcal{D}}$ is an interpolant to f , which is to say that $m_{\mathcal{D}}(\mathbf{x}_i) = f(\mathbf{x}_i)$ for every $i = 1, \dots, N$, and an element of $\mathcal{H}(k)$, we have from (7.4) that

$$\sum_{i=1}^N [f(\mathbf{x}_i) - m_{\mathcal{D},\lambda}(\mathbf{x}_i)]^2 + \lambda \|m_{\mathcal{D},\lambda}\|_{\mathcal{H}(k)}^2 \leq \lambda \|m_{\mathcal{D}}\|_{\mathcal{H}(k)}^2. \quad (7.5)$$

Consequently,

$$\left(\sum_{i=1}^N [f(\mathbf{x}_i) - m_{\mathcal{D},\lambda}(\mathbf{x}_i)]^2 \right)^{1/2} \leq \sqrt{\lambda} \|m_{\mathcal{D}}\|_{\mathcal{H}(k)}. \quad (7.6)$$

We also need to recall the minimum-norm interpolation property (2.19) of the conditional mean, which states that

$$m_{\mathcal{D}} = \arg \min_{s \in \mathcal{H}(k)} \{ \|s\|_{\mathcal{H}(k)} : s(\mathbf{x}_i) = f(\mathbf{x}_i) \text{ for every } i = 1, \dots, N \}, \quad (7.7)$$

and its particular consequence $\|m_{\mathcal{D}}\|_{\mathcal{H}(k)} \leq \|f\|_{\mathcal{H}(k)}$ if $f \in \mathcal{H}(k)$.

Observe that (2.9) and its regularised variant, followed by Jensen's inequality, yield

$$\begin{aligned} |I_P(f) - \mu_{\mathcal{D},\lambda}| &= \left| \int_D [f(\mathbf{x}) - m_{\mathcal{D},\lambda}(\mathbf{x})] \, dP(\mathbf{x}) \right| \\ &\leq \int_D |f(\mathbf{x}) - m_{\mathcal{D},\lambda}(\mathbf{x})| \, dP(\mathbf{x}). \end{aligned} \quad (7.8)$$

Assumption 5.1 and a second application of Jensen's inequality then produce

$$\begin{aligned} |I_P(f) - \mu_{N,\lambda}| &\leq p_{\max} \int_{[0,1]^d} |f(\mathbf{x}) - m_{\mathcal{D},\lambda}(\mathbf{x})| \, d\mathbf{x} \\ &\leq p_{\max} \|f - m_{\mathcal{D},\lambda}\|_{L^2([0,1]^d)}. \end{aligned} \quad (7.9)$$

To bound the integration error it is therefore sufficient to bound the error of approximating f with $m_{\mathcal{D},\lambda}$ in $L^2([0,1]^d)$ norm. This is what is done in most of the proofs below.

Proof of Proposition 5.3. Because the node sets are assumed nested, m_{N+1} interpolates f at $\mathbf{x}_1, \dots, \mathbf{x}_N$. It thus follows from (7.7) that for any sequence of nodes the norm $\|m_N\|_{\mathcal{H}(k)}$ is non-decreasing in N . Moreover, $\|m_N\|_{\mathcal{H}(k)} \leq \|f\|_{\mathcal{H}(k)}$ since $f \in \mathcal{H}(k)$. From (2.1) and the reproducing property (7.1) we get

$$\langle m_{N'}, m_N \rangle_{\mathcal{H}(k)} = \langle m_N, m_N \rangle_{\mathcal{H}(k)} = \mathbf{f}_X^\top \mathbf{K}_{XX}^{-1} \mathbf{f}_X$$

for any $N' \geq N$. Therefore

$$\begin{aligned} \|m_{N'} - m_N\|_{\mathcal{H}(k)}^2 &= \|m_{N'}\|_{\mathcal{H}(k)}^2 - 2\langle m_{N'}, m_N \rangle_{\mathcal{H}(k)} + \|m_N\|_{\mathcal{H}(k)}^2 \\ &= \|m_{N'}\|_{\mathcal{H}(k)}^2 - \|m_N\|_{\mathcal{H}(k)}^2, \end{aligned}$$

which tends to zero as $N \rightarrow \infty$ because $\|m_N\|_{\mathcal{H}(k)}$ is non-decreasing and bounded. Hence $(m_N)_{N=1}^\infty$ is Cauchy in $\mathcal{H}(k)$ and consequently has a limit $m \in \mathcal{H}(k)$. We show that the assumptions imply $m = f$. It is a standard result that the continuity of $k(\cdot, \mathbf{x})$ for all $\mathbf{x} \in D$ and the boundedness of $k(\mathbf{x}, \mathbf{x})$ imply that every element of $\mathcal{H}(k)$ is continuous (e.g., Steinwart and Christmann, 2008, Lem. 4.28). Hence m and f are continuous. The assumption $h_N \rightarrow 0$ is equivalent to $\{\mathbf{x}_i\}_{i=1}^\infty$ being dense in D . A continuous function is fully determined by its values at a dense set. Because $m(\mathbf{x}_i) = f(\mathbf{x}_i)$ for every $i \in \mathbb{N}$, it thus follows that $m = f$. Therefore $\|f - m_N\|_{\mathcal{H}(k)} \rightarrow 0$ as $N \rightarrow \infty$. From the reproducing property and the Cauchy–Schwarz inequality it then follows that m_N tends to f uniformly on D :

$$\begin{aligned} \sup_{\mathbf{x} \in D} |m_N(\mathbf{x}) - f(\mathbf{x})| &= \sup_{\mathbf{x} \in D} |\langle m_N - f, k(\cdot, \mathbf{x}) \rangle_{\mathcal{H}(k)}| \\ &\leq \|m_N - f\|_{\mathcal{H}(k)} \sup_{\mathbf{x} \in D} \|k(\cdot, \mathbf{x})\|_{\mathcal{H}(k)} \\ &= \|m_N - f\|_{\mathcal{H}(k)} \sup_{\mathbf{x} \in D} \sqrt{k(\mathbf{x}, \mathbf{x})} \\ &= C_k \|m_N - f\|_{\mathcal{H}(k)}, \end{aligned}$$

where $C_k = \sup_{\mathbf{x} \in D} \sqrt{k(\mathbf{x}, \mathbf{x})} < \infty$. Equation (7.8) and Assumption 5.1 yield

$$\begin{aligned} |I_P(f) - \mu_N| &\leq p_{\max} \int_{[0,1]^d} |f(\mathbf{x}) - m_N(\mathbf{x})| \, d\mathbf{x} \leq C_k p_{\max} \|f - m_N\|_{\mathcal{H}(k)} \\ &\rightarrow 0. \end{aligned}$$

This concludes the proof. $\square$

Proposition 5.4 is likely an old result. We follow the proof of Corollary 2.8 in Gräf (2013).

Proof of Proposition 5.4. Let $\mathbf{u} = (\frac{1}{N}, \dots, \frac{1}{N}) \in \mathbb{R}^N$. By Proposition 2.6 and (2.16),

$$\begin{aligned} \Sigma_N &= \min_{\mathbf{v} \in \mathbb{R}^N} e_k(\mathbf{v}, X)^2 \leq e_k(\mathbf{u}, X)^2 \\ &= k_{PP} - 2\mathbf{u}^\top \mathbf{k}_{PX} + \mathbf{u}^\top \mathbf{K}_{XX} \mathbf{u} \\ &= k_{PP} - \frac{2}{N} \sum_{i=1}^N k_P(\mathbf{x}_i) + \frac{1}{N^2} \sum_{i=1}^N \sum_{j=1}^N k(\mathbf{x}_i, \mathbf{x}_j). \end{aligned}$$

Since $\mathbf{x}_i \sim P$ are i.i.d., taking expectation and using the definitions of k_P and k_{PP} yields

$$\begin{aligned} \mathbb{E}[\Sigma_N] &\leq \mathbb{E}[k_{PP}] - \frac{2}{N} \sum_{i=1}^N \mathbb{E}[k_P(\mathbf{x}_i)] + \frac{1}{N^2} \sum_{i=1}^N \sum_{j=1}^N \mathbb{E}[k(\mathbf{x}_i, \mathbf{x}_j)] \\ &= k_{PP} - \frac{2}{N} \sum_{i=1}^N k_{PP} + \frac{1}{N^2} \sum_{i \neq j} k_{PP} + \frac{1}{N^2} \sum_{i=1}^N \int_{[0,1]^d} k(\mathbf{x}, \mathbf{x}) \, dP(\mathbf{x}) \\ &= \left(1 - 2 + \frac{N^2 - N}{N^2}\right) k_{PP} + \frac{1}{N} \int_{[0,1]^d} k(\mathbf{x}, \mathbf{x}) \, dP(\mathbf{x}) \\ &= \frac{1}{N} \left(\int_{[0,1]^d} k(\mathbf{x}, \mathbf{x}) \, dP(\mathbf{x}) - k_{PP} \right). \end{aligned}$$

The claim follows from (2.18). $\square$

7.3.2 Proofs for Section 5.1.1

In this section we use $\lesssim$ to denote an inequality which holds up to a constant coefficient independent of f , X , the nugget λ , the length-scale ℓ and the scale σ . Let $\alpha > d/2$ and $\nu = \alpha - d/2 > 0$. Here we assume that k is the isotropic Matérn kernel of order ν given by

$$\begin{aligned} k(\mathbf{x}, \mathbf{x}') &= \Phi_k(\|\mathbf{x} - \mathbf{x}'\|) \\ &= \sigma^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu} \|\mathbf{x} - \mathbf{x}'\|}{\ell} \right)^\nu K_\nu \left(\frac{\sqrt{2\nu} \|\mathbf{x} - \mathbf{x}'\|}{\ell} \right), \end{aligned}$$

where Γ denotes the gamma function and K_ν the modified Bessel function of the second kind. The length-scale $\ell > 0$ controls how “wiggly” the processes are, while the regularity parameter $\nu > 0$ determines the differentiability of a process with Matérn covariance. Because the Fourier transform of a Matérn kernel is

$$\widehat{\Phi}_k(\boldsymbol{\xi}) = \sigma^2 \frac{2^{d/2} \Gamma(\nu + d/2)}{\Gamma(\nu)} \left(\frac{2\nu}{\ell^2} \right)^\nu \left(\frac{2\nu}{\ell^2} + \|\boldsymbol{\xi}\|^2 \right)^{-(\nu+d/2)},$$

one may compute from the Fourier definition of the Sobolev norm that the RKHS of a Matérn kernel is *norm-equivalent* to a Sobolev space of order ν , which in this case means that $\mathcal{H}(k) = H^\alpha([0, 1]^d)$ as sets of functions and

$$C_k \|g\|_{H^\alpha([0,1]^d)} \leq \|g\|_{\mathcal{H}(k)} \leq C'_k \|g\|_{H^\alpha([0,1]^d)}. \quad (7.10)$$

for the positive constants

$$C_k = c \frac{1}{\sigma \ell^{d/2}} \max \left\{ 1, \frac{\sqrt{2\nu}}{\ell} \right\}^{-1}, \quad C'_k = c' \frac{1}{\sigma \ell^{d/2}} \max \left\{ 1, \frac{\ell}{\sqrt{2\nu}} \right\}, \quad (7.11)$$

where c and c' are positive constants that depend only on d and ν . See, for example, Lemma 3.4 and its proof in Teckentrup (2020) for the details of this derivation. Consequently,

$$C_k^{-1} C'_k \lesssim \max \left\{ 1, \frac{\ell}{\sqrt{2\nu}} \right\} \max \left\{ 1, \frac{\sqrt{2\nu}}{\ell} \right\} = \max \left\{ \frac{\ell}{\sqrt{2\nu}}, \frac{\sqrt{2\nu}}{\ell} \right\}. \quad (7.12)$$

The following result is Theorem 3.2 in Arcangéli *et al.* (2012) with $n = d$, $p = q = 2$, $s = \tau$, $r = \kappa$, $\varkappa = 2$ and $\Omega = [0, 1]^d$. The requirement in Arcangéli *et al.* (2012) that h_X be sufficiently small can be eliminated by inflating the constant C .

Theorem 7.1 (Sobolev sampling inequality). Let $\kappa > d/2$ and $\tau \in [0, \kappa]$. Then there is a positive constant C , which does not depend on u , X , λ , ℓ or σ , such that

$$\|u\|_{H^\tau([0,1]^d)} \leq C \left(h_X^{\kappa-\tau} \|u\|_{H^\kappa([0,1]^d)} + h_X^{d/2-\tau} \left[\sum_{i=1}^N u(\mathbf{x}_i)^2 \right]^{1/2} \right)$$

for any $u \in H^\kappa([0, 1]^d)$ and any nodes $X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subset [0, 1]^d$.

The following result is a version of Theorem 4.2 in Narcowich *et al.* (2006) that makes explicit the dependency on the norm-equivalence constants. See also Theorem 16 in Wynne *et al.* (2021). A proof is contained in the proof of Theorem 4.2 in Karvonen *et al.* (2020).

Proposition 7.2. Let $\alpha \geq \beta > d/2$ and suppose that $f \in H^\beta([0, 1]^d)$. Then

$$\|f - m_{\mathcal{D}}\|_{H^\beta([0, 1]^d)} \lesssim C_k^{-1} C'_k \rho_X^{\alpha-\beta} \|f\|_{H^\beta([0, 1]^d)}$$

for any pairwise distinct nodes $X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subset [0, 1]^d$.

Proposition 7.3. Let $\alpha \geq \beta > d/2$ and suppose that $f \in H^\beta([0, 1]^d)$. Then

$$\|m_{\mathcal{D}}\|_{\mathcal{H}(k)} \lesssim C'_k q_X^{-(\alpha-\beta)} \|f\|_{H^\beta([0, 1]^d)}$$

and

$$\|m_{\mathcal{D}}\|_{H^\alpha([0, 1]^d)} \lesssim C_k^{-1} C'_k q_X^{-(\alpha-\beta)} \|f\|_{H^\beta([0, 1]^d)}$$

for any pairwise distinct nodes $X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subset [0, 1]^d$.

Proof. The proof is a combination of the a standard bound for band-limited functions and the norm-equivalence (7.10). By Lemma A.1 in Karvonen *et al.* (2020) there is a function $f_\beta \in H^\alpha([0, 1]^d)$ such that $f_\beta(\mathbf{x}_i) = f(\mathbf{x}_i)$ for every $i = 1, \dots, N$ and

$$\|f_\beta\|_{H^\alpha([0, 1]^d)} \lesssim q_X^{-(\alpha-\beta)} \|f\|_{H^\beta([0, 1]^d)}. \quad (7.13)$$

Since f_β interpolates f and, by the norm-equivalence, is an element of $\mathcal{H}(k)$, it is one of the functions over which the minimum in (7.7) is taken. Therefore $\|m_{\mathcal{D}}\|_{\mathcal{H}(k)} \leq \|f_\beta\|_{\mathcal{H}(k)}$. Consequently, norm-equivalence and (7.13) give

$$\begin{aligned} \|m_{\mathcal{D}}\|_{H^\alpha([0, 1]^d)} &\leq C_k^{-1} \|m_{\mathcal{D}}\|_{\mathcal{H}(k)} \leq C_k^{-1} \|f_\beta\|_{\mathcal{H}(k)} \\ &\leq C_k^{-1} C'_k \|f_\beta\|_{H^\alpha([0, 1]^d)} \\ &\lesssim C_k^{-1} C'_k q_X^{-(\alpha-\beta)} \|f\|_{H^\beta([0, 1]^d)}, \end{aligned}$$

which is the claim. $\square$

Proof of Theorem 5.5. The triangle inequality gives

$$\|f - m_{\mathcal{D}, \lambda}\|_{L^2([0, 1]^d)} \leq \|f - m_{\mathcal{D}}\|_{L^2([0, 1]^d)} + \|m_{\mathcal{D}} - m_{\mathcal{D}, \lambda}\|_{L^2([0, 1]^d)}.$$

Since $f - m_{\mathcal{D}} \in H^\beta([0, 1]^d)$ and $m_{\mathcal{D}} - m_{\mathcal{D}, \lambda} \in H^\alpha([0, 1]^d)$ by the norm-equivalence, we may apply Theorem 7.1 with (a) $\tau = 0$, $\kappa = \beta$ and $u = f - m_{\mathcal{D}, \lambda}$ and (b) $\tau = 0$, $\kappa = \alpha$ and $u = m_{\mathcal{D}} - m_{\mathcal{D}, \lambda}$. This yields

$$\begin{aligned} \|f - m_{\mathcal{D}, \lambda}\|_{L^2([0, 1]^d)} &\lesssim h_X^\beta \|f - m_{\mathcal{D}}\|_{H^\beta([0, 1]^d)} + h_X^\alpha \|m_{\mathcal{D}} - m_{\mathcal{D}, \lambda}\|_{H^\alpha([0, 1]^d)} \\ &\quad + h_X^{d/2} \left(\sum_{i=1}^N [m_{\mathcal{D}}(\mathbf{x}_i) - m_{\mathcal{D}, \lambda}(\mathbf{x}_i)]^2 \right)^{1/2}. \end{aligned}$$

From (7.6) we get

$$h_X^{d/2} \left(\sum_{i=1}^N [m_{\mathcal{D}}(\mathbf{x}_i) - m_{\mathcal{D}, \lambda}(\mathbf{x}_i)]^2 \right)^{1/2} \leq h_X^{d/2} \sqrt{\lambda} \|m_{\mathcal{D}}\|_{\mathcal{H}(k)},$$

while (7.5) gives

$$\begin{aligned} \|m_{\mathcal{D}} - m_{\mathcal{D}, \lambda}\|_{H^\alpha([0, 1]^d)} &\leq \|m_{\mathcal{D}}\|_{H^\alpha([0, 1]^d)} + \|m_{\mathcal{D}, \lambda}\|_{H^\alpha([0, 1]^d)} \\ &\leq 2 \|m_{\mathcal{D}}\|_{H^\alpha([0, 1]^d)}. \end{aligned}$$

Therefore

$$\begin{aligned} \|f - m_{\mathcal{D}, \lambda}\|_{L^2([0, 1]^d)} &\lesssim h_X^\beta \|f - m_{\mathcal{D}}\|_{H^\beta([0, 1]^d)} + h_X^\alpha \|m_{\mathcal{D}}\|_{H^\alpha([0, 1]^d)} \\ &\quad + h_X^{d/2} \sqrt{\lambda} \|m_{\mathcal{D}}\|_{\mathcal{H}(k)}. \end{aligned}$$

Applying Propositions 7.2 and 7.3 and the definition $\rho_X = h_X/q_X$ then yields

$$\begin{aligned} \|f - m_{\mathcal{D}, \lambda}\|_{L^2([0, 1]^d)} &\lesssim C_k^{-1} C'_k h_X^\beta \rho_X^{\alpha-\beta} \|f\|_{H^\beta([0, 1]^d)} \\ &\quad + C'_k h_X^{d/2} q_X^{-(\alpha-\beta)} \sqrt{\lambda} \|f\|_{H^\beta([0, 1]^d)}. \end{aligned}$$

Using (7.11) and (7.12) yields

$$\begin{aligned} \|f - m_{\mathcal{D}, \lambda}\|_{L^2([0, 1]^d)} &\lesssim \max \left\{ \frac{\ell}{\sqrt{2\nu}}, \frac{\sqrt{2\nu}}{\ell} \right\} h_X^\beta \rho_X^{\alpha-\beta} \|f\|_{H^\beta([0, 1]^d)} \\ &\quad + \frac{1}{\sigma \ell^{d/2}} \max \left\{ 1, \frac{\ell}{\sqrt{2\nu}} \right\} h_X^{d/2} q_X^{-(\alpha-\beta)} \sqrt{\lambda} \|f\|_{H^\beta([0, 1]^d)}, \end{aligned}$$

which, when inserted in (7.9), concludes the proof. $\square$

Proof of Theorem 5.7. Recall the definitions and results from Section 2.3.1. Because $\mathcal{H}(k)$ is norm-equivalent to the Sobolev space $H^\alpha([0, 1]^d)$, Theorems 1 and 2 and Corollary 2 in Krieg and Sonnleitner (2024) imply that

$$\begin{aligned} \mathbb{E}[\Sigma_N^{1/2}] &= \mathbb{E}\left[\min_{\mathbf{v} \in \mathbb{R}^N} \sup_{\|g\|_{\mathcal{H}(k)} \leq 1} \left| \int_{[0,1]^d} g(\mathbf{x}) \, d\mathbf{x} - \sum_{i=1}^N v_i g(\mathbf{x}_i) \right| \right] \\ &\leq C_k^{-1} \mathbb{E}\left[\min_{\mathbf{v} \in \mathbb{R}^N} \sup_{\|g\|_{H^\alpha([0,1]^d)} \leq 1} \left| \int_{[0,1]^d} g(\mathbf{x}) \, d\mathbf{x} - \sum_{i=1}^N v_i g(\mathbf{x}_i) \right| \right] \end{aligned}$$

decays as $N^{-\alpha/d}$, where the second inequality uses (7.10). From (2.18) we then obtain

$$\mathbb{E}[|I_P(f) - \mu_N|] \leq \|f\|_{\mathcal{H}(k)} \mathbb{E}[\Sigma_N^{1/2}],$$

so that the claim follows from (7.10) and (7.12) and the assumption on the boundedness of the length-scale. $\square$

7.3.3 Proofs for Section 5.1.3

Proof of Theorem 5.8. The theorem is an immediate consequence of (7.9) and Theorem 6.1 in Rieger and Zwicknagl (2010) with $q = 2$. $\square$

Theorems 5.8 and 5.9 have previously appeared as Theorem 2.20 in Karvonen (2019). Theoretical guarantees for explicitly active Bayesian quadrature based on the square exponential kernel may be found in Kanagawa and Hennig (2019, Sec. 4.1). Further guarantees, including on the unbounded domain $D = \mathbb{R}^d$, could be derived from the results in Kuo and Woźniakowski (2012), Kuo *et al.* (2017), Karvonen *et al.* (2021), and Karvonen and Suzuki (2025).

7.4 Proofs for Section 5.2

Proof of Theorem 5.10. Because the transformation ψ is strictly increasing, maximization of (5.9) is equivalent to maximization of

$$\mathbf{k}_{PX(\tilde{\mathbf{x}})}^\top \mathbf{K}_{X(\tilde{\mathbf{x}})X(\tilde{\mathbf{x}})}^{-1} \mathbf{k}_{PX(\tilde{\mathbf{x}})},$$

which in turn is equivalent to minimization of the integral variance

$$\Sigma_{\mathcal{D}_n(\tilde{\mathbf{x}})} = k_{PP} - \mathbf{k}_{PX(\tilde{\mathbf{x}})}^\top \mathbf{K}_{X(\tilde{\mathbf{x}})X(\tilde{\mathbf{x}})}^{-1} \mathbf{k}_{PX(\tilde{\mathbf{x}})}$$

given the augmented data $\mathcal{D}_n(\tilde{\mathbf{x}}) = \mathcal{D}_n \cup \{(\tilde{\mathbf{x}}, f(\tilde{\mathbf{x}}))\} = \{(\mathbf{x}_i, f(\mathbf{x}_i))\}_{i=1}^n \cup \{(\tilde{\mathbf{x}}, f(\tilde{\mathbf{x}}))\}$. From the interpretation (see Sections 2.3.1 and 4.2.3) of the integral variance as a worst-case error in the RKHS $\mathcal{H}(k)$, we see that the sequential nodes that we consider are equal to the nodes selected by the *greedy algorithm* in Section 4 of Santin *et al.* (2022). We may therefore invoke Theorem 5.1 in Santin *et al.* (2022) and Proposition 2.6, which together give

$$\Sigma_{\mathcal{D}_n}^{1/2} = \min_{\mathbf{v} \in \mathbb{R}^N} e_k(\mathbf{v}, X_N) \leq CN^{-1/2}$$

for a positive C that depends only on D , P and k . The claim then follows from (2.18). $\square$

Acknowledgements

MM's work was mainly performed while at the University of Tübingen. MM gratefully acknowledges financial support by the European Research Council through ERC StG Action 757275 / PANAMA; the DFG Cluster of Excellence “Machine Learning - New Perspectives for Science”, EXC 2064/1, project number 390727645; the German Federal Ministry of Education and Research (BMBF) through the Tübingen AI Center (FKZ: 01IS18039A); and funds from the Ministry of Science, Research and Arts of the State of Baden-Württemberg. TK was generously supported by the Research Council of Finland projects 338567 (“Scalable, adaptive and reliable probabilistic integration”), 359183 (“Flagship of Advanced Mathematics for Sensing, Imaging and Modelling”), and 368086 (“Inference and approximation under misspecification”). TK acknowledges the research environment provided by ELLIS Institute Finland. We thank Vesa Kaarnioja for correcting some of our misapprehensions about Bayesian quadrature with periodic covariance and two anonymous reviewers for comments and suggestions that helped to improve the survey.

References

- Acerbi, L. (2018a). “An exploration of acquisition and mean functions in variational Bayesian Monte Carlo”. In: *1st Symposium on Advances in Approximate Bayesian Inference*.
- Acerbi, L. (2018b). “Variational Bayesian Monte Carlo”. In: *Advances in Neural Information Processing Systems*. Vol. 31. 8213–8223.
- Adachi, M., S. Hayakawa, S. Hamid, Jørgensen, H. Oberhauser, and M. A. Osborne. (2023a). “SOBER: Scalable batch Bayesian optimization and quadrature using recombination constraints”. *arXiv:2301.11832v1*.
- Adachi, M., S. Hayakawa, Jørgensen, S. Hamid, H. Oberhauser, and M. A. Osborne. (2024). “A quadrature approach for general-purpose batch Bayesian optimization via probabilistic lifting”. *arXiv:2404.12219v1*.
- Adachi, M., S. Hayakawa, M. Jørgensen, H. Oberhauser, and M. A. Osborne. (2022). “Fast Bayesian inference with batch Bayesian quadrature via kernel recombination”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 16533–16547.
- Adachi, M., S. Hayakawa, X. Wan, M. Jørgensen, H. Oberhauser, and M. A. Osborne. (2023b). “Domain-agnostic batch Bayesian optimization with diverse constraints via Bayesian quadrature”. *arXiv:2306.05843v1*.

- Adachi, M., Y. Kuhn, B. Horstmann, A. Latz, M. A. Osborne, and D. A. Howey. (2023c). “Bayesian model selection of lithium-ion battery models via Bayesian quadrature”. *IFAC-PapersOnLine*. 56(2): 10521–10526.
- Adrianakis, I. and P. G. Challenor. (2012). “The effect of the nugget on Gaussian process emulators for computer models”. *Computational Statistics and Data Analysis*. 56(12): 4215–4228.
- Akella, R. T., K. Azizzadenesheli, M. Ghavamzadeh, A. Anandkumar, and Y. Yue. (2021). “Deep Bayesian quadrature policy optimization”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. No. 8. 6600–6608.
- Alpay, D. and T. M. Mills. (2003). “A family of Hilbert spaces which are not reproducing kernel Hilbert”. *Journal of Analysis and Applications*. 1(2): 107–111.
- Arcangéli, R., M. C. L. de Silanes, and J. J. Torrens. (2012). “Extension of sampling inequalities to Sobolev semi-norms of fractional order and derivative data”. *Numerische Mathematik*. 121(3): 587–608.
- Aronszajn, N. (1950). “Theory of reproducing kernels”. *Transactions of the American Mathematical Society*. 68(3): 337–404.
- Bach, F. (2017). “On the equivalence between kernel quadrature rules and random feature expansions”. *Journal of Machine Learning Research*. 18(19): 1–38.
- Bachoc, F. and R. Furrer. (2016). “On the smallest eigenvalues of covariance matrices of multivariate spatial processes”. *Stat.* 5(1): 102–107.
- Balandat, M., B. Karrer, D. Jiang, S. Daulton, B. Letham, A. G. Wilson, and E. Bakshy. (2020). “BoTorch: A framework for efficient Monte-Carlo Bayesian optimization”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 21524–21538.
- Barthelmé, S., P.-O. Amblard, N. Tremblay, and K. Usevich. (2023). “Gaussian process regression in the flat limit”. *The Annals of Statistics*. 51(6): 2471–2505.
- Belhadji, A. (2020). “Subspace Sampling Using Determinantal Point Processes”. *PhD thesis*. University of Lille.

- Belhadji, A. (2021). “An analysis of Ermakov-Zolotukhin quadrature using kernels”. In: *Advances in Neural Information Processing Systems*. Vol. 34. 27278–27289.
- Belhadji, A., R. Bardenet, and P. Chainais. (2019). “Kernel quadrature with DPPs”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 12927–12937.
- Belhadji, A., R. Bardenet, and P. Chainais. (2020). “Kernel interpolation with continuous volume sampling”. In: *37th International Conference on Machine Learning*. Vol. 119. *Proceedings of Machine Learning Research*. 725–735.
- Berlinet, A. and C. Thomas-Agnan. (2004). *Reproducing Kernel Hilbert Spaces in Probability and Statistics*. Springer.
- Bezhaev, A. Y. (1991). “Cubature formulae on scattered meshes”. *Russian Journal of Numerical Analysis and Mathematical Modelling*. 6(2): 95–106.
- Bogachev, V. I. (1998). *Gaussian Measures*. American Mathematical Society.
- Briol, F.-X., A. Gessner, T. Karvonen, and M. Mahsereci. (2025). “A dictionary of closed-form kernel mean embeddings”. In: *1st International Conference on Probabilistic Numerics*. Vol. 271. *Proceedings of Machine Learning Research*. 84–94.
- Briol, F.-X., C. J. Oates, J. Cockayne, W. Y. Chen, and M. Girolami. (2017). “On the sampling problem for kernel quadrature”. In: *34th International Conference on Machine Learning*. Vol. 70. *Proceedings of Machine Learning Research*. 586–595.
- Briol, F.-X., C. J. Oates, M. Girolami, and M. A. Osborne. (2015). “Frank-Wolfe Bayesian quadrature: Probabilistic integration with theoretical guarantees”. In: *Advances in Neural Information Processing Systems*. Vol. 25. 1162–1170.
- Briol, F.-X., C. J. Oates, M. Girolami, M. A. Osborne, and D. Sejdinovic. (2019). “Probabilistic integration: A role in statistical computation? (with discussion and rejoinder)”. *Statistical Science*. 34(1): 1–22.
- Brouillat, J., C. Bouville, B. Loos, C. Hansen, and K. Bouatouch. (2009). “A Bayesian Monte Carlo approach to global illumination”. *Computer Graphics Forum*. 28(8): 2315–2329.

- Bui, T. D., J. Yan, and R. E. Turner. (2017). “A unifying framework for Gaussian process pseudo-point approximations using power expectation propagation”. *Journal of Machine Learning Research*. 18(104): 1–72.
- Caffisch, R. E. (1998). “Monte Carlo and quasi-Monte Carlo methods”. *Acta Numerica*. 7: 1–49.
- Cai, X., C. T. Lam, and J. Scarlett. (2023). “On average-case error bounds for kernel-based Bayesian quadrature”. *arXiv:2202.10615v2*.
- Cai, X. and J. Scarlett. (2024). “Kernelized normalizing constant estimation: Bridging Bayesian quadrature and Bayesian optimization”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 38. No. 10. 11150–11158.
- Chai, H. and R. Garnett. (2019). “Improving quadrature for constrained integrands”. In: *22nd International Conference on Artificial Intelligence and Statistics*. Vol. 89. *Proceedings of Machine Learning Research*. 2751–2759.
- Chai, H., J.-F. Ton, M. A. Osborne, and R. Garnett. (2019). “Automated model selection with Bayesian quadrature”. In: *36th International Conference on Machine Learning*. Vol. 97. *Proceedings of Machine Learning Research*. 931–940.
- Chatalic, A., N. Schreuder, E. De Vito, and L. Rosasco. (2025). “Efficient numerical integration in reproducing kernel Hilbert spaces via leverage scores sampling”. *Journal of Machine Learning Research*. 26(101): 1–55.
- Chen, Z., M. Naslidnyk, and F.-X. Briol. (2025). “Nested expectations with kernel quadrature”. In: *26th International Conference on Machine Learning*. Vol. 267. *Proceedings of Machine Learning Research*. 8760–8793.
- Chen, Z., M. Naslidnyk, A. Gretton, and F.-X. Briol. (2024). “Conditional Bayesian quadrature”. In: *40th Conference on Uncertainty in Artificial Intelligence*. Vol. 244. *Proceedings of Machine Learning Research*. 648–684.
- Choi, S.-C. T., F. J. Hickernell, Y. Ding, L. Jiang, L. A. J. Rugama, D. Li, J. Rathinavel, X. Tong, K. Zhang, Y. Zhang, and X. Zhou. (2020). “GAIL: Guaranteed Automatic Integration Library (version 2.3.1), MATLAB software”. URL: http://gailgithub.github.io/GAIL_Dev/.

- Cockayne, J., M. M. Graham, C. J. Oates, T. J. Sullivan, and O. Teymur. (2022). “Testing whether a learning procedure is calibrated”. *Journal of Machine Learning Research*. 23(203): 1–36.
- Cockayne, J., C. J. Oates, T. J. Sullivan, and M. Girolami. (2019). “Bayesian probabilistic numerical methods”. *SIAM Review*. 61(4): 756–789.
- Cook, T. D. and M. K. Clayton. (1998). “Sequential Bayesian quadrature”. *Tech. rep.* Department of Statistics, University of Wisconsin.
- Cools, R. (1997). “Constructing cubature formulae: the science behind the art”. *Acta Numerica*. 6: 1–54.
- Cressie, N. A. C. (1993). *Statistics for Spatial Data*. Wiley.
- Dang, C., P. Wei, M. G. R. Faes, M. A. Valdebenito, and M. Beer. (2021). “Parallel adaptive Bayesian quadrature for rare event estimation”. *Reliability Engineering & System Safety*. 225: 108621.
- Davis, P. J. and P. Rabinowitz. (1984). *Methods of Numerical Integration*. 2nd edition. Academic Press.
- Deisenroth, M. P., M. F. Huber, and U. D. Hanebeck. (2009). “Analytic moment-based Gaussian process filtering”. In: *26th International Conference on Machine Learning*. 225–232.
- Deisenroth, M. P., R. D. Turner, M. F. Huber, U. D. Hanebeck, and C. E. Rasmussen. (2012). “Robust filtering and smoothing with Gaussian processes”. *IEEE Transactions on Automatic Control*. 57(7): 1865–1871.
- DeVore, R., S. Foucart, G. Petrova, and P. Wojtaszczyk. (2019). “Computing a quantity of interest from observational data”. *Constructive Approximation*. 49(3): 461–508.
- Diaconis, P. (1988). “Bayesian numerical analysis”. In: *Statistical Decision Theory and Related Topics IV*. Vol. 1. Springer-Verlag New York. 163–175.
- Dick, J., F. Y. Kuo, and I. H. Sloan. (2013). “High-dimensional integration: The quasi-Monte Carlo way”. *Acta Numerica*. 22: 133–288.
- Diederichs, B. and A. Iske. (2019). “Improved estimates for condition numbers of radial basis function interpolation matrices”. *Journal of Approximation Theory*. 238: 38–51.

- Driscoll, M. F. (1973). “The reproducing kernel Hilbert space structure of the sample paths of a Gaussian process”. *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete*. 26(4): 309–316.
- Duc-Jacquet, M. (1973). “Approximation des fonctionnelles linéaires sur les espaces hilbertiens autoreproduisants”. *PhD thesis*. Université Joseph-Fourier-Grenoble I.
- Ehler, M., M. G., and C. J. Oates. (2019). “Optimal Monte Carlo integration on closed manifolds”. *Statistics and Computing*. 29(6): 1203–1214.
- Epperly, E. N. and E. Moreno. (2023). “Kernel quadrature with randomly pivoted Cholesky”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 65850–65868.
- Fasshauer, G. E. (2011). “Positive definite kernels: Past, present and future”. *Dolomites Research Notes on Approximation*. 4: 21–63.
- Filip, S., A. Javeed, and L. N. Trefethen. (2019). “Smooth random functions, random ODEs, and Gaussian processes”. *SIAM Review*. 61(1): 185–205.
- Fisher, M., C. Oates, C. Powell, and A. Teckentrup. (2020). “A locally adaptive Bayesian cubature method”. In: *23rd International Conference on Artificial Intelligence and Statistics*. Vol. 108. *Proceedings of Machine Learning Research*. 1265–1275.
- Fisher, R. A. (1936). “Design of experiments”. *British Medical Journal*. 1(3923): 554.
- Fitzsimons, J., K. Cutajar, M. Osborne, S. Roberts, and M. Filippone. (2017). “Bayesian inference of log determinants”. In: *33rd Conference on Uncertainty in Artificial Intelligence*.
- Fröhlich, C., A. Gessner, P. Hennig, B. Schölkopf, and G. Arvanitidis. (2021). “Bayesian quadrature on Riemannian data manifolds”. In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. *Proceedings of Machine Learning Research*. 3459–3468.
- Fuselier, E., T. Hangelbroek, F. J. Narcowich, J. D. Ward, and G. B. Wright. (2014). “Kernel based quadrature on spheres and other homogeneous spaces”. *Numerische Mathematik*. 127(1): 57–92.
- Galvis-Flores, C. A., A. Farooq, and S. Särkkä. (2025). “Provable quantum algorithm advantage for Gaussian process quadrature”. *Transactions on Machine Learning Research*.

- Garnett, R. (2023). *Bayesian Optimization*. Cambridge University Press.
- Gautschi, W. (2004). *Orthogonal Polynomials: Computation and Approximation. Numerical Mathematics and Scientific Computation*. Oxford University Press.
- Gavrilov, A. V. (1998). “On best quadrature formulas in the reproducing kernel Hilbert space”. *Sibirsky zhurnal vychislitelnoy matematiki*. 1(4): 313–320.
- Gavrilov, A. V. (2007). “On optimal quadrature formulas”. *Journal of Applied and Industrial Mathematics*. 1(2): 190–192.
- Gerstner, T. and M. Griebel. (2003). “Dimension-adaptive tensor-product quadrature”. *Computing*. 71: 65–87.
- Gessner, A., J. Gonzalez, and M. Mahsereci. (2020). “Active multi-information source Bayesian quadrature”. In: *35th Conference on Uncertainty in Artificial Intelligence*. Vol. 115. *Proceedings of Machine Learning Research*. 712–721.
- Glaubitz, J. and J. Reeger. (2023). “Towards stability results for global radial basis function based quadrature formulas”. *BIT Numerical Mathematics*. 63(6).
- GPY authors. (2012). “GPY: A Gaussian process framework in Python”. URL: <http://github.com/SheffieldML/GPY>.
- Gräf, M. (2013). “Efficient Algorithms for the Computation of Optimal Quadrature Points on Riemannian Manifolds”. *PhD thesis*. Chemnitz University of Technology.
- Gramacy, R. B. and H. K. H. Lee. (2012). “Cases for the nugget in modeling computer experiments”. *Statistics and Computing*. 22: 713–722.
- Gunter, T., M. A. Osborne, R. Garnett, P. Hennig, and S. J. Roberts. (2014). “Sampling for inference in probabilistic models with fast Bayesian quadrature”. In: *Advances in Neural Information Processing Systems*. Vol. 24. 2789–2797.
- Hamid, S., S. Schulze, M. A. Osborne, and S. J. Roberts. (2022). “Marginalising over stationary kernels with Bayesian quadrature”. In: *25th International Conference on Artificial Intelligence and Statistics*. Vol. 151. *Proceedings of Machine Learning Research*. 9776–9792.

- Hamid, S., X. Wan, M. Jørgensen, B. Ru, and M. Osborne. (2023). “Bayesian quadrature for neural ensemble search”. *arXiv:2303.08874v1*.
- Hamrick, J. B. and T. L. Griffiths. (2013). “Mental rotation as Bayesian quadrature”. In: *NIPS 2013 Workshop on Bayesian Optimization in Theory and Practice*.
- Hayakawa, S. (2023). “Random Convex Hulls and Kernel Quadrature”. *PhD thesis*. University of Oxford.
- Hayakawa, S., H. Oberhauser, and T. Lyons. (2022). “Positively weighted kernel quadrature via subsampling”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 6886–6900.
- Hayakawa, S., H. Oberhauser, and T. Lyons. (2023a). “Hypercontractivity meets random convex hulls: analysis of randomized multivariate cubatures”. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*. 479(2273): 20220725.
- Hayakawa, S., H. Oberhauser, and T. Lyons. (2023b). “Sampling-based Nyström approximation and kernel quadrature”. In: *40th International Conference on Machine Learning*. 12678–12699.
- Henderson, I. (2024). “Sobolev regularity of Gaussian random fields”. *Journal of Functional Analysis*. 286(3): 110241.
- Hennig, P., M. A. Osborne, and M. Girolami. (2015). “Probabilistic numerics and uncertainty in computations”. *Proceedings of the Royal Society of London A: Mathematical, Physical and Engineering Sciences*. 471(2179).
- Hennig, P., M. A. Osborne, and H. P. Kersting. (2022). *Probabilistic Numerics: Computation as Machine Learning*. Cambridge University Press.
- Hong, F., P. Wei, and M. Beer. (2024). “Parallelization of adaptive Bayesian cubature using multimodal optimization algorithms”. *Engineering Computations*. 41(2): 413–437.
- Huszár, F. and D. Duvenaud. (2012). “Optimally-weighted herding is Bayesian quadrature”. In: *28th Conference on Uncertainty in Artificial Intelligence*. 377–385.
- Iwazaki, S., I. Y., and I. Takeuchi. (2021). “Bayesian quadrature optimization for probability threshold robustness measure”. *Neural Computation*. 33(12): 3413–3466.

- Jagadeeswaran, R. (2019). “Fast automatic Bayesian cubature using matching kernels and designs”. *PhD thesis*. Illinois Institute of Technology.
- Jagadeeswaran, R. and F. J. Hickernell. (2019). “Fast automatic Bayesian cubature using lattice sampling”. *Statistics and Computing*. 29(6): 1215–1229.
- Jagadeeswaran, R. and F. J. Hickernell. (2022). “Fast automatic Bayesian cubature using Sobol’ sampling”. In: *Advances in Modeling and Simulation: Festschrift for Pierre L’Ecuyer*. Springer. 301–318.
- Jiang, S., H. Chai, J. Gonzalez, and R. Garnett. (2019). “Efficient non-myopic Bayesian optimization and quadrature”. *arXiv:1909.04568v3*.
- Jones, D. R. (2001). “A taxonomy of global optimization methods based on response surfaces”. *Journal of Global Optimization*. 21: 345–383.
- Kaarnioja, V., I. Klebanov, C. Schillings, and Y. Suzuki. (2025). “Lattice rules meet kernel cubature”. In: *Monte Carlo and Quasi-Monte Carlo Methods 2024*.
- Kanagawa, M. and P. Hennig. (2019). “Convergence guarantees for adaptive Bayesian quadrature methods”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 6237–6248.
- Kanagawa, M., P. Hennig, D. Sejdinovic, and B. K. Sriperumbudur. (2018). “Gaussian processes and kernel methods: A review on connections and equivalences”. *arXiv:1807.02582v1*.
- Kanagawa, M., B. K. Sriperumbudur, and K. Fukumizu. (2016). “Convergence guarantees for kernel-based quadrature rules in misspecified settings”. In: *Advances in Neural Information Processing Systems*. Vol. 29. 3288–3296.
- Kanagawa, M., B. K. Sriperumbudur, and K. Fukumizu. (2020). “Convergence analysis of deterministic kernel-based quadrature rules in misspecified settings”. *Foundations of Computational Mathematics*. 20(1): 155–194.
- Kang, S. S., F.-X. Briol, T. Karvonen, and Z. Chen. (2025). “BayesSum: Bayesian quadrature in discrete spaces”. *arXiv:2512.16105v1*.
- Karvonen, T. (2019). “Kernel-Based and Bayesian Methods for Numerical Integration”. *PhD thesis*. Aalto University.

- Karvonen, T., M. Kanagawa, and S. Särkkä. (2019a). “On the positivity and magnitudes of Bayesian quadrature weights”. *Statistics and Computing*. 29(6): 1317–1333.
- Karvonen, T. and C. J. Oates. (2023). “Maximum likelihood estimation in Gaussian process regression is ill-posed”. *Journal of Machine Learning Research*.
- Karvonen, T., C. J. Oates, and M. Girolami. (2021). “Integration in reproducing kernel Hilbert spaces of Gaussian kernels”. *Mathematics of Computation*. 90(331): 2209–2233.
- Karvonen, T., C. J. Oates, and S. Särkkä. (2018). “A Bayes–Sard cubature method”. In: *Advances in Neural Information Processing Systems*. Vol. 31. 5882–5893.
- Karvonen, T., G. Santin, and T. Wenzel. (2025). “General superconvergence for kernel-based approximation”. *arXiv:2505.11435v1*.
- Karvonen, T. and S. Särkkä. (2017). “Classical quadrature rules via Gaussian processes”. In: *27th IEEE International Workshop on Machine Learning for Signal Processing*.
- Karvonen, T. and S. Särkkä. (2018). “Fully symmetric kernel quadrature”. *SIAM Journal on Scientific Computing*. 40(2): A697–A720.
- Karvonen, T. and S. Särkkä. (2019). “Gaussian kernel quadrature at scaled Gauss–Hermite nodes”. *BIT Numerical Mathematics*. 59(4): 877–902.
- Karvonen, T. and S. Särkkä. (2020). “Worst-case optimal approximation with increasingly flat Gaussian kernels”. *Advances in Computational Mathematics*. 46(2).
- Karvonen, T., S. Särkkä, and C. J. Oates. (2019b). “Symmetry exploits for Bayesian cubature methods”. *Statistics and Computing*. 29(6): 1231–1248.
- Karvonen, T. and Y. Suzuki. (2025). “Approximation in Hilbert spaces of the Gaussian and related analytic kernels”. *IMA Journal of Numerical analysis*.
- Karvonen, T., G. Wynne, F. Tronarp, C. J. Oates, and S. Särkkä. (2020). “Maximum likelihood estimation and uncertainty quantification for Gaussian process approximation of deterministic functions”. *SIAM/ASA Journal on Uncertainty Quantification*. 8(3): 926–958.

- Kennedy, M. (1998). “Bayesian quadrature with non-normal approximating functions”. *Statistics and Computing*. 8(4): 365–375.
- Kennedy, M. (2000). “A Bayesian approach to computing posterior distribution and quantile functions”. *Journal of Statistical Planning and Inference*. 83(1): 183–201.
- Kennedy, M. C. and A. O’Hagan. (1996). “Iterative rescaling for Bayesian quadrature”. In: *Bayesian Statistics 5: Proceedings of the Fifth Valencia International Meeting, June 5–9, 1994*. Clarendon Press. 639–646.
- Kersting, H. and P. Hennig. (2016). “Active uncertainty calibration in Bayesian ODE solvers”. In: *32nd Conference on Uncertainty in Artificial Intelligence*. 309–318.
- Khatri, C. G. and R. Mukerjee. (1987). “Characterization of bijective and bimeasurable transformations for bivariate normal variates”. *Sankhyā: The Indian Journal of Statistics, Series A*. 49(3): 405–411.
- Kimeldorf, G. S. and G. Wahba. (1970). “A correspondence between Bayesian estimation on stochastic processes and smoothing by splines”. *The Annals of Mathematical Statistics*. 41(2): 495–502.
- Krieg, D. and M. Sonnleitner. (2024). “Random points are optimal for the approximation of Sobolev functions”. *IMA Journal of Numerical Analysis*. 44(3): 1346–1371.
- Kumar, A., P. B. Nair, A. J. Keane, and S. Shahpar. (2008). “Robust design using Bayesian Monte Carlo”. *International Journal for Numerical Methods in Engineering*. 73(11): 1497–1517.
- Kuo, F. Y., I. H. Sloan, and H. Woźniakowski. (2017). “Multivariate integration for analytic functions with Gaussian kernels”. *Mathematics of Computation*. 86: 829–853.
- Kuo, F. Y. and H. Woźniakowski. (2012). “Gauss–Hermite quadratures for functions from Hilbert spaces with Gaussian reproducing kernels”. *BIT Numerical Mathematics*. 52(2): 425–436.
- Larkin, F. M. (1970). “Optimal approximation in Hilbert spaces with reproducing kernel functions”. *Mathematics of Computation*. 24(112): 911–921.
- Larkin, F. M. (1972). “Gaussian measure in Hilbert space and applications in numerical analysis”. *Rocky Mountain Journal of Mathematics*. 2(3): 379–422.

- Larkin, F. M. (1974). “Probabilistic error estimates in spline interpolation and quadrature”. In: *Information Processing 74: Proceedings of IFIP Congress 74*. North-Holland. 605–609.
- Li, C., G. Clartó, M. Jørgensen, and L. Acerbi. (2025). “Fast post-process Bayesian inference with variational sparse Bayesian quadrature”. *Statistics and Computing*. 35(167).
- Li, K., D. Giles, T. Karvonen, S. Guillas, and F.-X. Briol. (2023). “Multilevel Bayesian quadrature”. In: *26th International Conference on Artificial Intelligence and Statistics*. Vol. 206. *Proceedings of Machine Learning Research*. 1845–1868.
- Liu, H., Y.-S. Ong, X. Shen, and J. Cai. (2020). “When Gaussian process meets big data: A review of scalable GPs”. *IEEE Transactions on Neural Networks and Learning Systems*. 31(11): 4405–4423.
- Llorente Fernández, F., L. Martino, V. Elvira, D. Delgado, and J. López-Santiago. (2020). “Adaptive quadrature schemes for Bayesian inference via active learning”. *IEEE Access*. 8: 208462–208483.
- Ma, Y., R. Garnett, and J. Schneider. (2014). “Active area search via Bayesian quadrature”. In: *17th International Conference on Artificial Intelligence and Statistics*. Vol. 33. *Proceedings of Machine Learning Research*. 595–603.
- Marques, R., C. Bouville, M. Ribardière, L. P. Santos, and K. Bouatouch. (2013). “A spherical Gaussian framework for Bayesian Monte Carlo rendering of glossy surfaces”. *IEEE Transactions on Visualization and Computer Graphics*. 19(10): 1619–1932.
- Marques, R., C. Bouville, L. P. Santos, and K. Bouatouch. (2015). *Efficient Quadrature Rules for Illumination Integrals: From Quasi Monte Carlo to Bayesian Monte Carlo*. Morgan & Claypool Publishers.
- Mase, S. (1977). “Some theorems on normality-preserving transformations”. *Sankhyā: The Indian Journal of Statistics, Series A*. 39(2): 186–190.
- McKay, M. D., R. J. Beckman, and W. J. Conover. (1979). “A comparison of three methods for selecting values of input variables in the analysis of output from a computer code”. *Technometrics*. 21(2): 239–245.

- Minh, H. Q. (2010). “Some properties of Gaussian reproducing kernel Hilbert spaces and their implications for function approximation and learning theory”. *Constructive Approximation*. 32(2): 307–338.
- Minka, T. (2000). “Deriving quadrature rules from Gaussian processes”. *Tech. rep.* Statistics Department, Carnegie Mellon University.
- Mockus, J. (1989). *Bayesian Approach to Global Optimization*. Springer.
- Muandet, K., K. Fukumizu, B. Sriperumbudur, and B. Schölkopf. (2017). “Kernel mean embedding of distributions: A review and beyond”. *Foundations and Trends® in Machine Learning*. 10(1–2): 1–141.
- Müller, S. and R. Schaback. (2009). “A Newton basis for kernel spaces”. *Journal of Approximation Theory*. 161(2): 645–655.
- Narcowich, F. J., J. D. Ward, and H. Wendland. (2006). “Sobolev error estimates and a Bernstein inequality for scattered data interpolation via radial basis functions”. *Constructive Approximation*. 24(2): 175–186.
- Naslidnyk, M., J. Gonzalez, and M. Mahsereci. (2021). “Invariant priors for Bayesian quadrature”. In: *Your Model is Wrong: Robustness and misspecification in probabilistic modeling Workshop, NeurIPS*.
- Naslidnyk, M., M. Kanagawa, T. Karvonen, and M. Mahsereci. (2025). “Comparing scale parameter estimators for Gaussian process interpolation with the Brownian motion prior: Leave-one-out cross validation and maximum likelihood”. *SIAM/ASA Journal on Uncertainty Quantification*. 13(2): 679–717.
- Neil, R. M. (1999). “Regression and classification using Gaussian process priors”. *Bayesian Statistics*. 6: 475–501.
- Neumann, J. von and O. Morgenstern. (1944). *Theory of Games and Economic Behavior*. Princeton University Press.
- Nguyen, T., S. Gupta, H. Ha, and S. Venkatesh. (2020). “Distributionally robust Bayesian quadrature optimization”. In: *23rd International Conference on Artificial Intelligence and Statistics*. Vol. 108. *Proceedings of Machine Learning Research*. 1921–1931.
- Nikolsky, S. M. (1950). “Concerning estimation for approximate quadrature formulas”. *Uspekhi matematicheskikh nauk*. 5(2): 165–177.
- Nobile, F., R. Tempone, and S. Wolfers. (2018). “Sparse approximation of multilinear problems with applications to kernel-based methods in UQ”. *Numerische Mathematik*. 139(1): 247–280.

- Novak, E. and H. Triebel. (2006). “Function spaces in Lipschitz domains and optimal rates of convergence for sampling”. *Constructive Approximation*. 23: 325–350.
- Novak, E. and H. Woźniakowski. (2008). *Tractability of Multivariate Problems. Volume I: Linear Information*. European Mathematical Society.
- Novak, E. and H. Woźniakowski. (2010). *Tractability of Multivariate Problems. Volume II: Standard Information for Functionals*. European Mathematical Society.
- O’Hagan, A. (1988). “Bayesian quadrature”. *Warwick Statistics Research Report 159*. Department of Statistics, University of Warwick.
- O’Hagan, A. (1991). “Bayes–Hermite quadrature”. *Journal of Statistical Planning and Inference*. 29(3): 245–260.
- O’Hagan, A. (1992). “Some Bayesian numerical analysis”. *Bayesian Statistics*. 4: 345–363.
- Oates, C. J., M. Girolami, and N. Chopin. (2017a). “Control functionals for Monte Carlo integration”. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*. 79(3): 695–718.
- Oates, C. J., S. Niederer, A. Lee, F.-X. Briol, and M. Girolami. (2017b). “Probabilistic models for integration error in the assessment of functional cardiac models”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 110–118.
- Oates, C. J. and T. J. Sullivan. (2019). “A modern retrospective on probabilistic numerics”. *Statistics and Computing*. 29(6): 1335–1351.
- Oettershagen, J. (2017). “Construction of Optimal Cubature Algorithms with Applications to Econometrics and Uncertainty Quantification”. *PhD thesis*. University of Bonn.
- Osborne, M. A., D. Duvenaud, R. Garnett, C. E. Rasmussen, S. J. Roberts, and Z. Ghahramani. (2012a). “Active learning of model evidence using Bayesian quadrature”. In: *Advances in Neural Information Processing Systems*. Vol. 25. 46–54.
- Osborne, M. A., R. Garnett, S. J. Roberts, C. Hart, S. Aigrain, and N. P. Gibson. (2012b). “Bayesian quadrature for ratios”. In: *15th International Conference on Artificial Intelligence and Statistics*. Vol. 22. *JMLR Workshop and Conference Proceedings*. 832–840.

- Paley, A., M. Mahsereci, and N. D. Lawrence. (2023). “Emukit: A Python toolkit for decision making under uncertainty”. *Proceedings of the 22nd Python in Science Conference*: 68–75.
- Paley, A., M. Pullin, M. Mahsereci, N. Lawrence, and J. Gonzalez. (2019). “Emulation of physical processes with Emukit”. In: *2nd Workshop on Machine Learning and the Physical Sciences, NeurIPS*.
- Paul, S., K. Chatzilygeroudis, K. Ciosek, J.-B. Mouret, M. A. Osborne, and S. Whiteson. (2018). “Alternating optimisation and quadrature for robust control”. In: *32nd AAAI Conference on Artificial Intelligence*.
- Paulsen, V. I. and M. Raghupathi. (2016). *An Introduction to the Theory of Reproducing Kernel Hilbert Spaces*. Cambridge University Press.
- Platte, R. B., L. N. Trefethen, and A. B. J. Kuijlaars. (2011). “Impossibility of fast stable approximation of analytic functions from equispaced samples”. *SIAM Review*. 53(2): 308–318.
- Press, W. H., S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery. (2007). *Numerical Recipes: The Art of Scientific Computing*. 3rd edition. Cambridge University Press.
- Pronzato, L. (2023). “Performance analysis of greedy algorithms for minimising a maximum mean discrepancy”. *Statistics and Computing*. 33(14).
- Pronzato, L. and A. Zhigljavsky. (2020). “Bayesian quadrature, energy minimization, and space-filling design”. *SIAM/ASA Journal on Uncertainty Quantification*. 8(3): 959–1011.
- Prüher, J., T. Karvonen, C. J. Oates, and O. Straka. (2021). “Improved calibration of numerical integration error in sigma-point filters”. *IEEE Transactions on Automatic Control*. 66(3): 1286–1292.
- Prüher, J. and S. Särkkä. (2016). “On the use of gradient information in Gaussian process quadratures”. In: *26th IEEE International Workshop on Machine Learning for Signal Processing*.
- Prüher, J. and M. Šimandl. (2016). “Bayesian quadrature variance in sigma-point filtering”. In: *International Conference on Informatics in Control, Automation and Robotics (revised selected papers)*. Vol. 12. Springer International Publishing. 355–370.

- Prüher, J. and O. Straka. (2018). “Gaussian process quadrature moment transform”. *IEEE Transactions on Automatic Control*. 63(9): 2844–2854.
- Prüher, J., F. Tronarp, T. Karvonen, S. Särkkä, and O. Straka. (2017). “Student- t process quadratures for filtering of non-linear systems with heavy-tailed noise”. In: *20th International Conference on Information Fusion*.
- Punzi, A., A. Sommariva, and M. Vianello. (2008). “Meshless cubature over the disk using thin-plate splines”. *Journal of Computational and Applied Mathematics*. 221(2): 430–436.
- Quiñonero-Candela, J. and C. E. Rasmussen. (2005). “A unifying view of sparse approximate Gaussian process regression”. *Journal of Machine Learning Research*. 6(65): 1939–1959.
- Ranjan, P., R. Haynes, and R. Karsten. (2011). “A computationally stable approach to Gaussian process interpolation of deterministic computer simulation data”. *Technometrics*. 53(4): 366–378.
- Rasmussen, C. E. and Z. Ghahramani. (2002). “Bayesian Monte Carlo”. In: *Advances in Neural Information Processing Systems*. Vol. 15. 505–512.
- Rasmussen, C. E. and C. K. I. Williams. (2006). *Gaussian Processes for Machine Learning*. MIT Press.
- Reeger, J. A. and B. Fornberg. (2015). “Numerical quadrature over the surface of a sphere”. *Studies in Applied Mathematics*. 137(2): 174–188.
- Reeger, J. A. and B. Fornberg. (2018). “Numerical quadrature over smooth surfaces with boundaries”. *Journal of Computational Physics*. 355: 176–190.
- Reeger, J. A., B. Fornberg, and M. L. Watts. (2016). “Numerical quadrature over smooth, closed surfaces”. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*. 472(2194): 20160401.
- Richter, N. (1970). “Properties of minimal integration rules”. *SIAM Journal on Numerical Analysis*. 7(1): 67–79.
- Richter-Dyn, N. (1971a). “Minimal interpolation and approximation in Hilbert spaces”. *SIAM Journal on Numerical Analysis*. 8(3): 583–597.

- Richter-Dyn, N. (1971b). “Properties of minimal integration rules. II”. *SIAM Journal on Numerical Analysis*. 8(3): 497–508.
- Rieger, C. and H. Wendland. (2017). “Sampling inequalities for sparse grids”. *Numerische Mathematik*. 136(2): 439–466.
- Rieger, C. and H. Wendland. (2020). “Sampling inequalities for anisotropic tensor product grids”. *IMA Journal on Numerical Analysis*. 40(1): 285–321.
- Rieger, C. and B. Zwicknagl. (2010). “Sampling inequalities for infinitely smooth functions, with applications to interpolation and machine learning”. *Advances in Computational Mathematics*. 32: 103–129.
- Rieger, C. and B. Zwicknagl. (2014). “Improved exponential convergence rates by oversampling near the boundary”. *Constructive Approximation*. 39(2): 323–341.
- Ritter, K. (2000). *Average-Case Analysis of Numerical Problems*. Springer.
- Roustant, O., N. Lüthen, and F. Gamboa. (2024). “Spectral decomposition of $H^1(\mu)$ and Poincaré inequality on a compact interval — Application to kernel quadrature”. *Journal of Approximation Theory*. 301(106041).
- Sadeghi, J., R. Mueller, and J. Redford. (2022). “Bayesian quadrature for probability threshold robustness of partially undefined functions”. *arXiv:2210.02168v1*.
- Santin, G., T. Karvonen, and B. Haasdonk. (2022). “Sampling based approximation of linear functionals in reproducing kernel Hilbert spaces”. *BIT Numerical Mathematics*. 62: 279–310.
- Santner, T. J., B. J. Williams, and W. I. Notz. (2003). *The Design and Analysis of Computer Experiments*. Springer.
- Sard, A. (1949). “Best approximate integration formulas; best approximation formulas”. *American Journal of Mathematics*. 71(1): 80–91.
- Särkkä, S. (2011). “Linear operators and stochastic partial differential equations in Gaussian process regression”. In: *International Conference on Artificial Neural Networks*. 151–158.
- Särkkä, S., J. Hartikainen, L. Svensson, and F. Sandblom. (2014). “Gaussian process quadratures in nonlinear sigma-point filtering and smoothing”. In: *17th International Conference on Information Fusion*.

- Särkkä, S., J. Hartikainen, L. Svensson, and F. Sandblom. (2016). “On the relation between Gaussian process quadratures and sigma-point methods”. *Journal of Advances in Information Fusion*. 11(1): 31–46.
- Schaback, R. (1995). “Error estimates and condition numbers for radial basis function interpolation”. *Advances in Computational Mathematics*. 3(3): 251–264.
- Schaback, R. (1999). “Improved error bounds for scattered data interpolation by radial basis functions”. *Mathematics of Computation*. 68(225): 201–216.
- Schaback, R. (2018). “Superconvergence of kernel-based interpolation”. *Journal of Approximation Theory*. 235: 1–19.
- Scheuerer, M. (2010). “Regularity of the sample paths of a general second order random field”. *Stochastic Processes and their Applications*. 120(10): 1879–1897.
- Scheuerer, M., R. Schaback, and M. Schlather. (2013). “Interpolation of spatial data – A stochastic or a deterministic problem?” *European Journal of Applied Mathematics*. 24(4): 601–629.
- Schober, M., D. K. Duvenaud, and P. Hennig. (2014). “Probabilistic ODE solvers with Runge-Kutta means”. In: *Advances in Neural Information Processing Systems*. Vol. 27. 739–747.
- Siraj-ul-Islam, I. Aziz, and W. Khan. (2012). “Numerical integration of multi-dimensional highly oscillatory, gentle oscillatory and non-oscillatory integrands based on wavelets and radial basis functions”. *Engineering Analysis with Boundary Elements*. 36(8): 1284–1295.
- Sloan, I. H. and V. Kaarnioja. (2025). “Doubling the rate: Improved error bounds for orthogonal projection with application to interpolation”. *BIT Numerical Mathematics*. 65(10).
- Snell, J. C. and T. L. Griffiths. (2025). “Conformal prediction as Bayesian quadrature”. In: *42nd International Conference on Machine Learning*. 56068–56084.
- Snelson, E., Z. Ghahramani, and C. Rasmussen. (2003). “Warped Gaussian processes”. In: *Advances in Neural Information Processing Systems*. Vol. 16.

- Solak, E., R. Murray-Smith, W. E. Leithead, D. J. Leith, and C. Rasmussen. (2002). “Derivative observations in Gaussian process models of dynamic systems”. In: *Advances in Neural Information Processing Systems*. Vol. 15. 1057–1064.
- Sommariva, A. and M. Vianello. (2006a). “Meshless cubature by Green’s formula”. *Applied Mathematics and Computation*. 183(2): 1098–1107.
- Sommariva, A. and M. Vianello. (2006b). “Numerical cubature on scattered data by radial basis functions”. *Computing*. 76(3–4): 295–310.
- Sommariva, A. and M. Vianello. (2021). “RBF moment computation and meshless cubature on general polygonal regions”. *Applied Mathematics and Computation*. 409(15): 126375.
- Sommariva, A. and R. Womersley. (2005). “Integration by RBF over the sphere”. *Tech. rep.* University of New South Wales.
- Sorokin, A. G., P. Robbe, G. Geraci, M. S. Eldred, and F. J. Hickernell. (2025). “Fast Bayesian multilevel quasi-Monte Carlo”. *arXiv:2510.24604v1*.
- South, L. F., T. Karvonen, C. Nemeth, and C. J. Oates. (2022). “Semi-exact control functionals from Sard’s method”. *Biometrika*. 109(2): 351–367.
- Stein, M. L. (1999). *Interpolation of Spatial Data: Some Theory for Kriging*. Springer.
- Steinwart, I. (2019). “Convergence types and rates in generic Karhunen-Loève expansions with applications to sample path properties”. *Potential Analysis*. 51: 361–395.
- Steinwart, I. and A. Christmann. (2008). *Support Vector Machines*. Springer.
- Suldin, A. V. (1959). “Wiener measure and its applications to approximation methods. I”. *Izvestiya vysshikh uchebnykh zavedeniy. Matematika*. (6): 145–158.
- Suldin, A. V. (1960). “Wiener measure and its applications to approximation methods. II”. *Izvestiya vysshikh uchebnykh zavedeniy. Matematika*. (5): 165–179.
- Tanaka, K. (2021). “Kernel quadrature by applying a point-wise gradient descent method to discrete energies”. *arXiv:2102.10887v2*.

- Tang, Y. (2024). “A note on Monte Carlo integration in high dimensions”. *The American Statistician*. 78(3): 290–296.
- Teckentrup, A. L. (2020). “Convergence of Gaussian process regression with estimated hyper-parameters and applications in Bayesian inverse problems”. *SIAM/ASA Journal on Uncertainty Quantification*. 8(4): 1310–1337.
- Toscano-Palmerin, S. and P. I. Frazier. (2022). “Bayesian optimization with expensive integrands”. *SIAM Journal on Optimization*. 32(2): 417–444.
- Traub, J. F., G. W. Wasilkowski, and H. Woźniakowski. (1988). *Information-Based Complexity*. Academic Press.
- Trefethen, L. N. (2008). “Is Gauss quadrature better than Clenshaw–Curtis?” *SIAM Review*. 50(1): 67–87.
- Tronarp, F., T. Karvonen, and S. Särkkä. (2018). “Mixture representation of the Matérn class with applications in state space approximations and Bayesian quadrature”. In: *28th IEEE International Workshop on Machine Learning for Signal Processing*.
- Tsuji, K. and K. Tanaka. (2026). “Sparse solutions of the kernel herding algorithm by improved gradient approximation”. *Japan Journal of Industrial and Applied Mathematics*. 43(22).
- Tsuji, K., K. Tanaka, and S. Pokutta. (2022). “Pairwise conditional gradients without swap steps and sparser kernel herding”. In: *39th International Conference on Machine Learning*. 21864–21883.
- Tuo, R., Y. Wang, and C. F. J. Wu. (2020). “On the improved rates of convergence for Matérn-type kernel ridge regression with application to calibration of computer models”. *SIAM/ASA Journal on Uncertainty Quantification*. 8(4): 1522–1547.
- Wagstaff, E., S. Hamid, and M. Osborne. (2018). “Batch selection for parallelisation of Bayesian quadrature”. *arXiv:1812.01553v1*.
- Wahba, G. (1990). *Spline Models for Observational Data*. Society for Industrial and Applied Mathematics.
- Wang, W. (2021). “On the inference of applying Gaussian process modeling to a deterministic function”. *Electronic Journal of Statistics*. 15(2): 5014–5066.

- Warren, H. and F. Ramos. (2024). “Fast Fourier Bayesian quadrature”. In: *27th International Conference on Artificial Intelligence and Statistics*. Vol. 238. *Proceedings of Machine Learning Research*. 4555–4563.
- Warren, H., R. Oliveira, and F. Ramos. (2022). “Generalized Bayesian quadrature with spectral kernels”. In: *38th Conference on Uncertainty in Artificial Intelligence*. Vol. 180. *Proceedings of Machine Learning Research*. 2085–2095.
- Wei, P., X. Zhang, and M. Beer. (2020). “Adaptive experiment design for probabilistic integration”. *Computer Methods in Applied Mechanics and Engineering*. 365: 113035.
- Weiland, T., T. Karvonen, and P. Hennig. (2026). “Hierarchical Bayesian quadrature”. In: *42nd Conference on Uncertainty in Artificial Intelligence. Proceedings of Machine Learning Research*.
- Wendland, H. (2005). *Scattered Data Approximation*. Cambridge University Press.
- Wenger, J., N. Krämer, M. Pförtner, J. Schmidt, N. Bosch, N. Effenberg, J. Zenn, A. Gessner, T. Karvonen, F.-X. Briol, M. Mahsereci, and P. Hennig. (2021). “ProbNum: Probabilistic numerics in Python”. *arXiv:2112.02100v1*.
- Wenzel, T., G. Santin, and B. Haasdonk. (2021). “A novel class of stabilized greedy kernel approximation algorithms: Convergence, stability and uniform point distribution”. *Journal of Approximation Theory*. 262: 105508.
- Wenzel, T., G. Santin, and B. Haasdonk. (2022). “Analysis of target data-dependent greedy kernel algorithms: Convergence rates for f -, $f \cdot P$ - and f/P -greedy”. *Constructive Approximation*.
- Wenzel, T., G. Santin, and B. Haasdonk. (2025). “Stability of convergence rates: Kernel interpolation on non-Lipschitz domains”. *IMA Journal of Numerical Analysis*. 45(2): 777–798.
- Wilson, A. and H. Nickisch. (2015). “Kernel interpolation for scalable structured Gaussian processes (KISS-GP)”. In: *Proceedings of the 32nd International Conference on Machine Learning*. Vol. 37. *Proceedings of Machine Learning Research*. 1775–1784.
- Wright, J. D. M. (1973). “All operators on a Hilbert space are bounded”. *Bulletin of the American Mathematical Society*. 79(6): 1247–1250.

- Wynne, G., F.-X. Briol, and M. Girolami. (2021). “Convergence guarantees for Gaussian process means with misspecified likelihoods and smoothness”. *Journal of Machine Learning Research*. 22(123): 1–40.
- Xi, X., F.-X. Briol, and M. Girolami. (2018). “Bayesian quadrature for multiple related integrals”. In: *35th International Conference on Machine Learning*. Vol. 80. *Proceedings of Machine Learning Research*. 5373–5382.
- Yarotsky, D. (2013). “Examples of inconsistency in optimization by expected improvement”. *Journal of Global Optimization*. 56: 1773–1790.
- Zhou, T. and Y. Peng. (2020). “Adaptive Bayesian quadrature based statistical moments estimation for structural reliability analysis”. *Reliability Engineering & System Safety*. 198: 106902.
- Zhu, H., X. Liu, R. Kang, Z. Shen, S. Flaxman, and F.-X. Briol. (2020). “Bayesian probabilistic numerical integration with tree-based models”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 5837–5849.